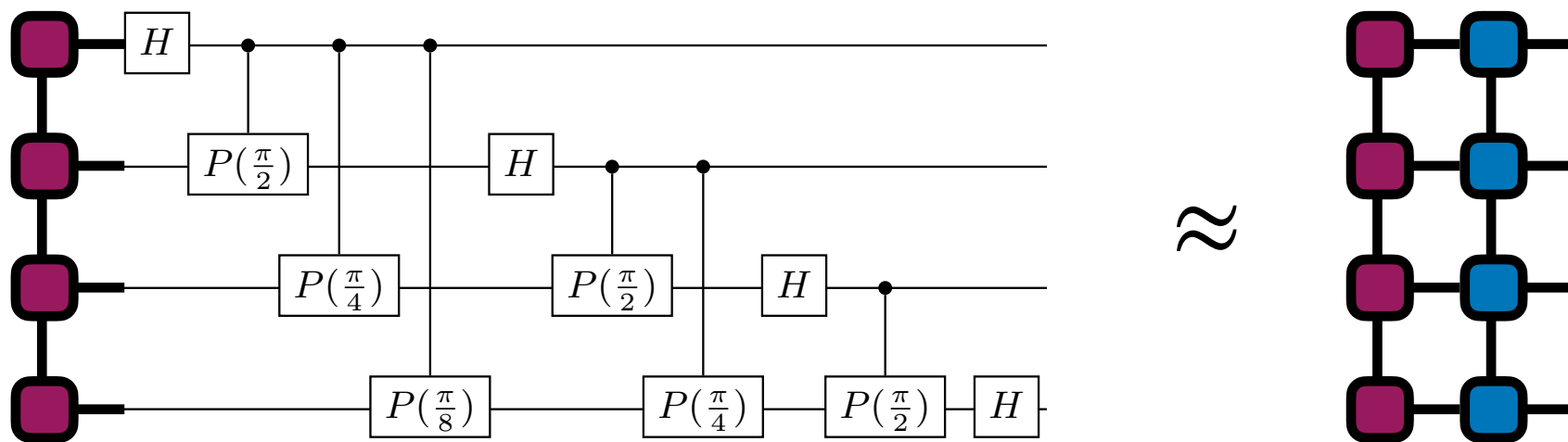
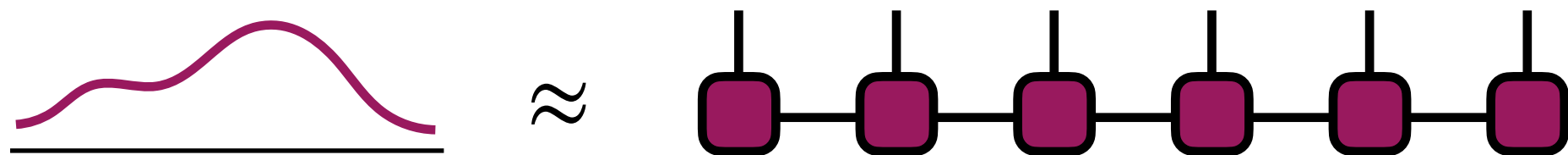


New Directions for Tensor Networks: Function Representations & Machine Learning



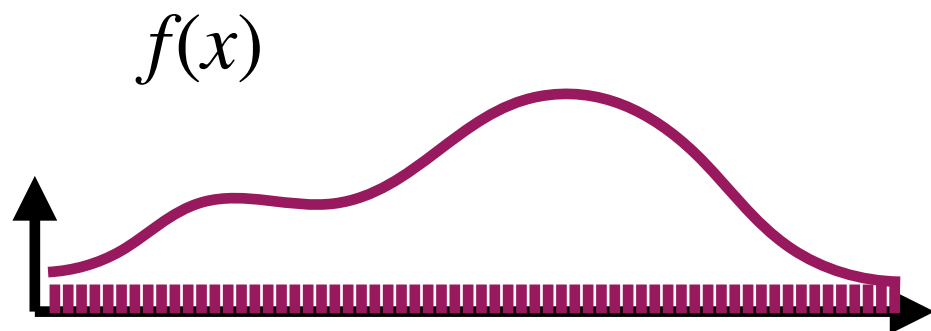
Motivation

Traditional view of tensor networks =
many-body quantum wavefunctions

But really just linear algebra...in high dimensions

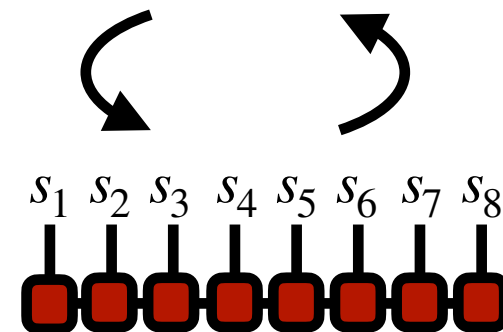
Two surprising directions for tensor networks

Continuum functions



Machine Learning

$$f(s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8)$$



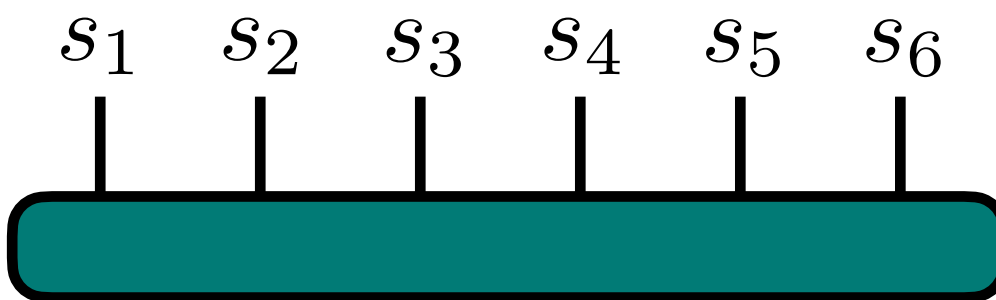
This Talk

- Encoding **Continuum Functions** as Tensor Networks and **Algorithms** for Tensorized Functions
- **Machine Learning** with Tensor Networks
 - Active Learning of Data (**Tensor Cross**)
 - Tensor **Recursive Sketching** Algorithm

Representing Functions as Tensor Networks

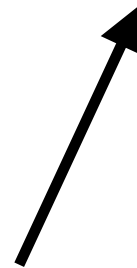
*Which functions have
low "quantum entanglement"?*

Any function of discrete variables can be represented as a tensor

$$f(s_1, s_2, s_3, s_4, s_5, s_6) =$$


The diagram shows a teal rounded rectangle representing a tensor. Above the rectangle, there are six vertical lines, each labeled with a variable $s_1, s_2, s_3, s_4, s_5, s_6$ from left to right. The rectangle is positioned to the right of the function notation $f(s_1, s_2, s_3, s_4, s_5, s_6) =$.

All values of f inside



Any function of discrete variables can be represented as a tensor

$$f(1, 2, 2, 2, 1, 2) = \text{[teal bar with indices 1, 2, 2, 2, 1, 2]} = 1.2$$

Any function of discrete variables can be represented as a tensor

$$f(2, 2, 2, 2, 2, 1) = \text{[teal bar with 6 indices: 2, 2, 2, 2, 2, 1]} = 0.3$$

Any function of discrete variables can be represented as a tensor

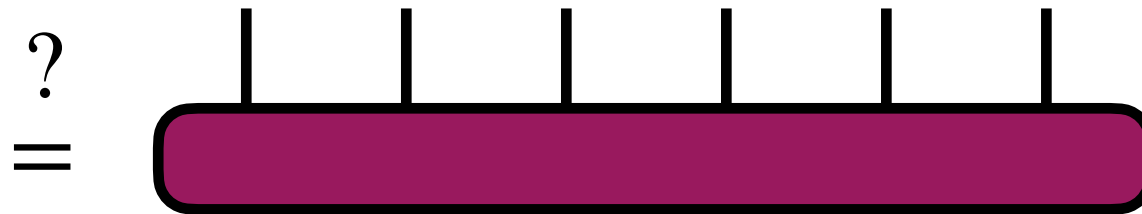
$$f(2, 1, 2, 2, 2, 2) = \text{[teal bar with indices 2, 1, 2, 2, 2, 2]} = -0.2$$

Any function of discrete variables can be represented as a tensor

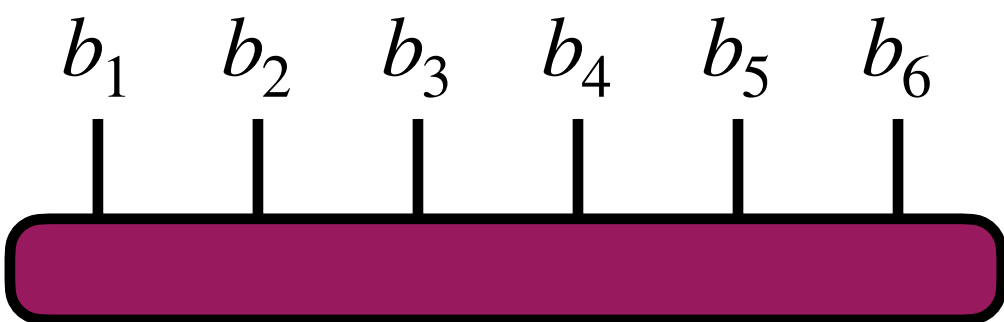
$$f(2, 1, 1, 2, 2, 2) = \text{[teal bar with indices 2, 1, 1, 2, 2, 2]} = 2.7$$

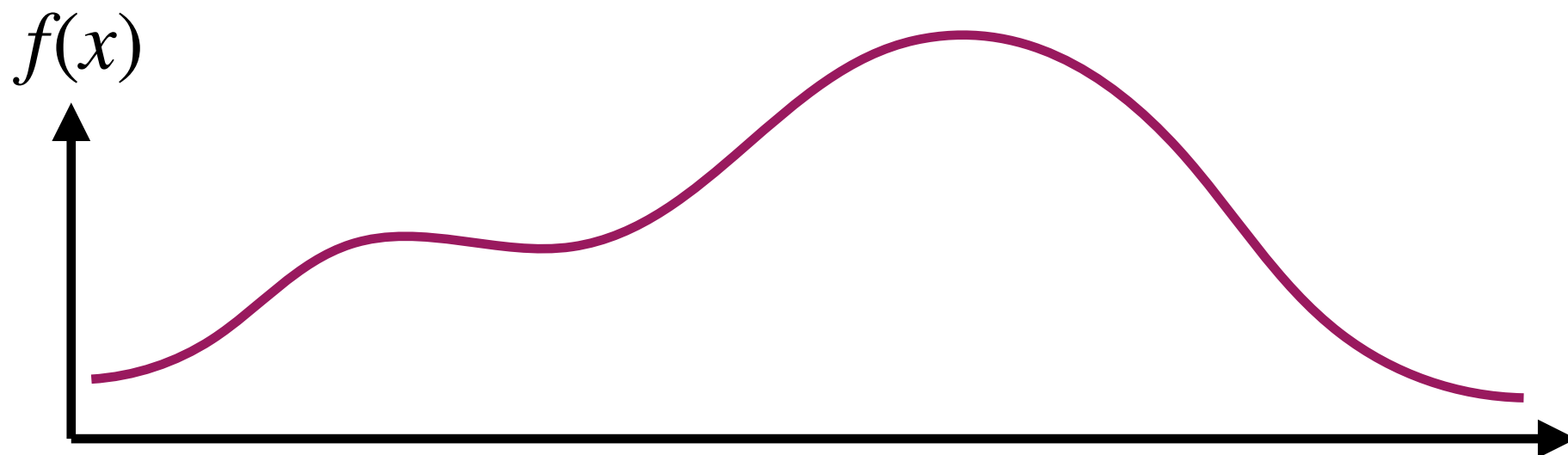
Can functions of **continuous variables** be
"tensorized" also?

$$f(x, y, z) = x^2 + 3y + \cos(z)$$



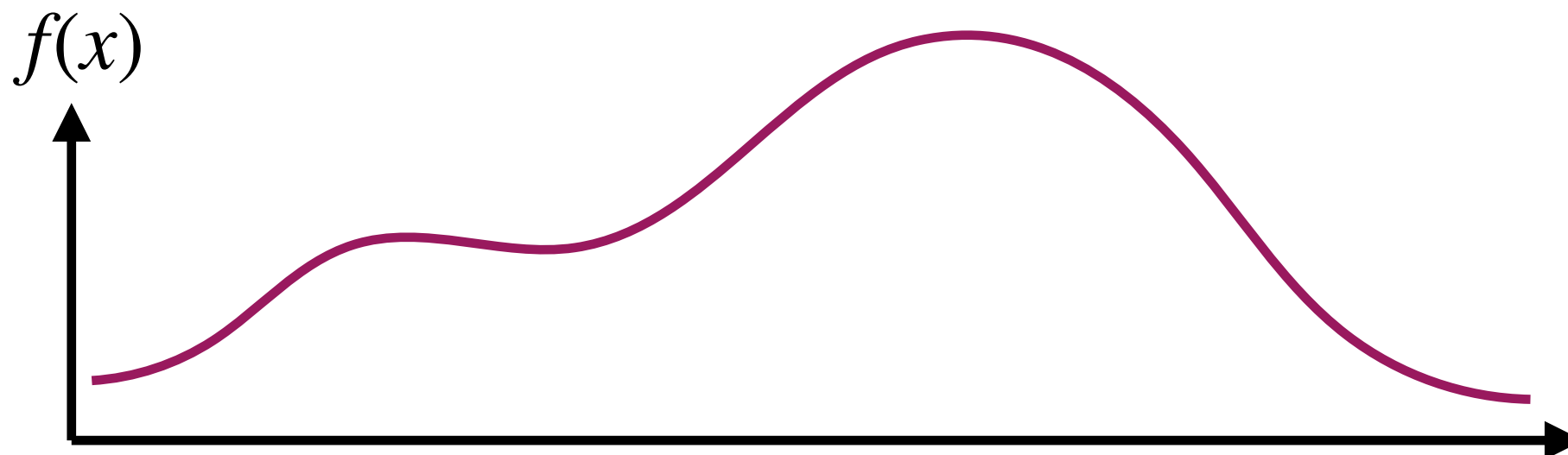
Yes. Using binary fraction approximation of $x \in [0,1)$

$$f(x) \approx f(0.b_1b_2b_3b_4b_5b_6) =$$




Let us understand in more detail...

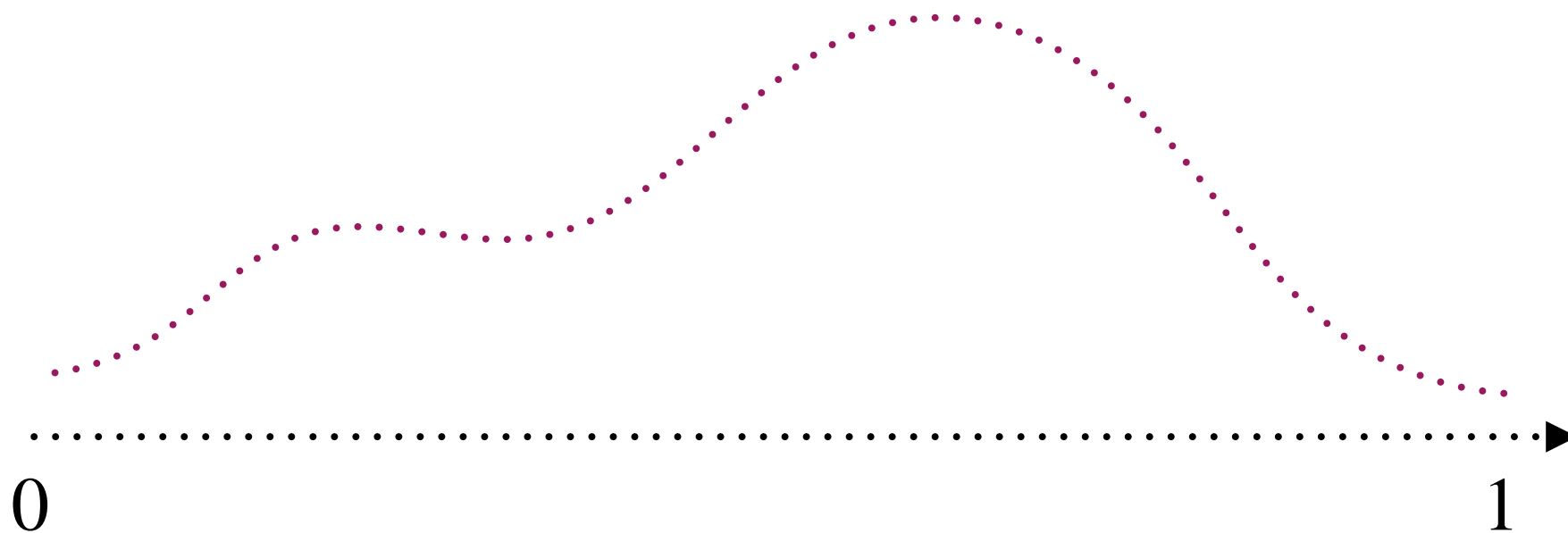
$$f(x) \approx f(0.b_1b_2b_3b_4b_5b_6) =$$

Quantum Functions

Discretize $[0,1)$ as grid of "binary fractions"

$f(x) =$



$$x \approx \frac{b_1}{2} + \frac{b_2}{2^2} + \frac{b_3}{2^3} + \frac{b_4}{2^4} + \dots + \frac{b_n}{2^n}$$

$$b_j = 0,1$$

$$\approx 0.b_1b_2b_3b_4\cdots b_n$$

Quantum Functions

On this grid, $f(x)$ takes values

$$f\left(\frac{b_1}{2} + \frac{b_2}{2^2} + \frac{b_3}{2^3} + \frac{b_4}{2^4} + \dots + \frac{b_n}{2^n}\right)$$

Quantum Functions

On this grid, $f(x)$ takes values

$$f\left(\frac{b_1}{2} + \frac{b_2}{2^2} + \frac{b_3}{2^3} + \frac{b_4}{2^4} + \dots + \frac{b_n}{2^n}\right)$$

defining a tensor

$$= F^{b_1 b_2 b_3 b_4 \cdots b_n} \quad b_j = 0, 1$$

all values of $f(0.b_1 b_2 b_3 \cdots b_n)$ inside

Quantum Functions

$$f\left(\frac{b_1}{2} + \frac{b_2}{2^2} + \frac{b_3}{2^3} + \frac{b_4}{2^4} + \dots + \frac{b_n}{2^n}\right) = F^{b_1 b_2 b_3 b_4 \dots b_n}$$

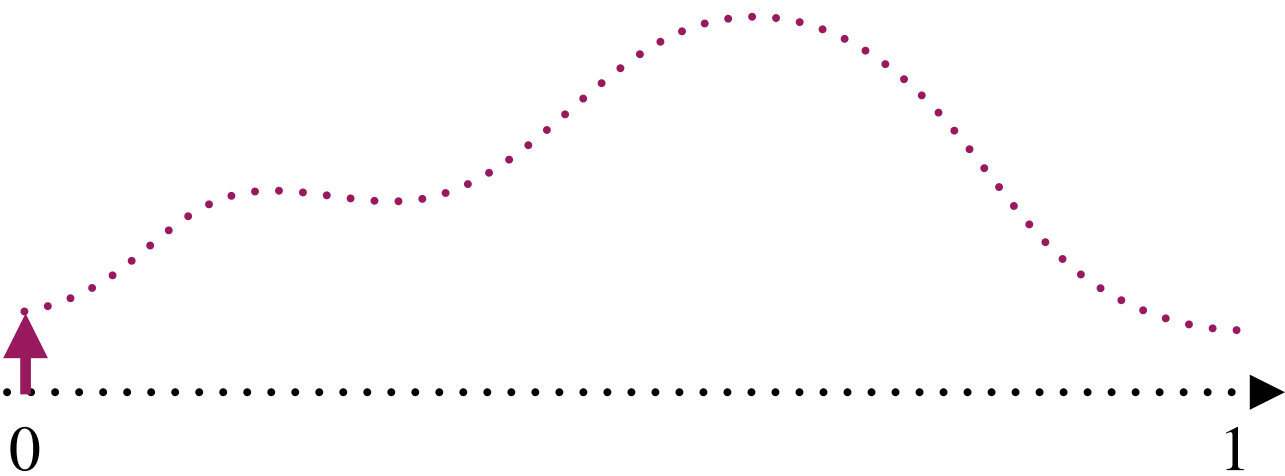
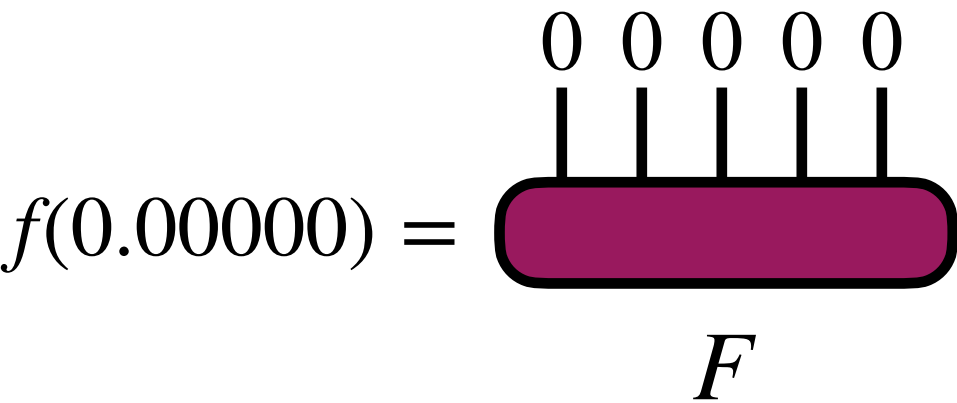
Nothing more than "amplitude encoding" idea
known in quantum computing literature...

$$|F\rangle = \sum_{\mathbf{b}} F^{b_1 b_2 b_3 \dots b_n} |b_1 b_2 b_3 \dots b_n\rangle$$

but with low entanglement / low-rank perspective

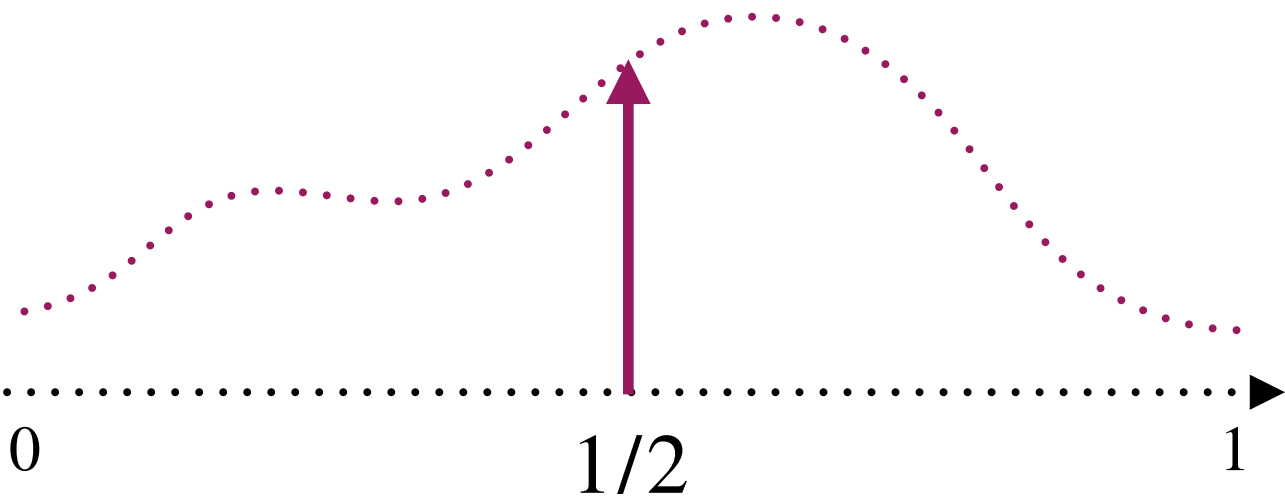
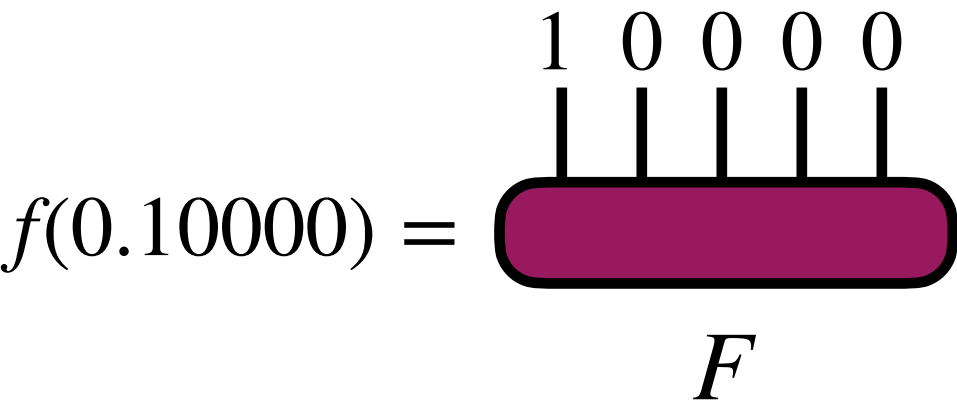
Quantum Functions

Examples of F entries



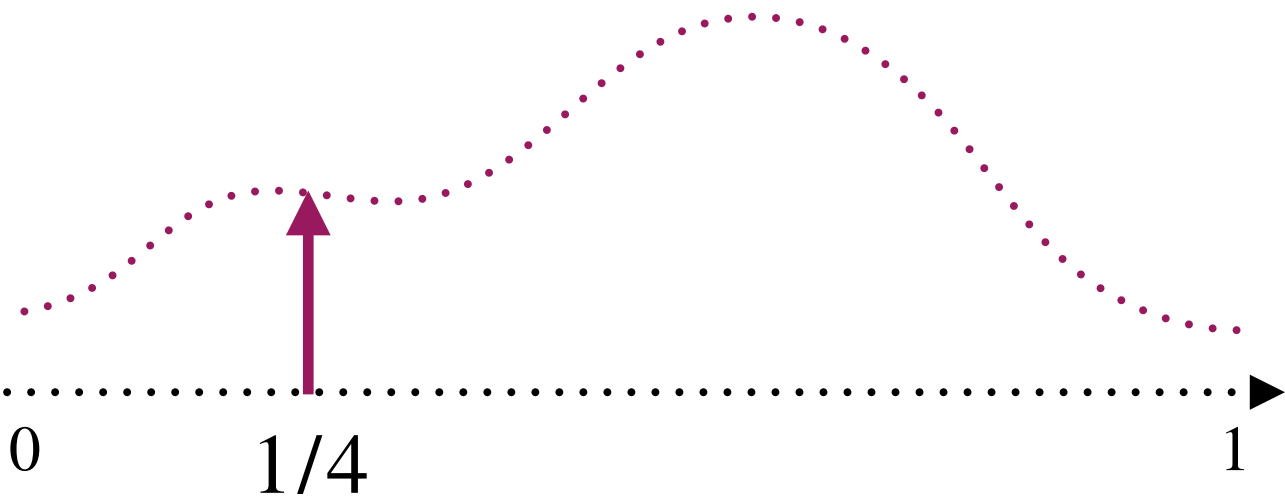
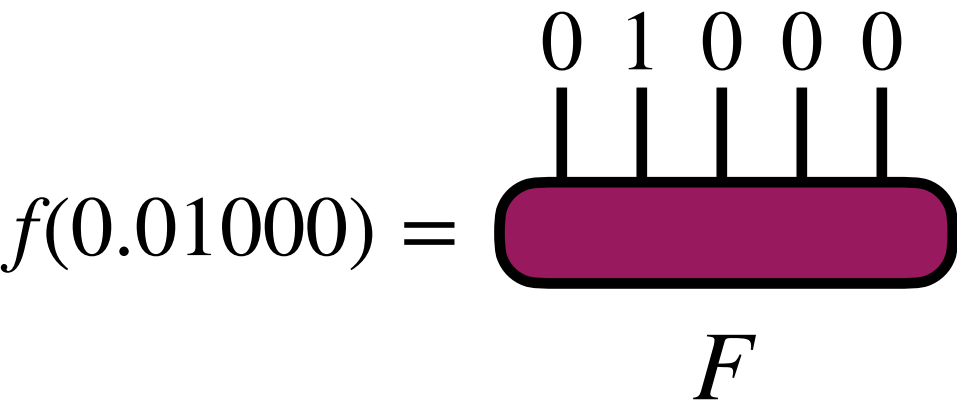
Quantum Functions

Examples of F entries



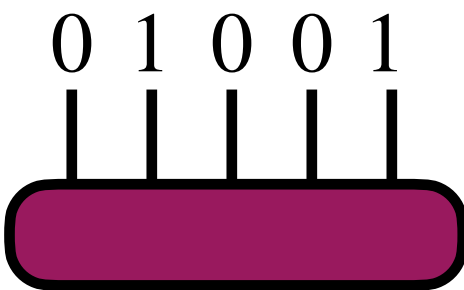
Quantum Functions

Examples of F entries

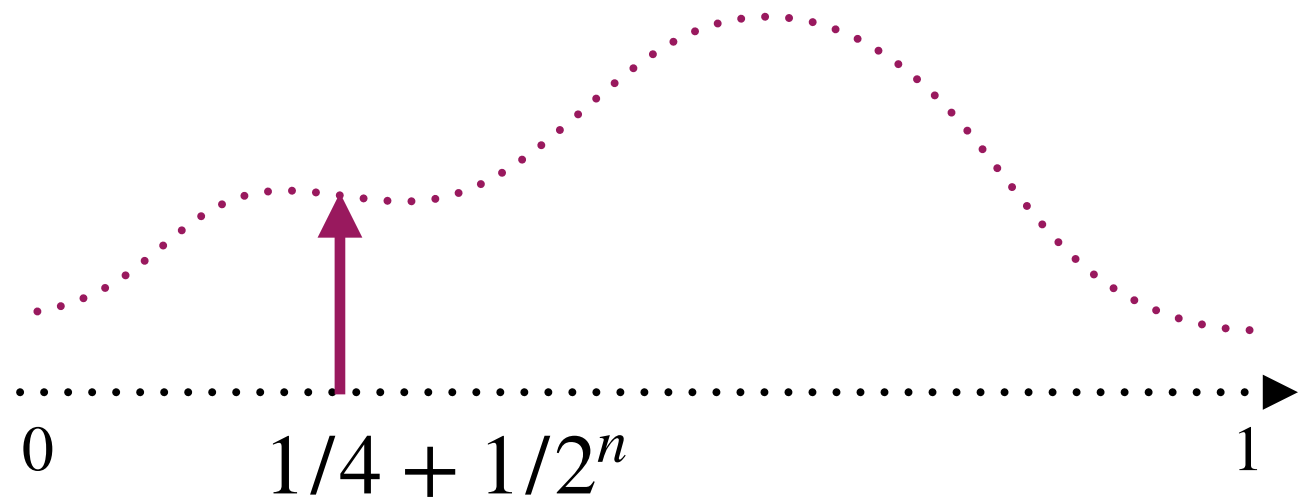


Quantum Functions

Examples of F entries

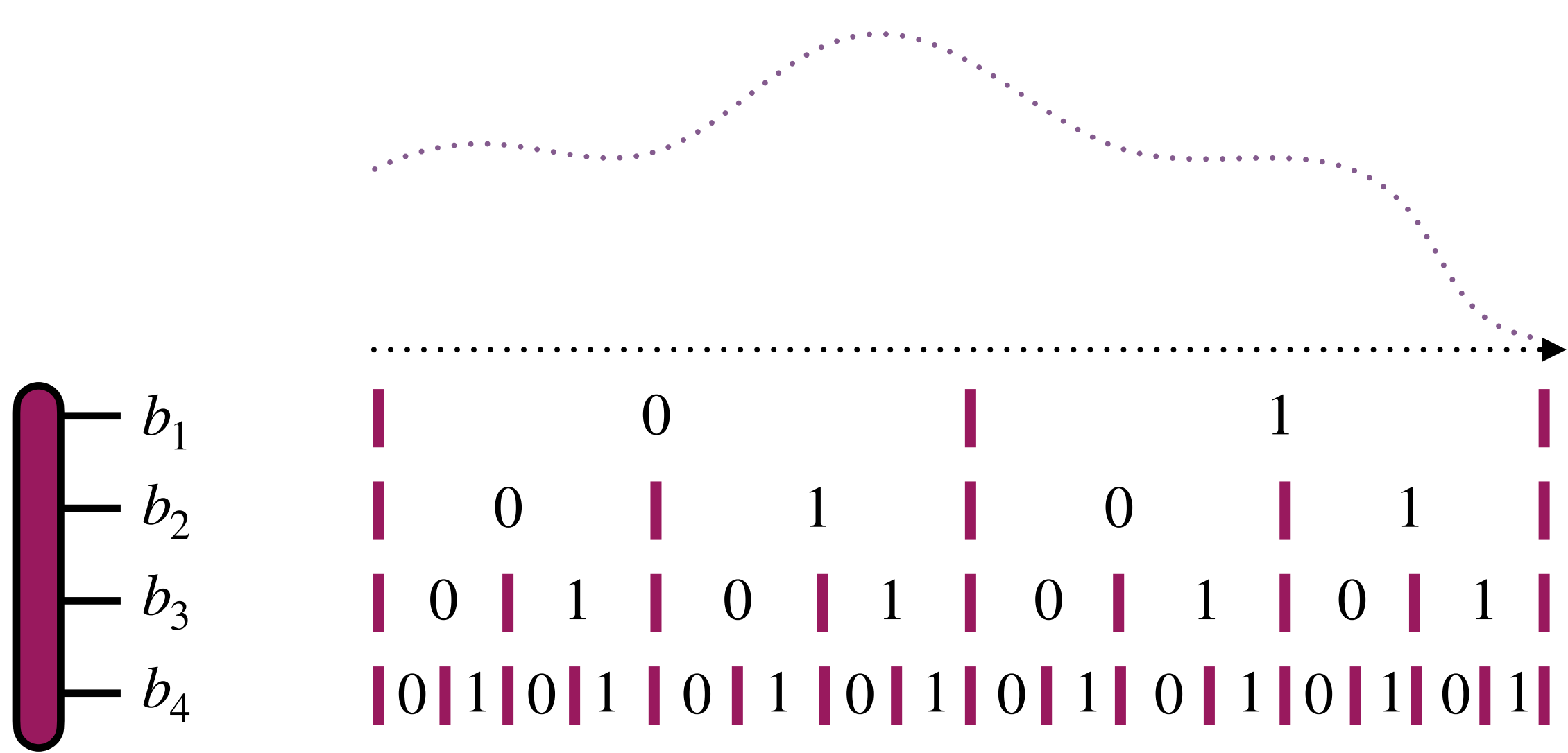
$$f(0.01001) =$$


F



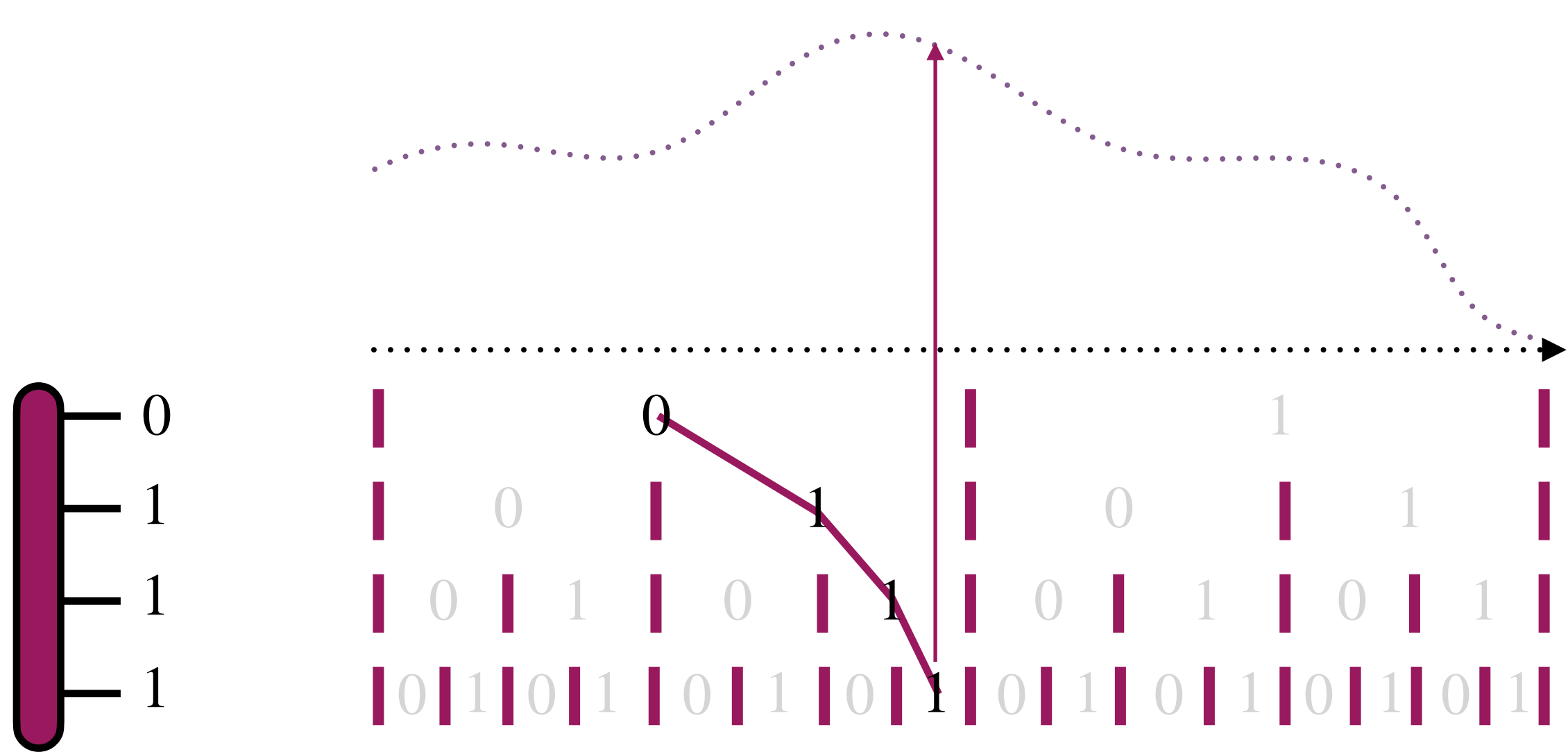
Quantum Functions

It is a multiscale representation



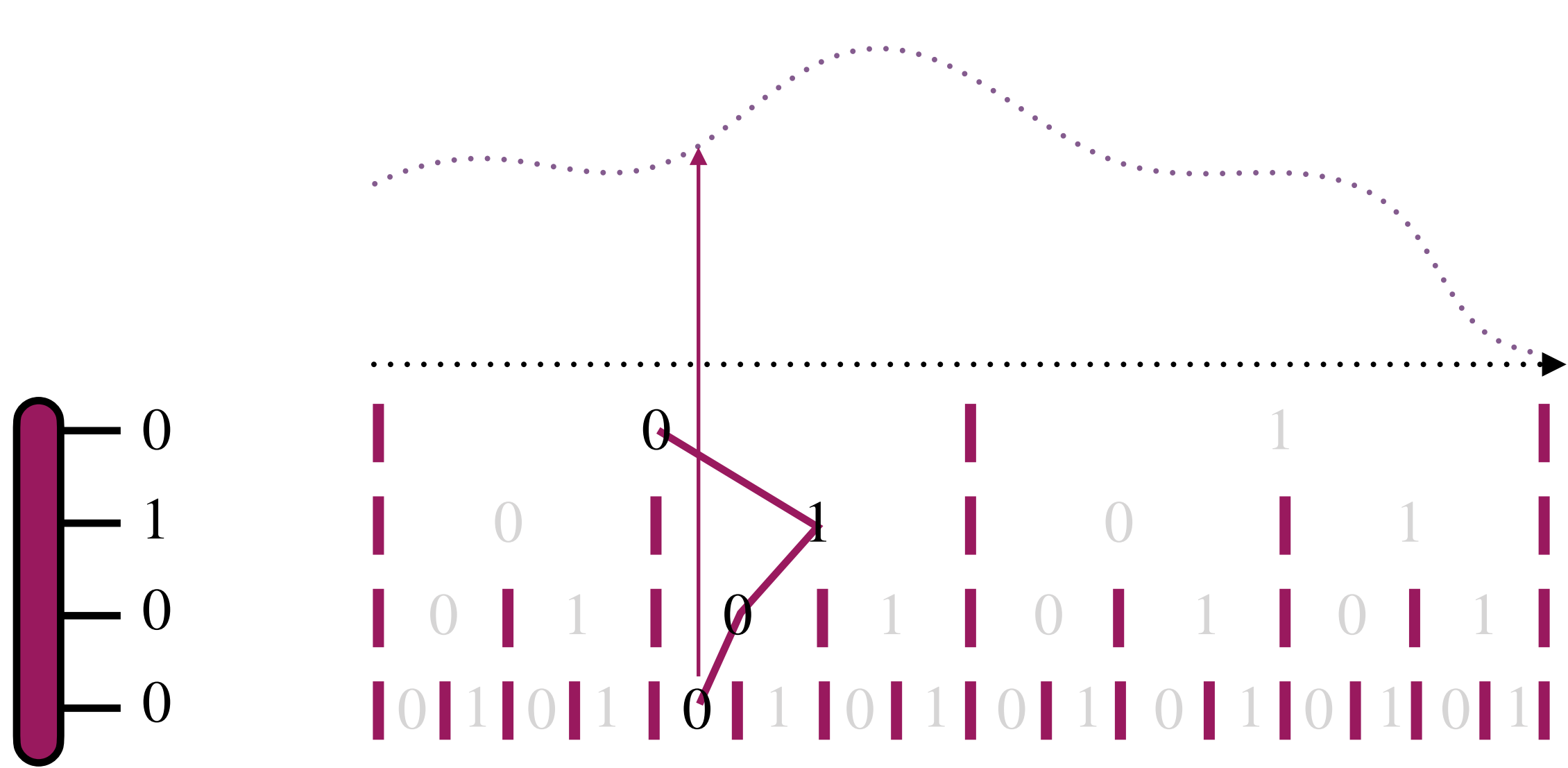
Quantum Functions

It is a multiscale representation



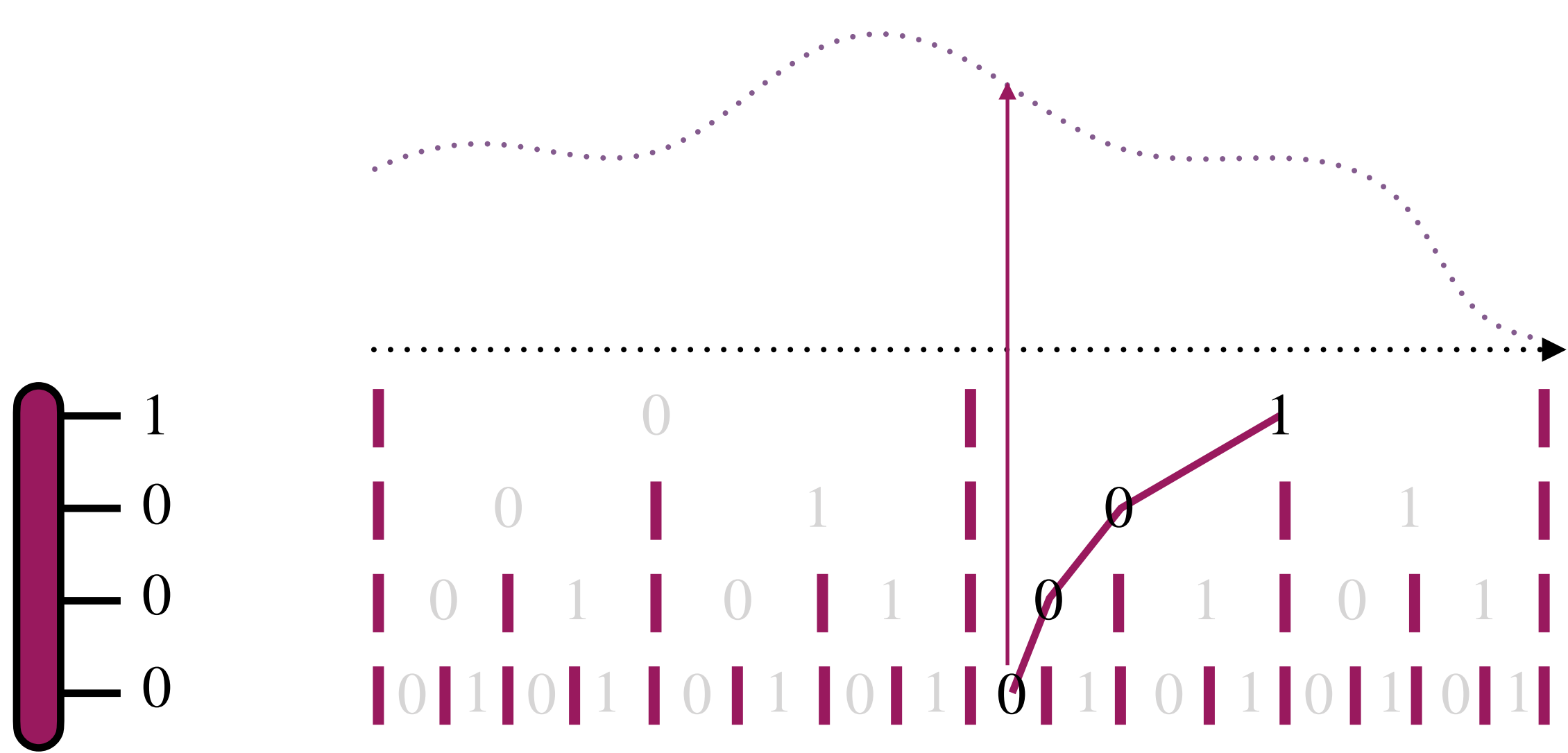
Quantum Functions

It is a multiscale representation



Quantum Functions

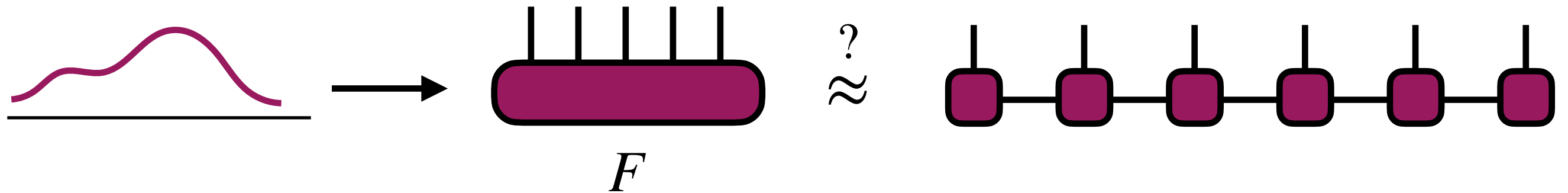
It is a multiscale representation



Quantum Functions

Key question: for some $f(x)$

is F low-rank as a tensor network? (i.e. "low entanglement")

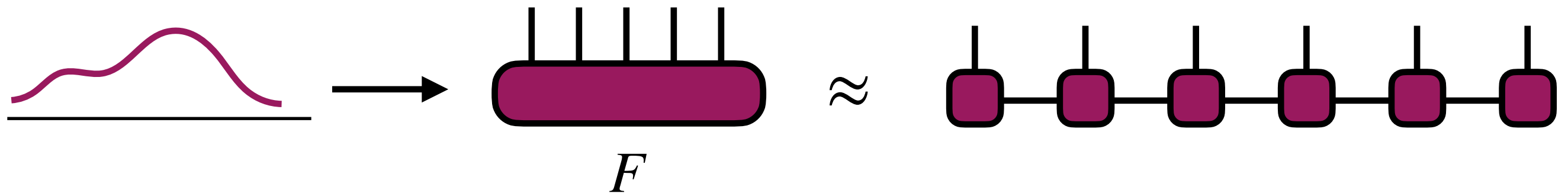


Otherwise exponential cost

Quantum Functions

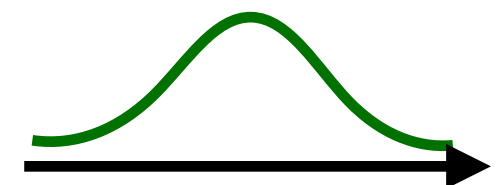
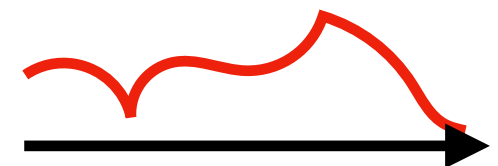
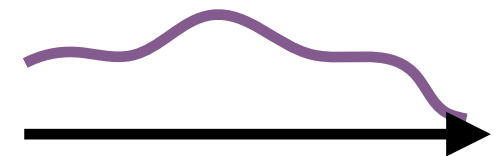
- [1] Mazen Ali, Anthony Nouy, Constr Approx
- [2] Mazen Ali, Anthony Nouy, arxiv:2101.11932
- [3] Chen, EMS, White, PRX Quantum, arxiv:2210.08468
- [4] Michael Lindsey, arxiv:2311.12554 (2023)
- [5] Chen, Lindsey, arxiv:2404.03182 (2024)

Low rank for many cases



Tensor network low rank for:

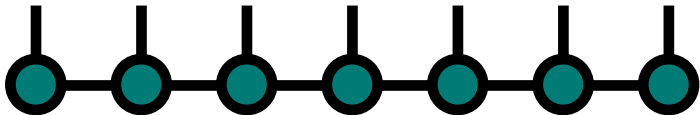
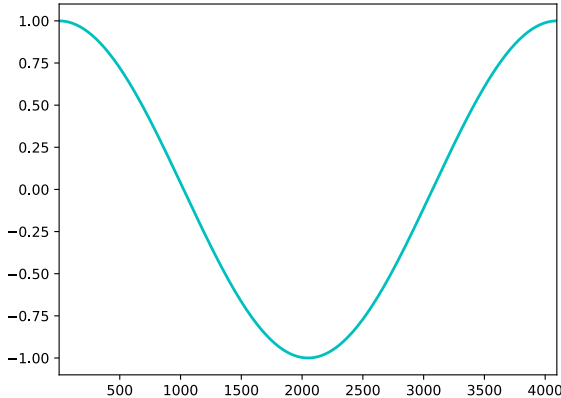
- all smooth enough functions [1,2,4]
- functions with finite number of cusps or discontinuities [1,2,4]
- any Fourier transform of these [3,5]



Quantum Functions

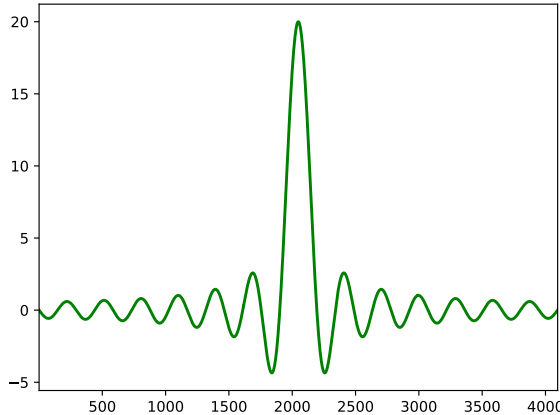
Examples:

$\cos \left[x - \frac{1}{2} \right]$



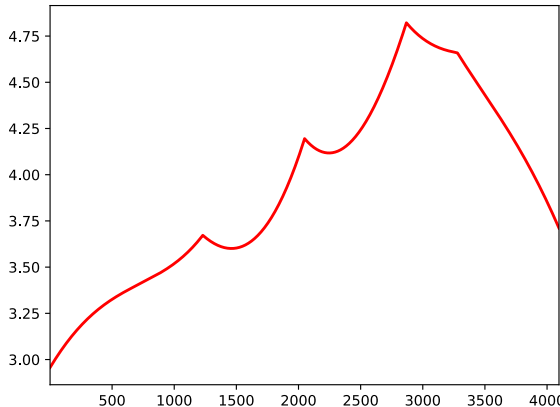
$\chi = 2$

sum of 20
cosines



$\chi = 18$

cosine
+ cusps

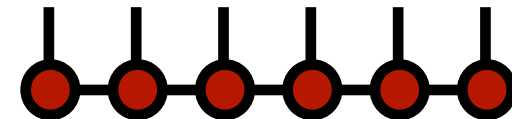


$\chi = 9$

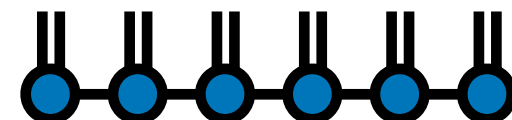
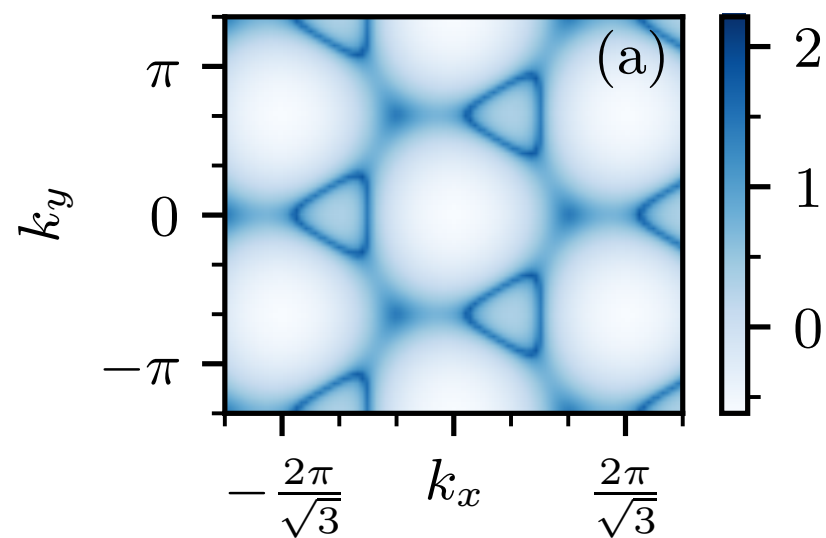
Quantum Functions

Also works for 2D, 3D, etc.

1D

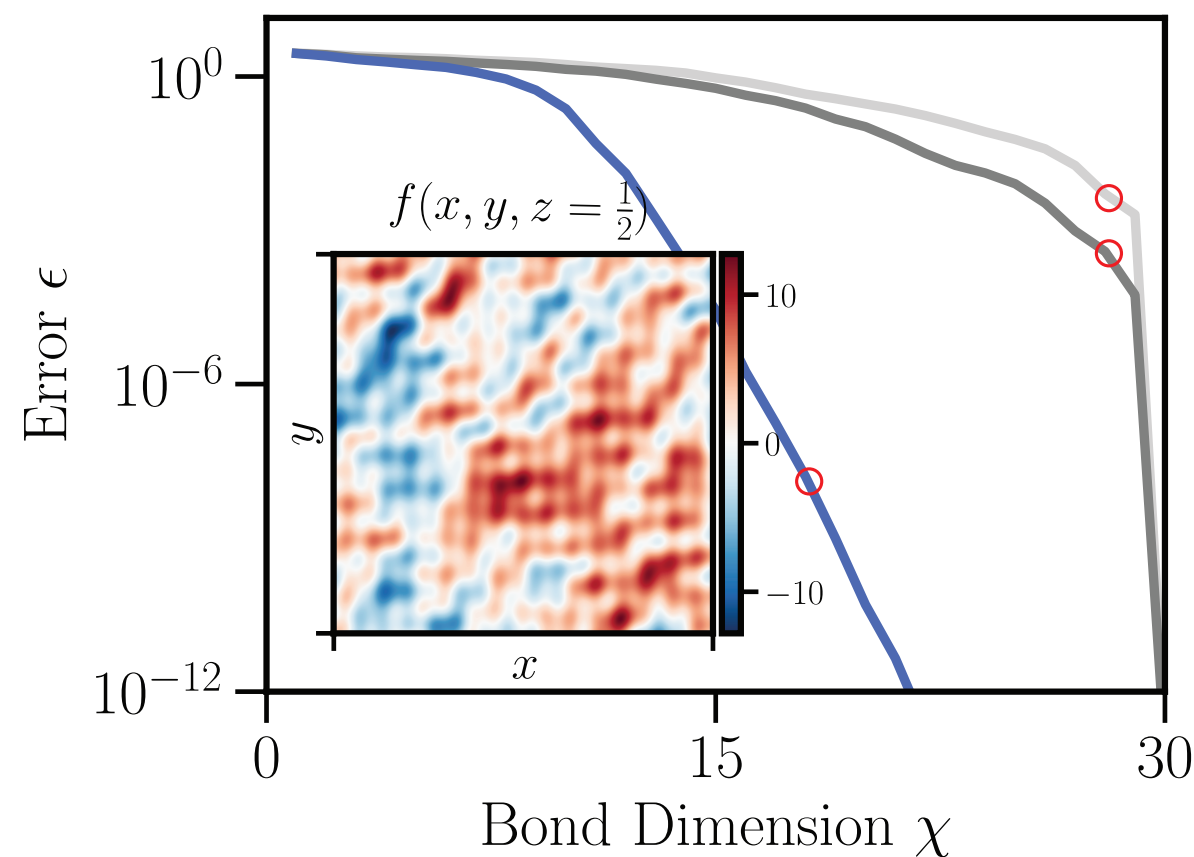
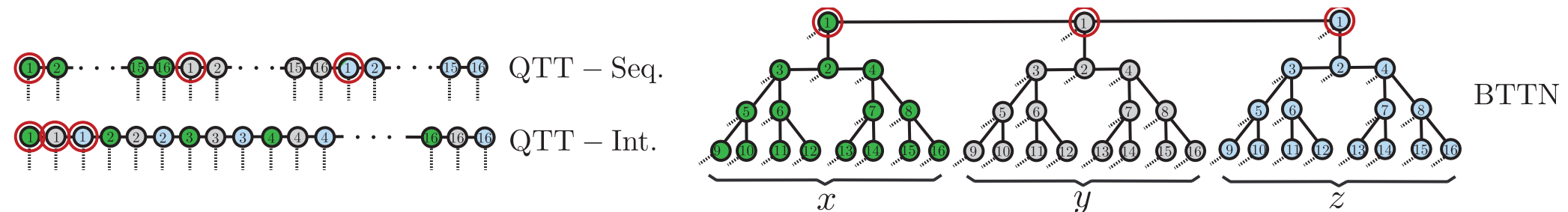


2D



Quantum Functions

Explored further at CCQ: tree architectures for functions



— interleaved
— sequential
— binary tree (BTTN)



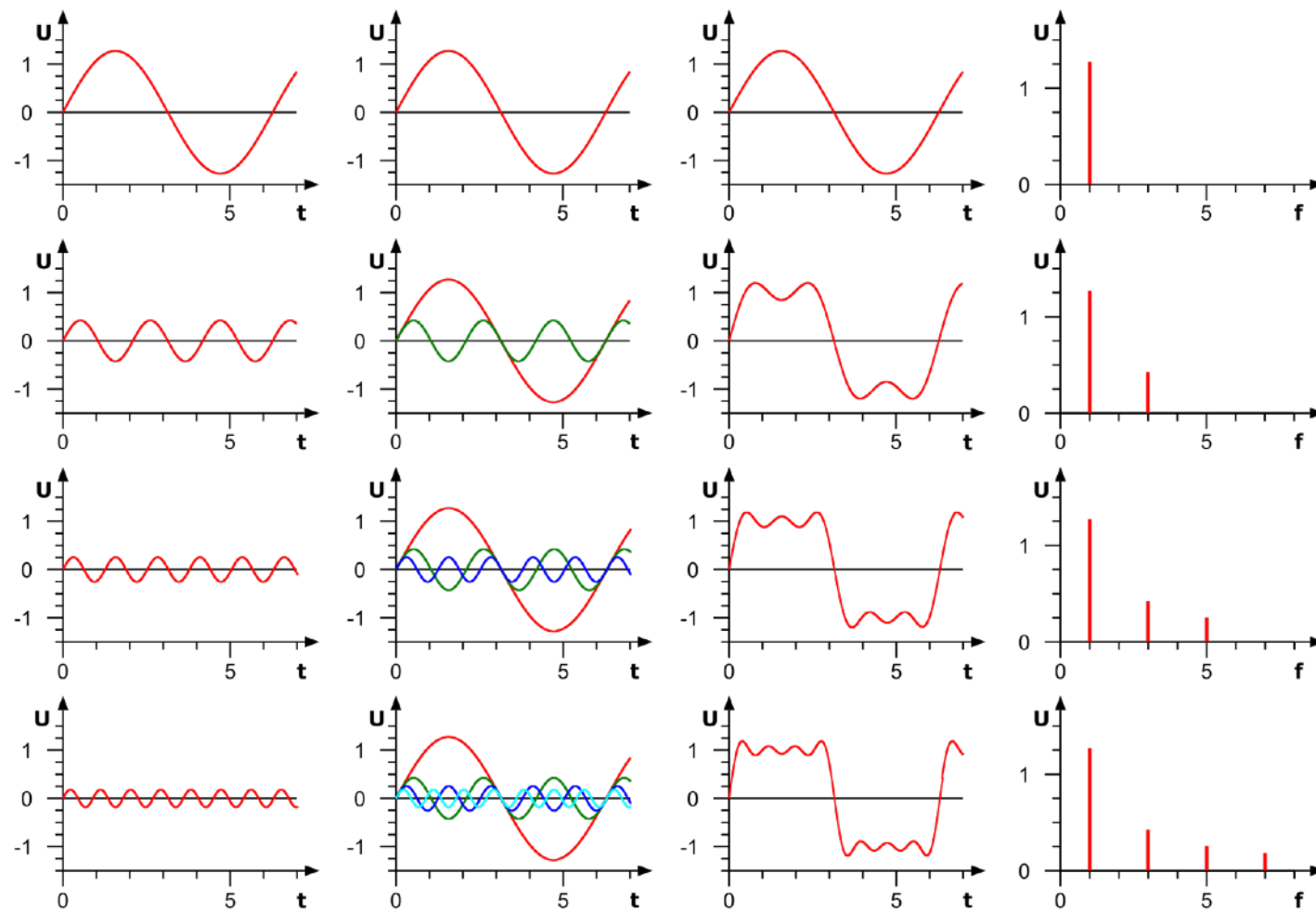
Ryan Levy



Joey Tindall

Quantum Functions

What about algorithms for "tensorized" functions?



Quantum-Inspired Algorithms

By now, many algorithms known for
tensor network functions [1]

- active learning of functions [2]
- integration [3]
- solving partial differential equations [4,5,6,7]
- Fourier transform and convolutions [8,6,13]
- find extrema (global min & max) without gradients [9]
- interpolation [1] & extrapolation of functions [10]
- perfect sampling [11,12]

[1] Garcia-Ripoll, Quantum 5, 431 (2021)

[2] Dolgov, Savostyanov, Computer Physics Communications, 246, 106869 (2020)

[3] Khoromskij, Constr Approx 34, 257–280 (2011)

[4] Khoromskij, arxiv:1408.4053

[5] Gourianov et al., Nature Computational Science 2, 30–37 (2022)

[6] Shinaoka et al., Phys. Rev. X 13, 021015 (2023)

[7] Ye, Loureiro, Phys. Rev. E 106, 035208 (2022)

[8] Chen, EMS, White, PRX Quantum, arxiv:2210.08468

[9] Chertkov et al., arxiv:2209.14808

[10] Lin, White, arxiv:2308.09001

[11] White, EMS, New J. Phys 12, 055026 (2010)

[12] Ferris, Vidal, Phys. Rev. B 85, 165146 (2012)

[13] Chen, Lindsey, arxiv:2404.03182 (2024)

Quantum-Inspired Algorithms

By now, many algorithms known for tensor network functions [1]

- active learning of functions [2]
- **integration** [3]
- solving partial differential equations [4,5,6,7]
- Fourier transform and convolutions [8,6,13]
- find extrema (global min & max) without gradients [9]
- interpolation [1] & extrapolation of functions [10]
- perfect sampling [11,12]

[1] Garcia-Ripoll, Quantum 5, 431 (2021)

[2] Dolgov, Savostyanov, Computer Physics Communications, 246, 106869 (2020)

[3] Khoromskij, Constr Approx 34, 257–280 (2011)

[4] Khoromskij, arxiv:1408.4053

[5] Gourianov et al., Nature Computational Science 2, 30–37 (2022)

[6] Shinaoka et al., Phys. Rev. X 13, 021015 (2023)

[7] Ye, Loureiro, Phys. Rev. E 106, 035208 (2022)

[8] Chen, EMS, White, PRX Quantum, arxiv:2210.08468

[9] Chertkov et al., arxiv:2209.14808

[10] Lin, White, arxiv:2308.09001

[11] White, EMS, New J. Phys 12, 055026 (2010)

[12] Ferris, Vidal, Phys. Rev. B 85, 165146 (2012)

[13] Chen, Lindsey, arxiv:2404.03182 (2024)

Quantum-Inspired Algorithms

Integration of a tensorized function

$$\int_0^1 dx f(x) \approx \sum_{b_1, b_2, \dots} \frac{1}{2^n} f\left(\frac{b_1}{2} + \frac{b_2}{2^2} + \frac{b_3}{2^3} + \frac{b_4}{2^4} + \dots + \frac{b_n}{2^n}\right)$$

$$= \sum_{b_1, b_2, \dots} \frac{1}{2^n} F^{b_1 b_2 b_3 b_4 \dots b_n}$$

$$= \text{Diagram of } F \text{ with 6 inputs} \quad \text{Input symbol} = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}$$

The diagram shows a purple rounded rectangle labeled F with six vertical lines extending upwards, each ending in a black circle. To the right, a single black circle on a vertical line is equated to a column vector $\begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}$.

$$\approx \text{Diagram of 6 qubits in a chain}$$

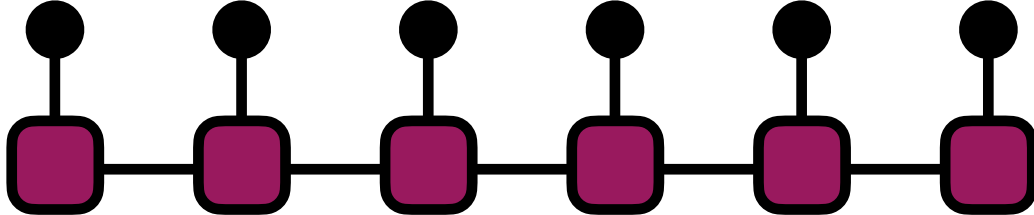
The diagram shows a horizontal chain of six purple squares, each with a black circle on top. The squares are connected by horizontal lines, representing a sequence of qubits.

Cost:

$$n\chi^2 = \log(N) \chi^2$$

Quantum-Inspired Algorithms

Integration of a tensorized function

$$\int_0^1 dx f(x) \approx$$


In the upcoming hands-on, we will ask you to implement this algorithm using ITensor

```
function integrate(psi::MPS)
    I = ITensor(1.)
    ...
    return scalar(I)
end
```

Quantum-Inspired Algorithms

By now, many algorithms known for tensor network functions [1]

- active learning of functions [2]
- integration [3]
- solving partial differential equations [4,5,6,7]
- Fourier transform and convolutions [8,6,13]
- find extrema (global min & max) without gradients [9]
- interpolation [1] & extrapolation of functions [10]
- perfect sampling [11,12]

[1] Garcia-Ripoll, Quantum 5, 431 (2021)

[2] Dolgov, Savostyanov, Computer Physics Communications, 246, 106869 (2020)

[3] Khoromskij, Constr Approx 34, 257–280 (2011)

[4] Khoromskij, arxiv:1408.4053

[5] Gourianov et al., Nature Computational Science 2, 30–37 (2022)

[6] Shinaoka et al., Phys. Rev. X 13, 021015 (2023)

[7] Ye, Loureiro, Phys. Rev. E 106, 035208 (2022)

[8] Chen, EMS, White, PRX Quantum, arxiv:2210.08468

[9] Chertkov et al., arxiv:2209.14808

[10] Lin, White, arxiv:2308.09001

[11] White, EMS, New J. Phys 12, 055026 (2010)

[12] Ferris, Vidal, Phys. Rev. B 85, 165146 (2012)

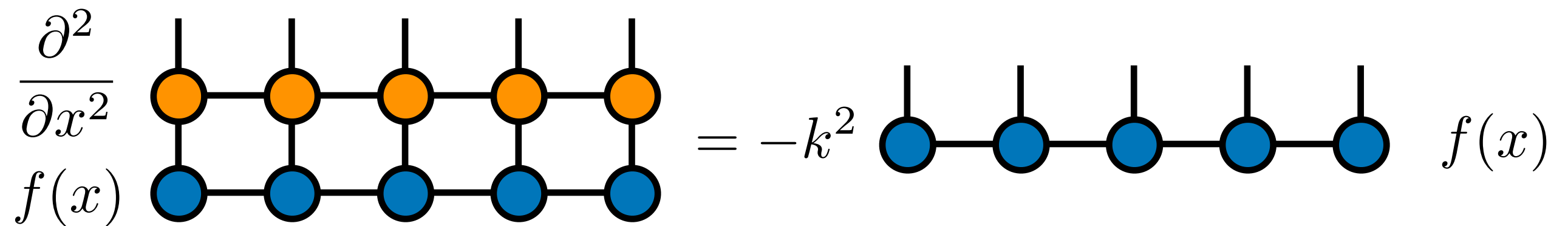
[13] Chen, Lindsey, arxiv:2404.03182 (2024)

Quantum-Inspired Algorithms

Given a diff. eq. such as wave equation

$$\frac{\partial^2}{\partial x^2} f(x) = -k^2 f(x)$$

Can encode as tensor network equation

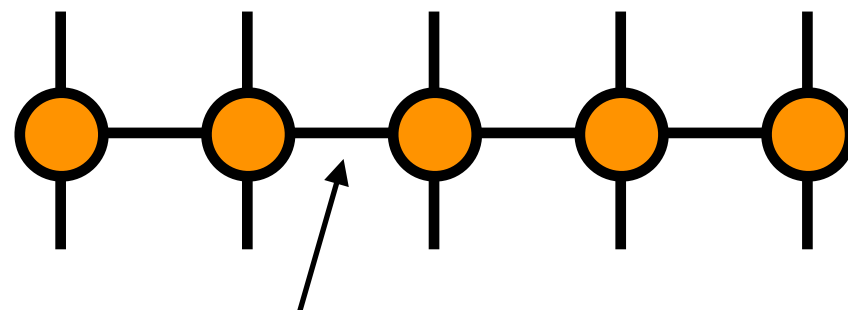

$$\frac{\partial^2}{\partial x^2} f(x) = -k^2 f(x)$$

Use DMRG algorithm to efficiently find eigenvector
or can time step

Quantum-Inspired Algorithms

Finite-difference formula for $\frac{\partial^2}{\partial x^2}$

translates into exact expression for
low-rank *matrix product operator* (MPO) tensor network

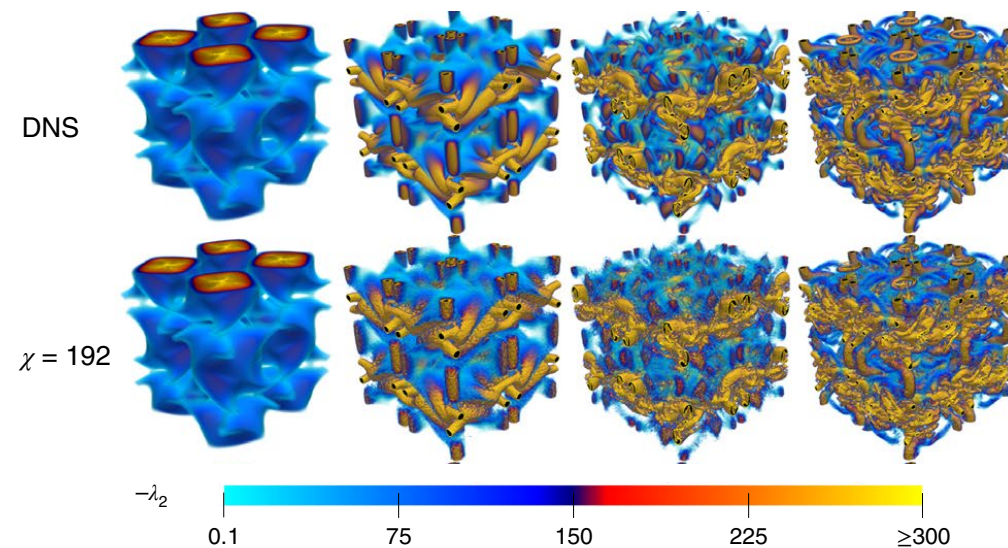


Basically: forward binary adder, backwards binary adder,
plus constant = $(x_{j+1} - 2x_j + x_{j-1})/a^2$

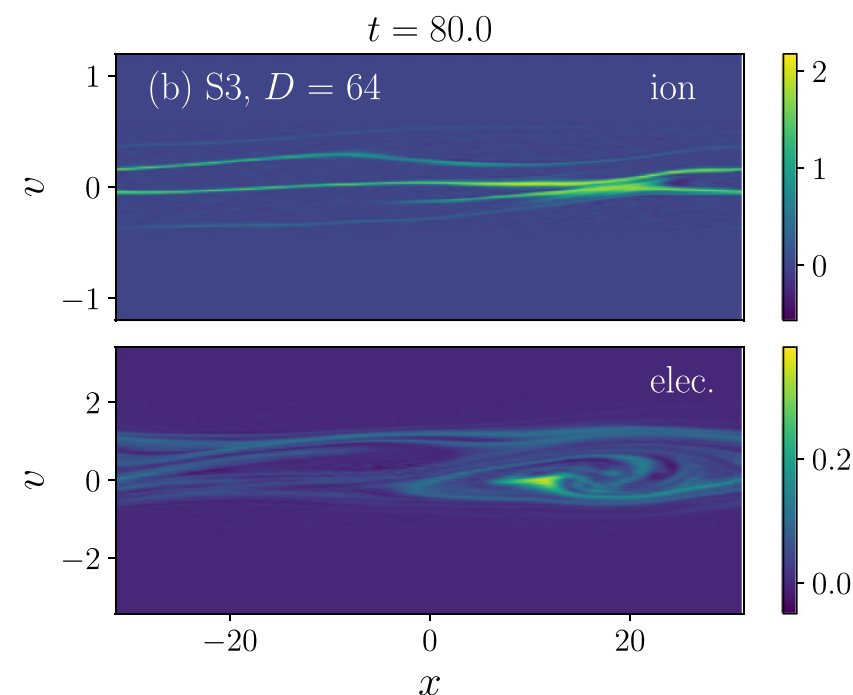
Quantum-Inspired Algorithms

Application: Fluid mechanics

Navier-Stokes equation:
Rank χ determined by
Reynolds number ^[1]



Plasma flows:
(Vlasov-Poisson equation) ^[2]



[1] Gourianov et al., Nature Comp. Sci., 2, 30-37 (2022)

[2] Ye, Loureiro, PRE106, 035208 (2022)

Quantum-Inspired Algorithms

By now, many algorithms known for tensor network functions [1]

- active learning of functions [2]
- integration [3]
- solving partial differential equations [4,5,6,7]
- **Fourier transform** and convolutions [8,6,13]
- find extrema (global min & max) without gradients [9]
- interpolation [1] & extrapolation of functions [10]
- perfect sampling [11,12]

[1] Garcia-Ripoll, Quantum 5, 431 (2021)

[2] Dolgov, Savostyanov, Computer Physics Communications, 246, 106869 (2020)

[3] Khoromskij, Constr Approx 34, 257–280 (2011)

[4] Khoromskij, arxiv:1408.4053

[5] Gourianov et al., Nature Computational Science 2, 30–37 (2022)

[6] Shinaoka et al., Phys. Rev. X 13, 021015 (2023)

[7] Ye, Loureiro, Phys. Rev. E 106, 035208 (2022)

[8] Chen, EMS, White, PRX Quantum, arxiv:2210.08468

[9] Chertkov et al., arxiv:2209.14808

[10] Lin, White, arxiv:2308.09001

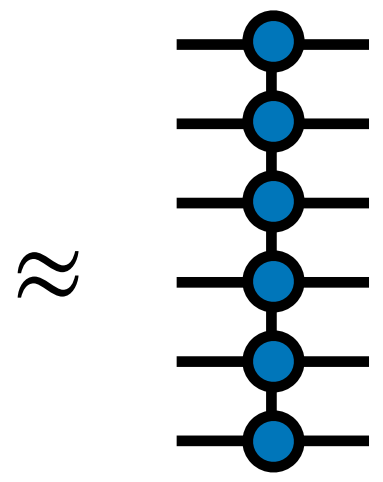
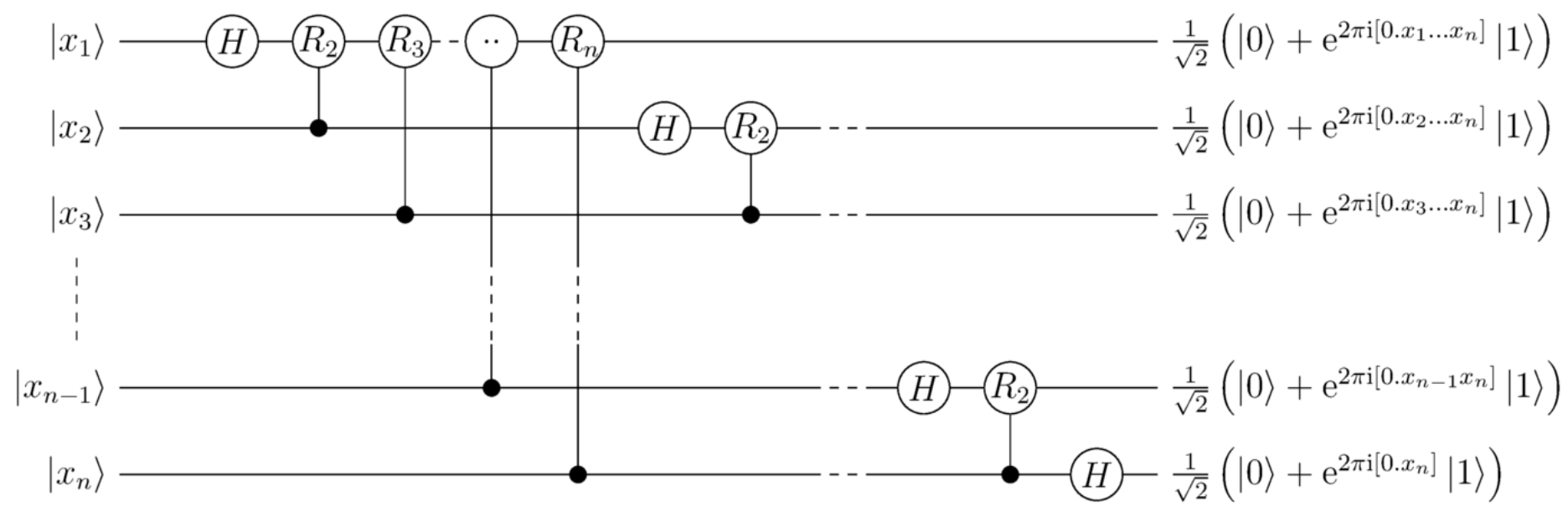
[11] White, EMS, New J. Phys 12, 055026 (2010)

[12] Ferris, Vidal, Phys. Rev. B 85, 165146 (2012)

[13] Chen, Lindsey, arxiv:2404.03182 (2024)

Analogy between quantum and classical so precise,
 some quantum algorithms have been classical all along

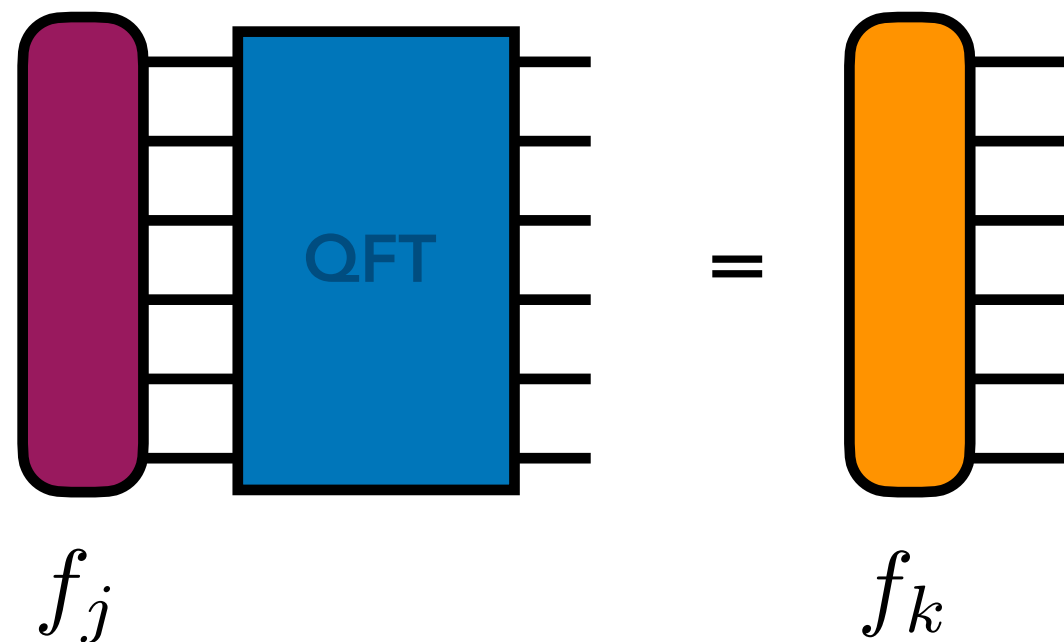
Quantum Fourier transform (QFT):



Equal to network of small tensors
 ("MPO" tensor network)

Quantum Fourier Transform

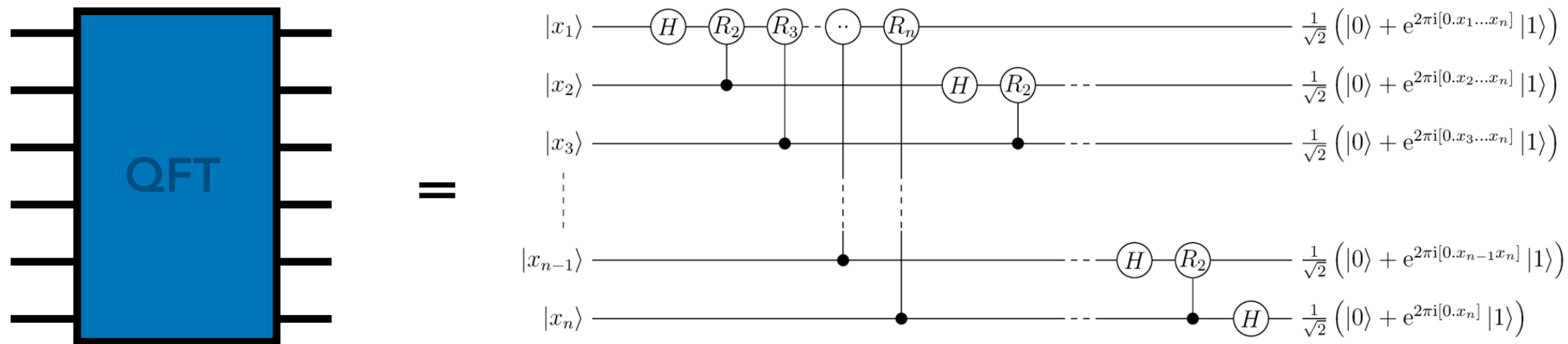
QFT performs discrete Fourier transform on a quantum state (viewed as a large vector)



$$\sum_j f_j e^{-\frac{i2\pi}{N}kj} = f_k$$

Quantum Fourier Transform

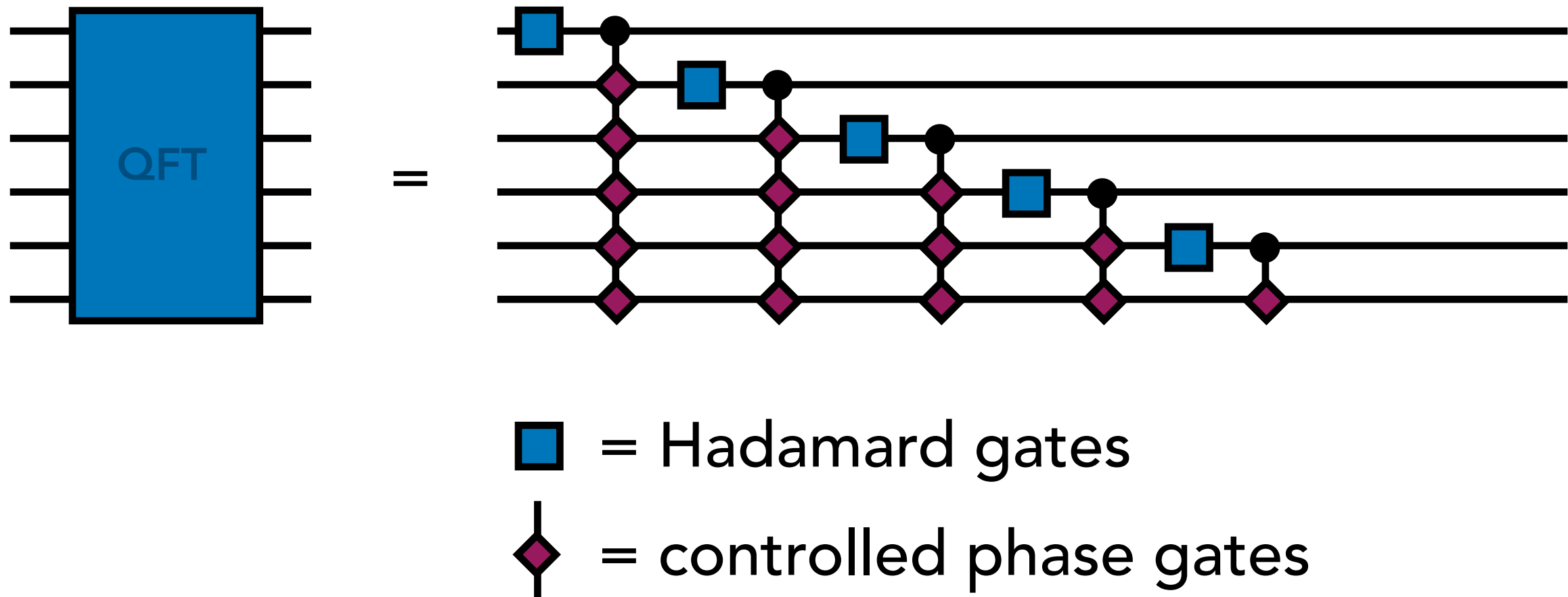
Circuit for QFT can be interpreted as a tensor network



Treat each column as tensor network and multiply together

Quantum Fourier Transform

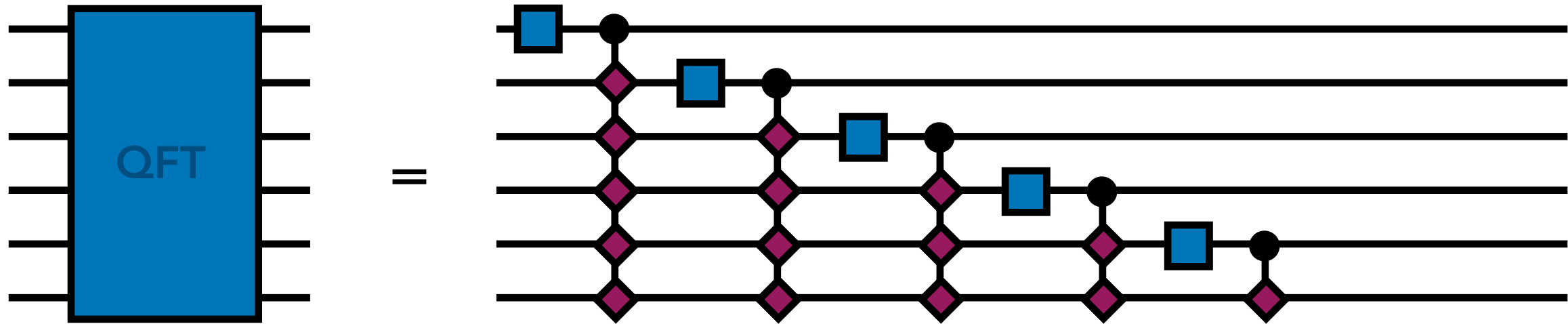
Circuit for QFT can be interpreted as a tensor network



Treat each column as tensor network and multiply together

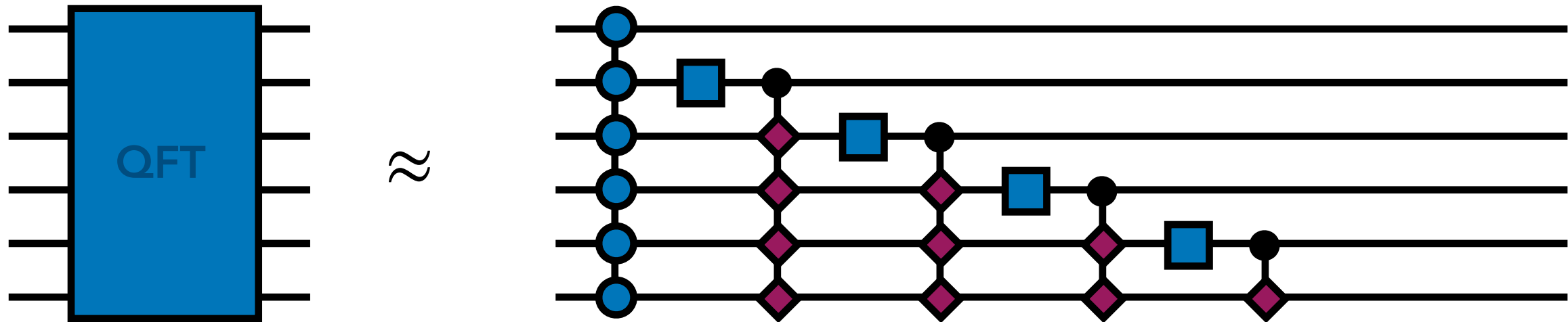
Quantum Fourier Transform

Treat each column as a tensor network
and multiply them together:



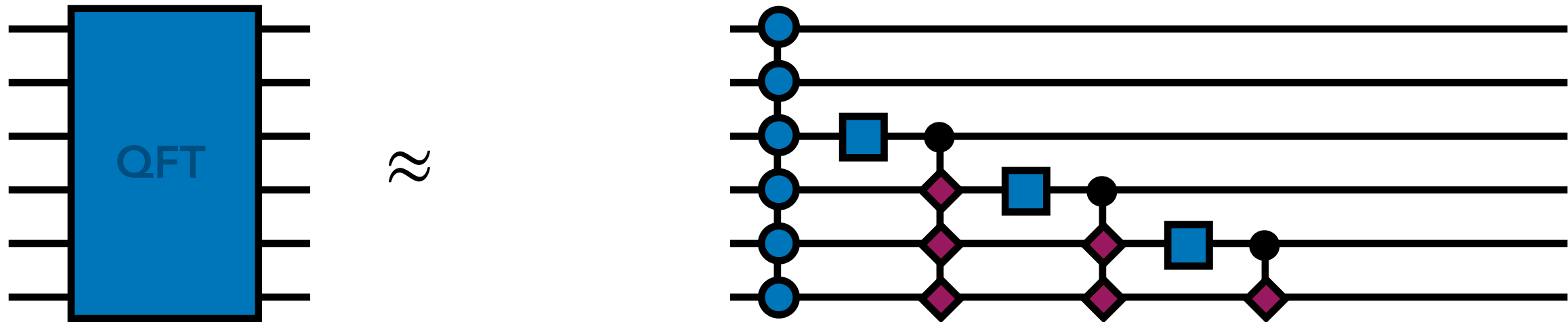
Quantum Fourier Transform

Treat each column as a tensor network
and multiply them together:



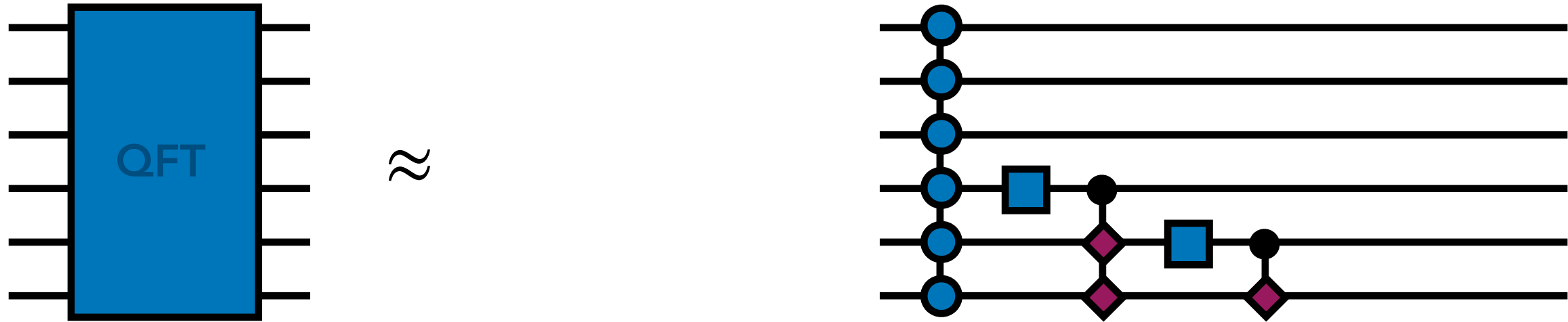
Quantum Fourier Transform

Treat each column as a tensor network
and multiply them together:



Quantum Fourier Transform

Treat each column as a tensor network
and multiply them together:



Quantum Fourier Transform

Treat each column as a tensor network
and multiply them together:



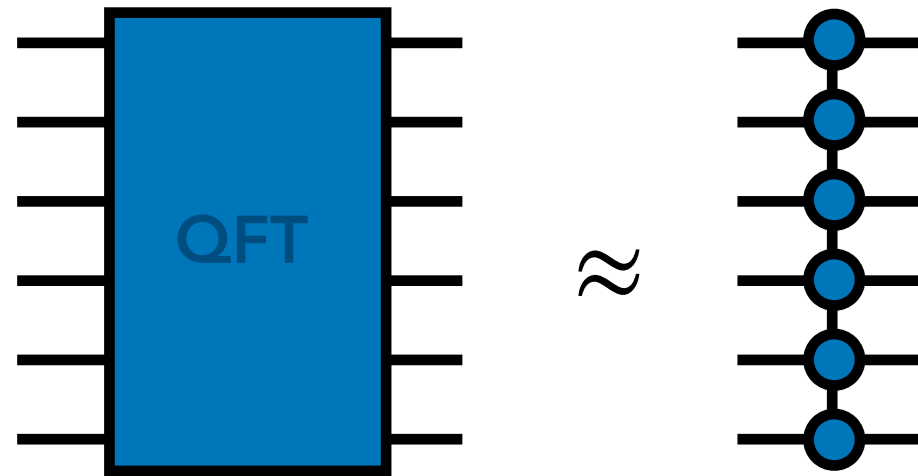
Quantum Fourier Transform

Treat each column as a tensor network
and multiply them together:



Quantum Fourier Transform

Treat each column as a tensor network and multiply them together:



Result is network of internal dimension $\chi = 10$!
(When working to double precision.)



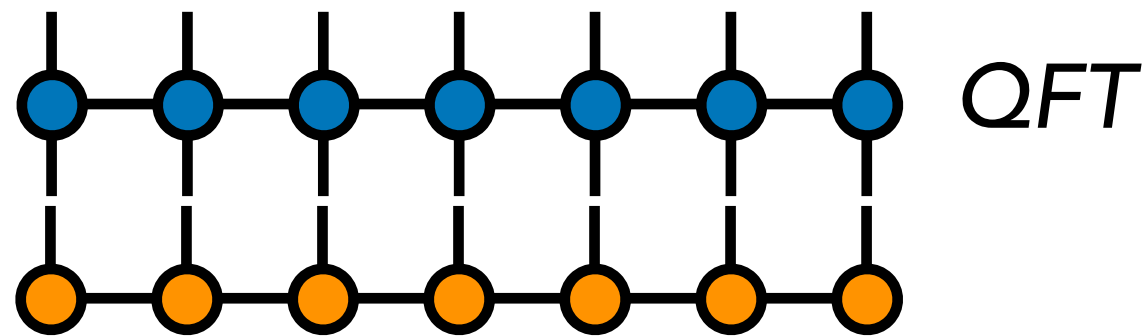
Chris Chen



Steve White

Quantum Fourier Transform

Shows that QFT algorithm has limited entangling power

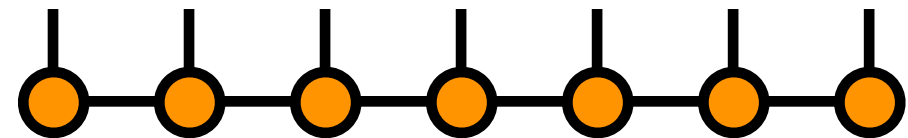
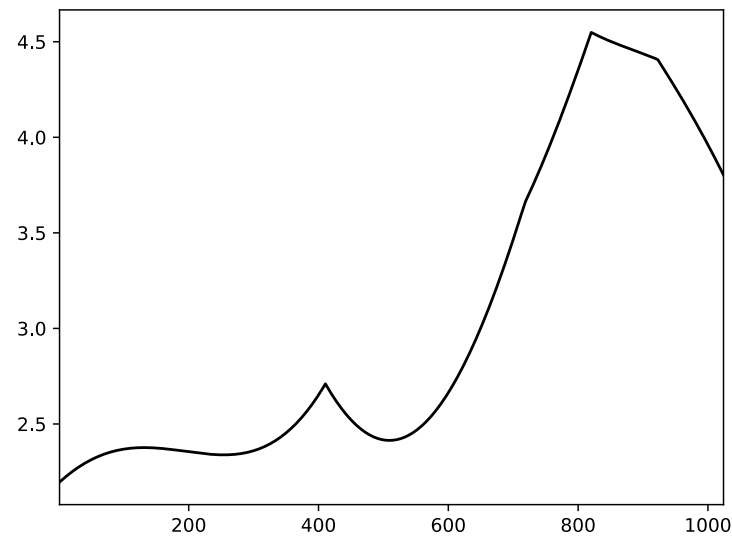


For any quantum state that can be stored classically,
can always perform QFT efficiently

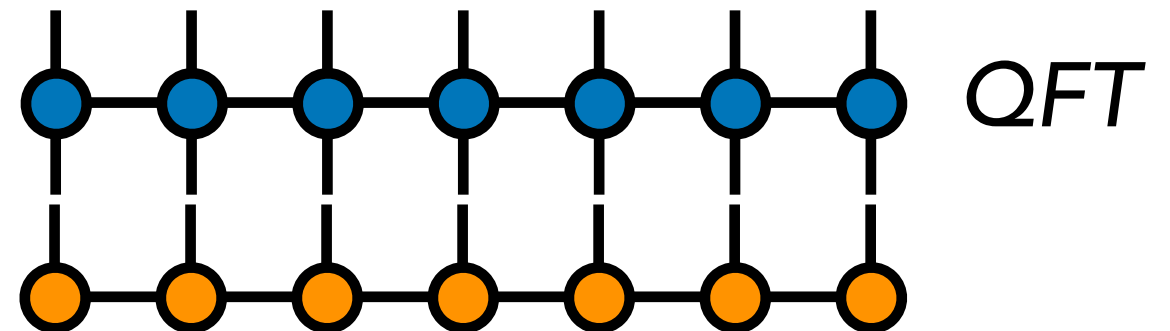
Quantum Fourier Transform

Can use to perform "superfast" Fourier transform

*compress
function*

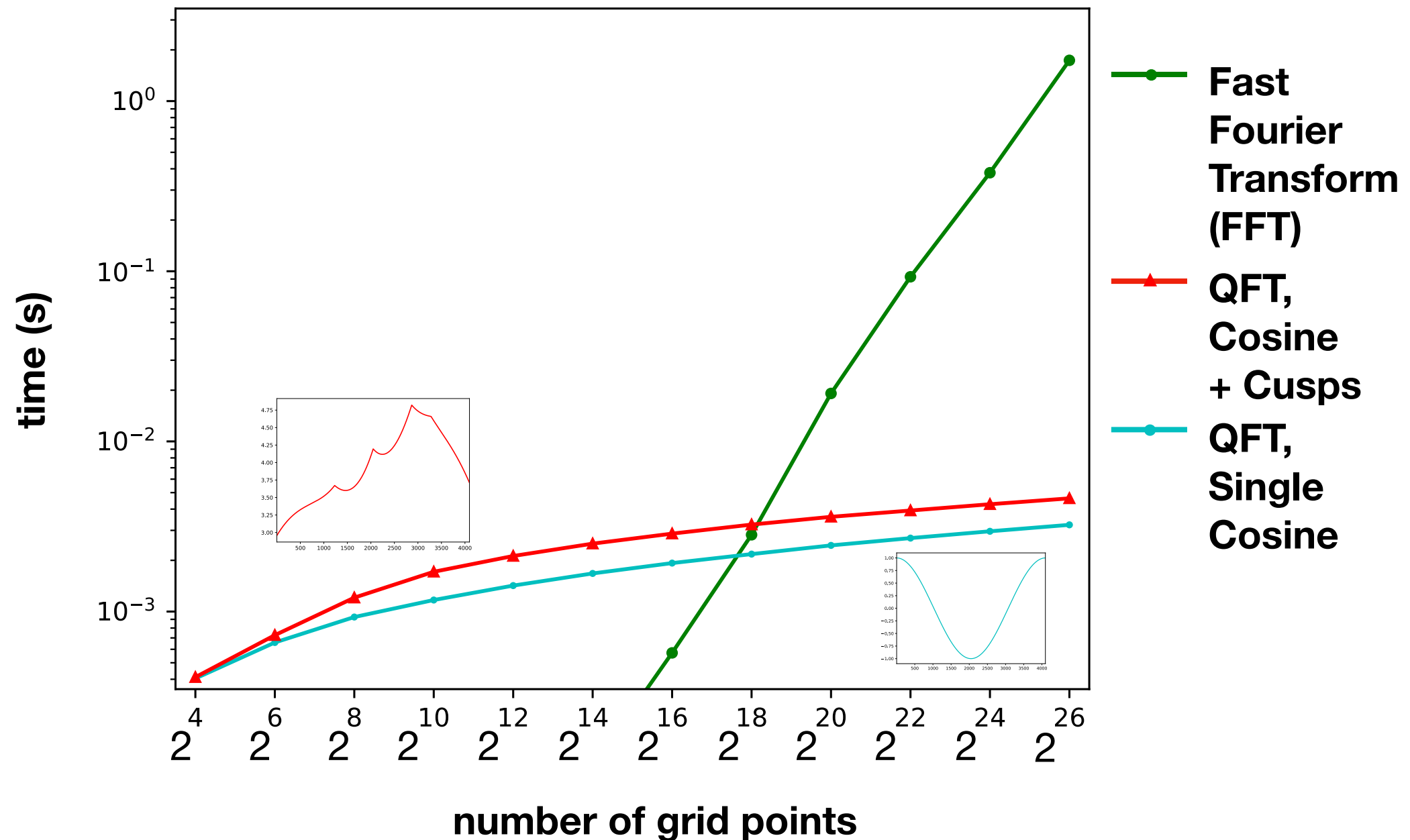


*discrete Fourier transform
using QFT*



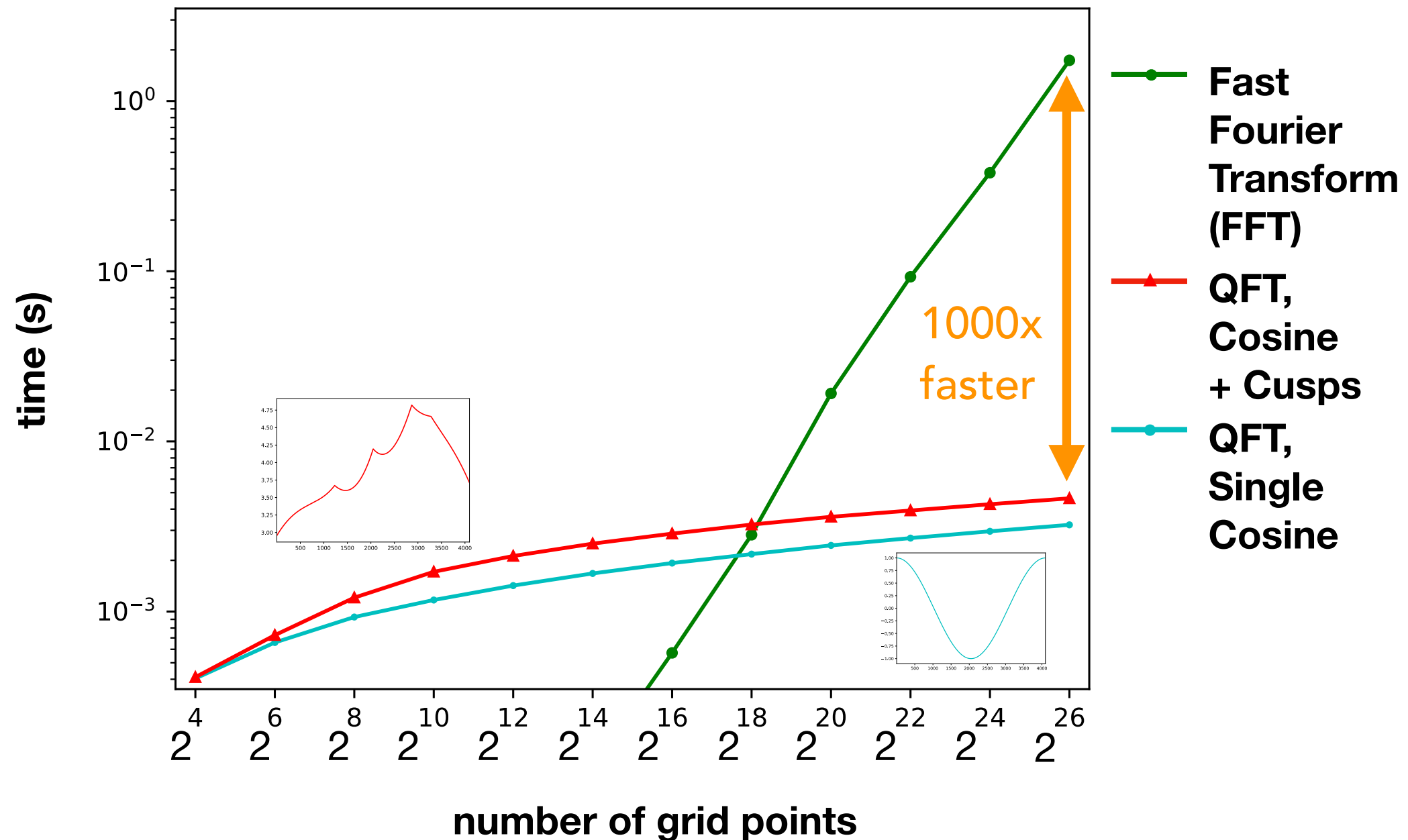
Quantum Fourier Transform

Compressed quantum Fourier transform can beat "fast Fourier transform" (FFT)



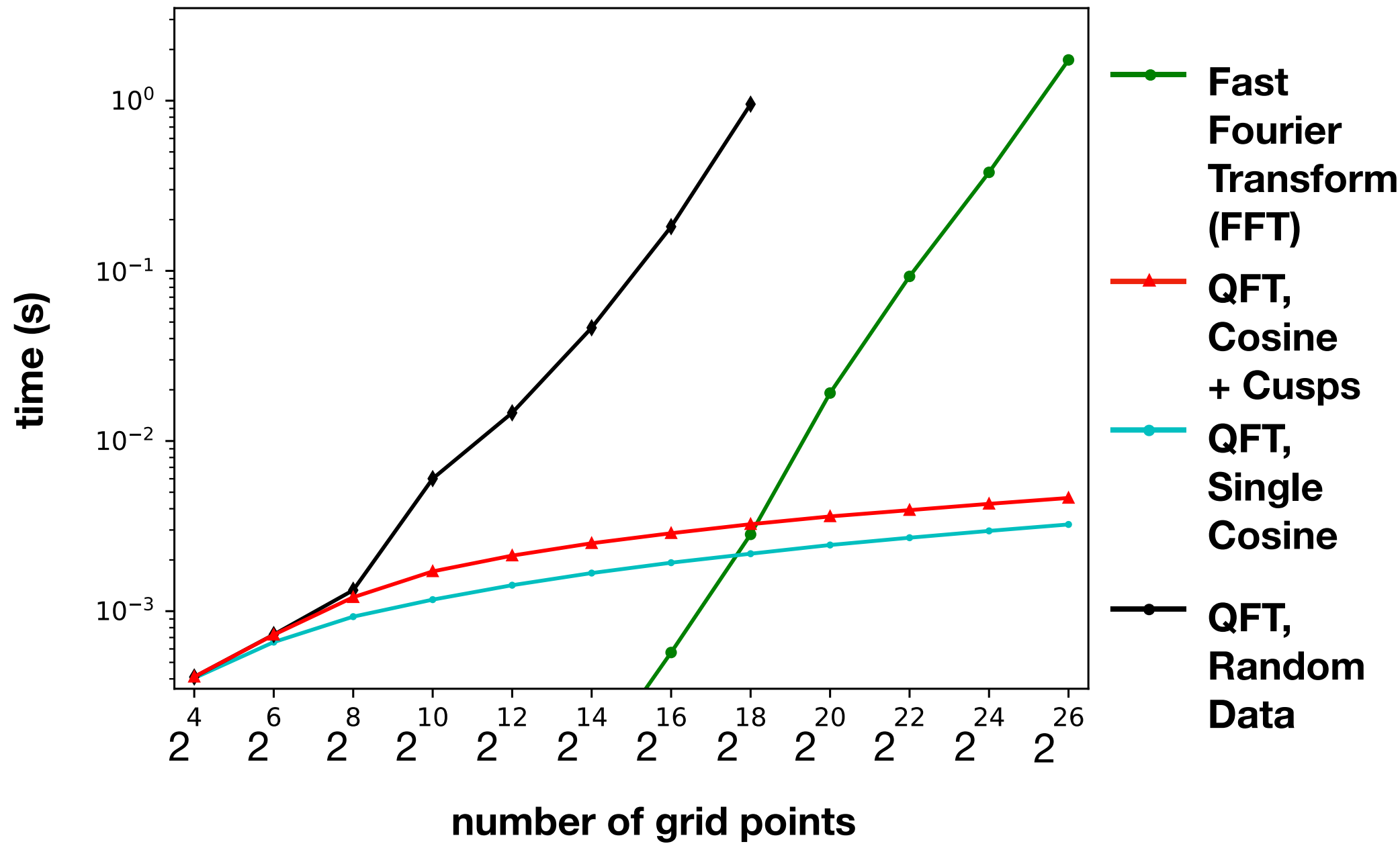
Quantum Fourier Transform

Compressed quantum Fourier transform can beat "fast Fourier transform" (FFT)



Quantum Fourier Transform

Though "no free lunch" – much worse for random data



Quantum-Inspired Algorithms

By now, many algorithms known for tensor network functions [1]

- active learning of functions [2]
- integration [3]
- solving partial differential equations [4,5,6,7]
- Fourier transform and convolutions [8,6,13]
- find extrema (global min & max) without gradients [9]
- interpolation [1] & extrapolation of functions [10]
- perfect sampling [11,12]

[1] Garcia-Ripoll, Quantum 5, 431 (2021)

[2] Dolgov, Savostyanov, Computer Physics Communications, 246, 106869 (2020)

[3] Khoromskij, Constr Approx 34, 257–280 (2011)

[4] Khoromskij, arxiv:1408.4053

[5] Gourianov et al., Nature Computational Science 2, 30–37 (2022)

[6] Shinaoka et al., Phys. Rev. X 13, 021015 (2023)

[7] Ye, Loureiro, Phys. Rev. E 106, 035208 (2022)

[8] Chen, EMS, White, PRX Quantum, arxiv:2210.08468

[9] Chertkov et al., arxiv:2209.14808

[10] Lin, White, arxiv:2308.09001

[11] White, EMS, New J. Phys 12, 055026 (2010)

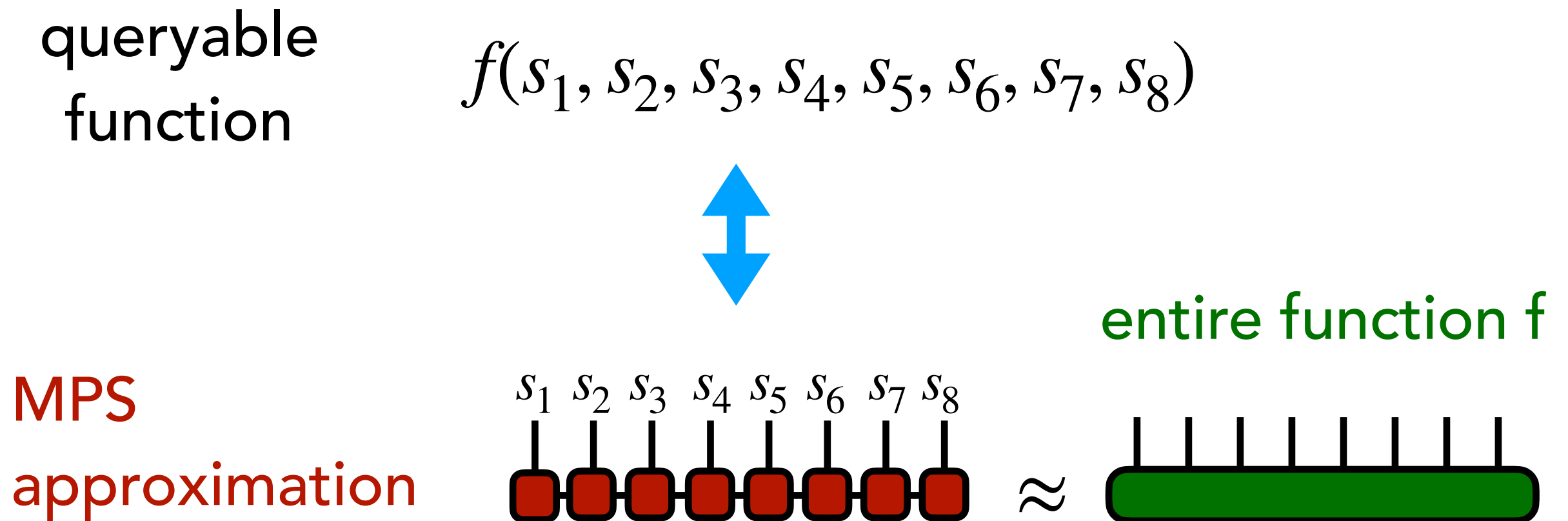
[12] Ferris, Vidal, Phys. Rev. B 85, 165146 (2012)

[13] Chen, Lindsey, arxiv:2404.03182 (2024)

Tensor Cross Interpolation

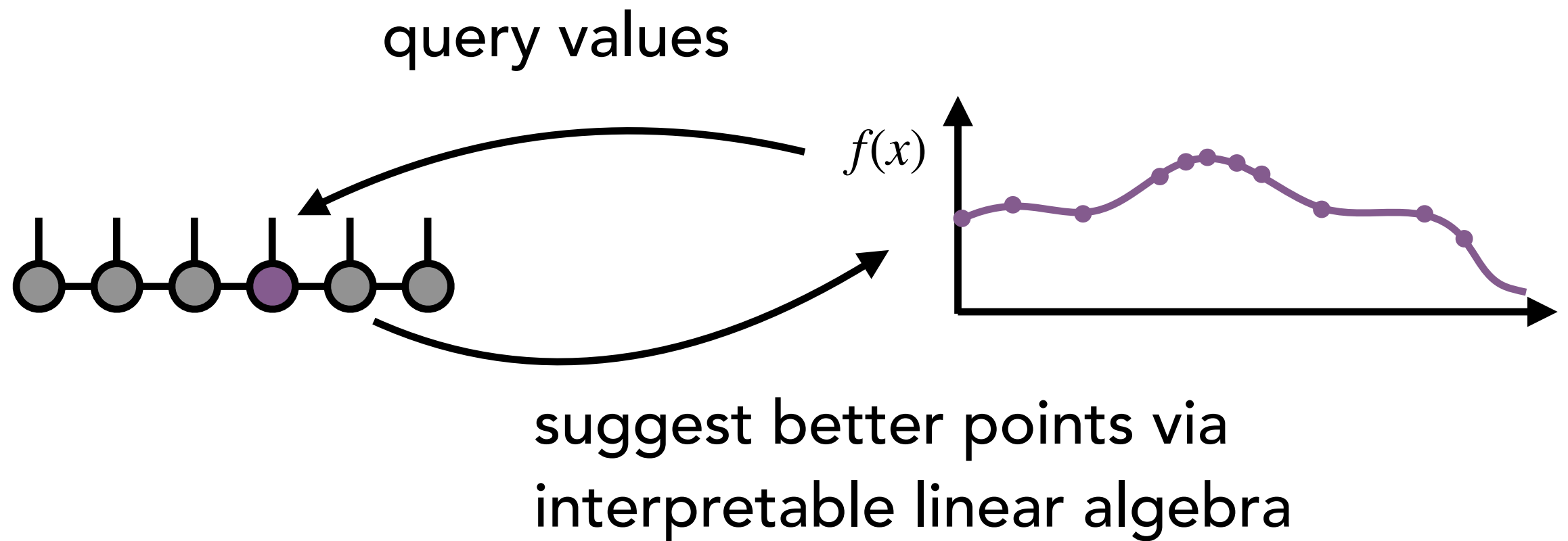
How do we get an complicated function into a tensor network?

Using tensor cross interpolation algorithm



Tensor Cross Interpolation

It is an "active learning" algorithm

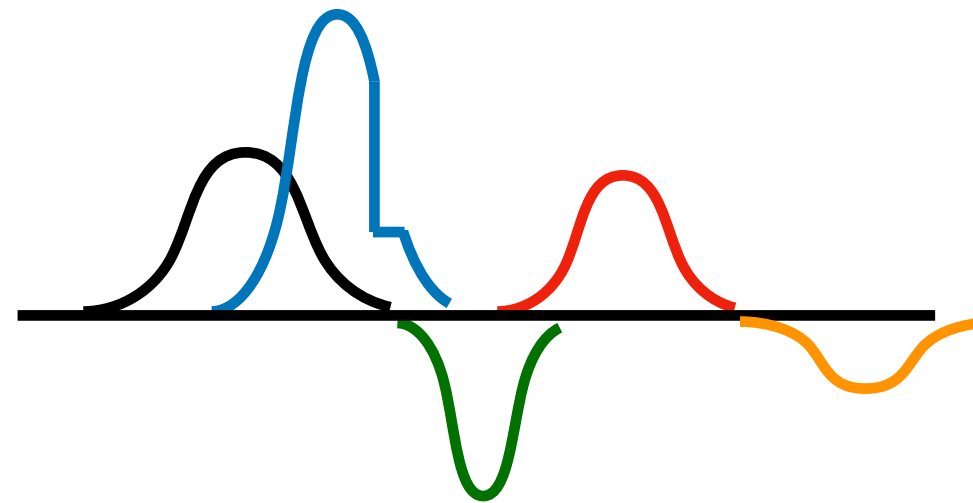


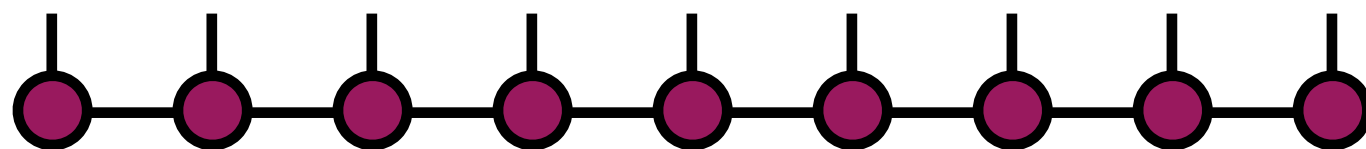
Tensor Cross Interpolation

Live demo – learning a 1D function:

40 Gaussians, random location, width, & height
+ a sharp step at 0.4

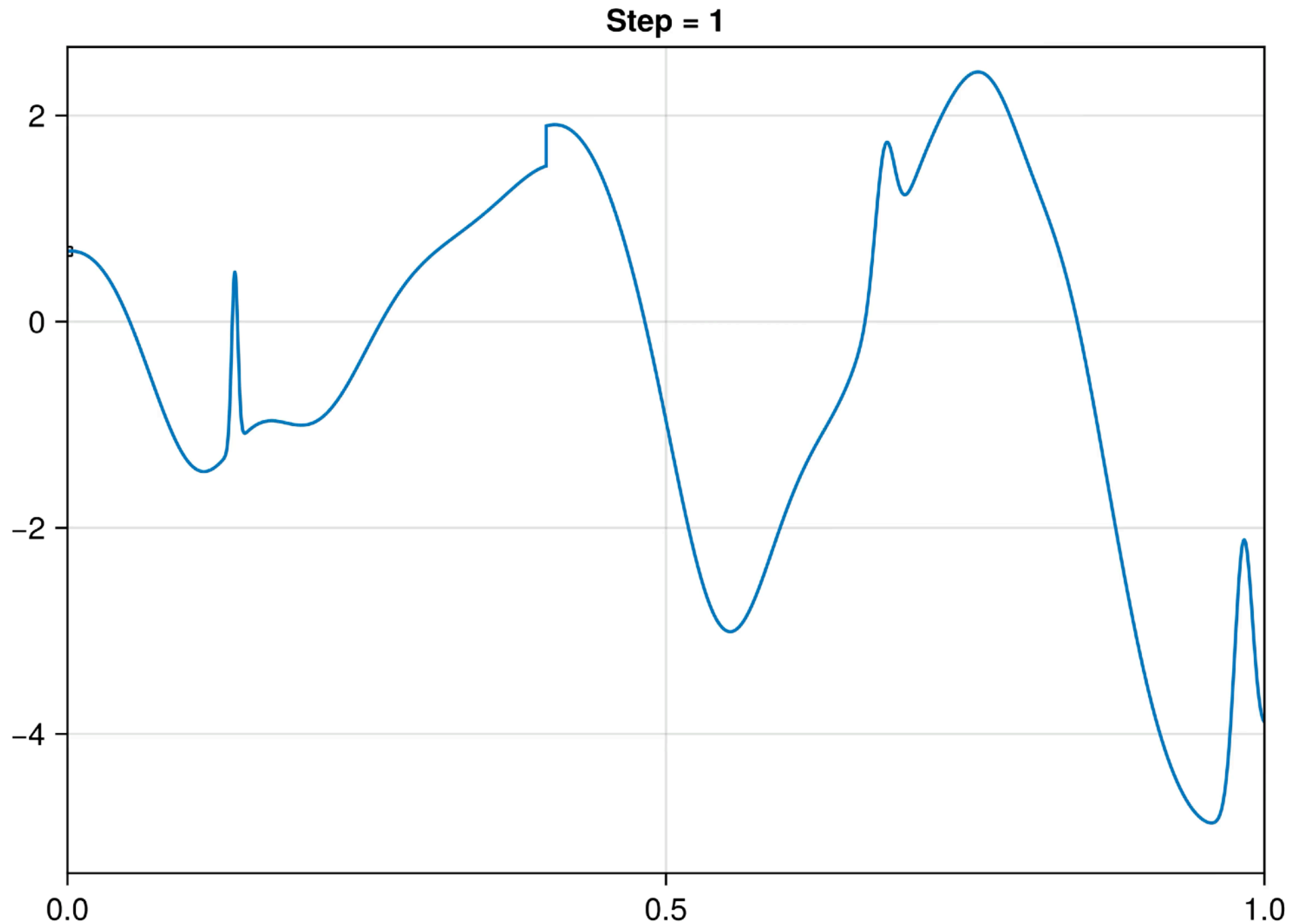
$$f(x) = \sum_{g=1}^{N_g} a_g e^{-w_g (x-x_g)^2} + 0.4 \cdot \Theta(x)$$



$$f(x) \approx$$


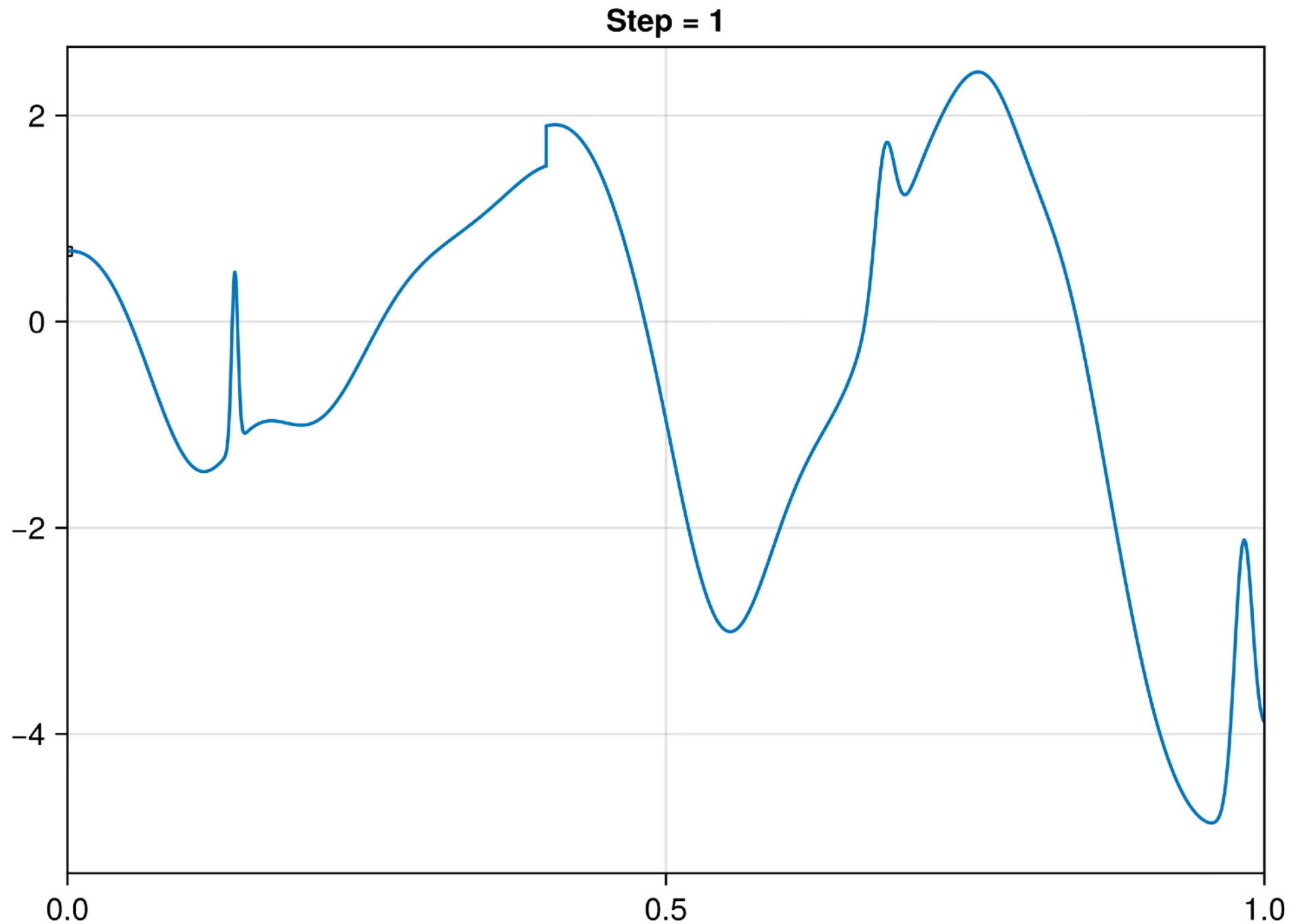
Tensor Network Algorithms

Demo of tensor cross interpolation (TCI)
for learning a function (sum of 40 Gaussians)



Tensor Network Algorithms

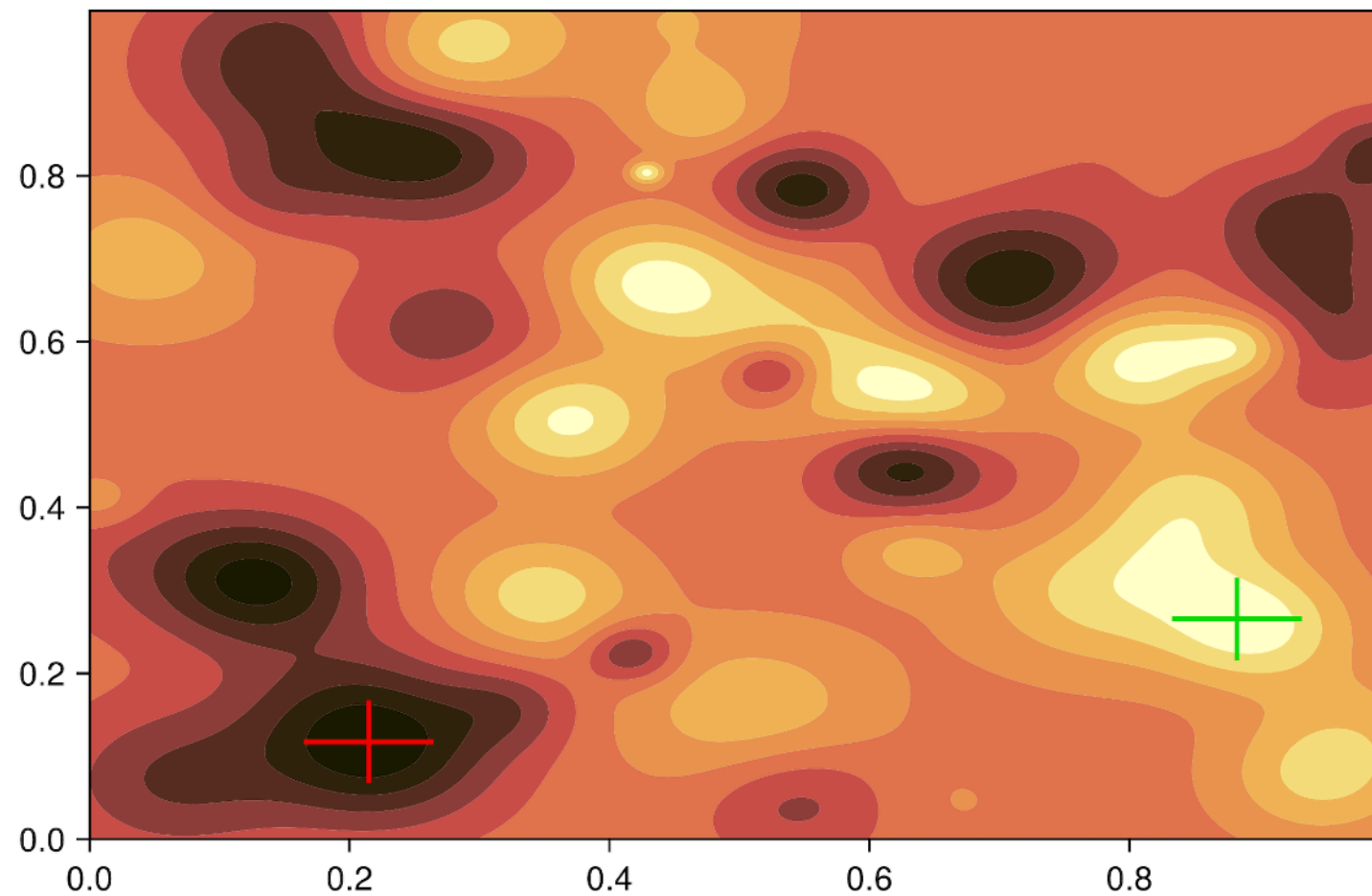
Demo of tensor cross interpolation (TCI)
for learning a function (sum of 40 Gaussians)



Tensor Cross Interpolation

Next demo: 2D function and optima

50 2D Gaussians, random location, width, & height



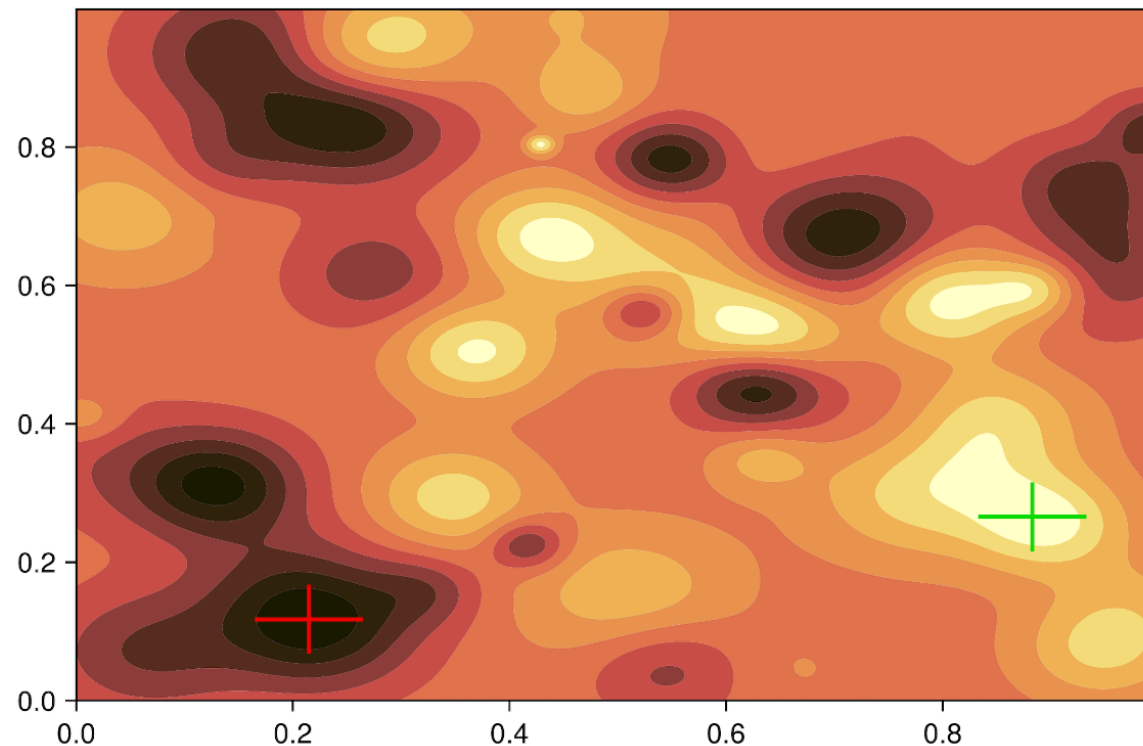
$$f(x) \approx$$

Tensor Cross Interpolation

How were optima (global min and max) found?

Very interesting proposal called "TTOpt" [1,2]

Claim: tensor interpolation usually
"encounters" global min and max [1]



[1] Sozykin, Chertkov, Schutski, Phan, Cichocki, Oseledets, TTOpt, NeurIPS (2022)

[2] Chertkov, Ryzhakov, Novikov, Oseledets, arxiv:2209.14808

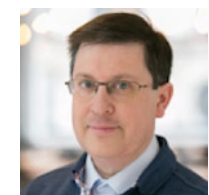
Tensor Cross Interpolation



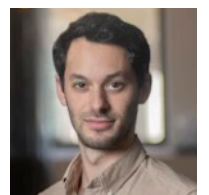
Nunez-
Fernandez



Waintal

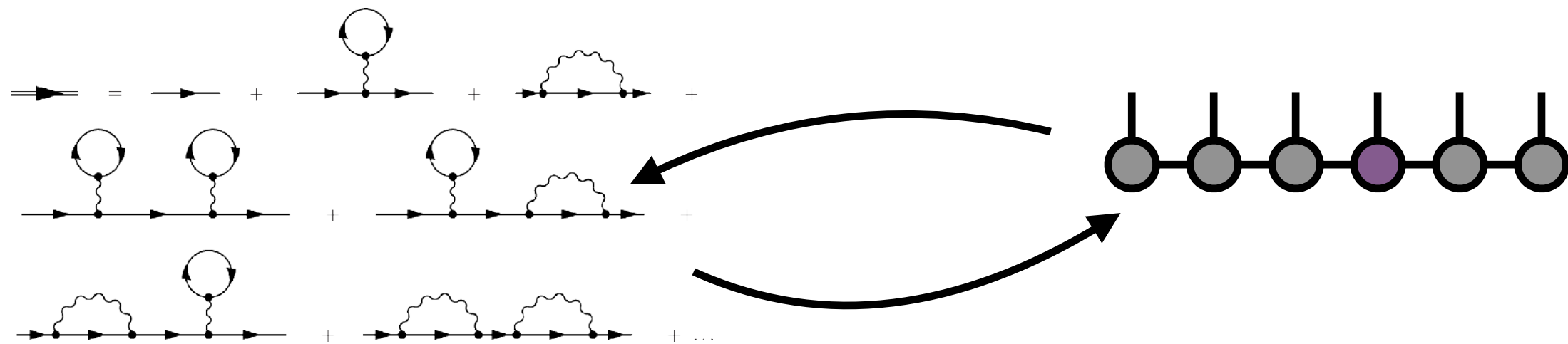


Parcollet



Kaye

TCI method used to
learn and sum entire orders of Feynman diagrams*

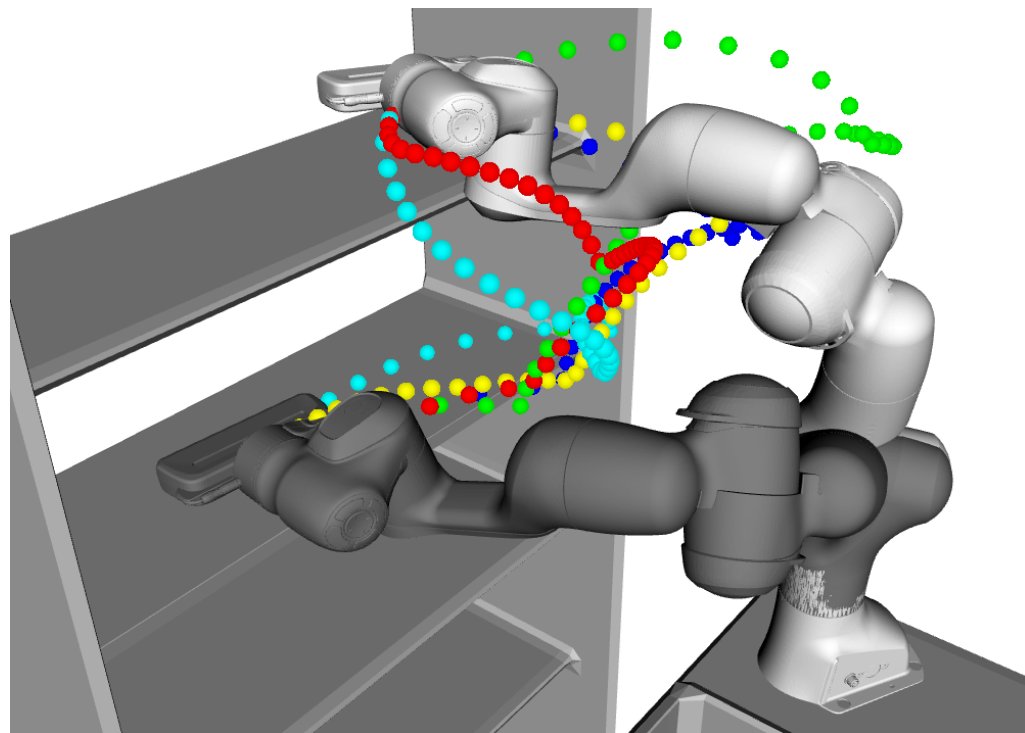


Solve quantum dots to long times, as post-processing

* Nunez Fernandez, et al. Phys. Rev. X 12, 041018 (2022)

Tensor Cross Interpolation

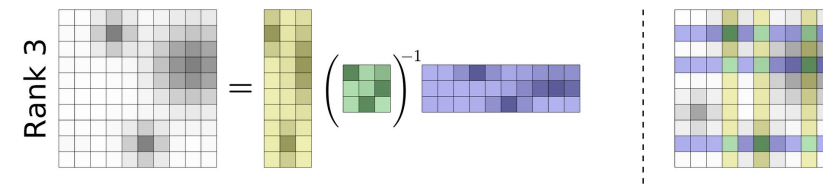
Fun paper using TTOpt to control a real robot arm! *



(a)

Figure 1. Solutions from TTGO for motion planning of a manipulator from a given initial configuration (white) to a final configuration (dark). The obtained joint angle trajectories result in different paths for the end.effector which are highlighted by dotted curves in different colors. The multimodality is clearly visible from these solutions.

TCI (= TT-Cross) algorithm



$$\begin{aligned}
 & \mathcal{P} \in \mathbb{R}^{n_1 \times n_2 \times n_3 \times n_4} \\
 & \mathcal{P}^k \in \mathbb{R}^{r_{k-1} \times n_k \times r_k} \\
 & \mathcal{P}_{i_1, i_2, i_3, i_4} = 1 \times \mathcal{P}^1_{:, i_1, :} \times r_1 \times \mathcal{P}^2_{:, i_2, :} \times r_2 \times \mathcal{P}^3_{:, i_3, :} \times r_3 \times \mathcal{P}^4_{:, i_4, :}
 \end{aligned}$$

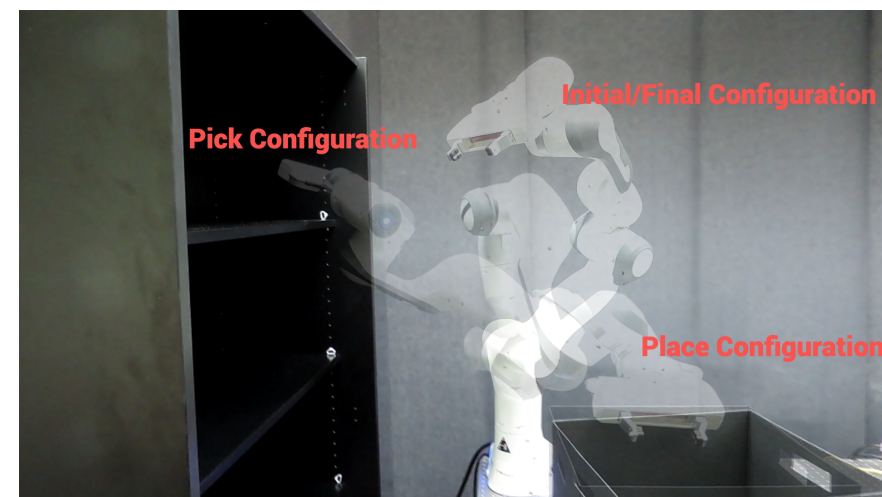
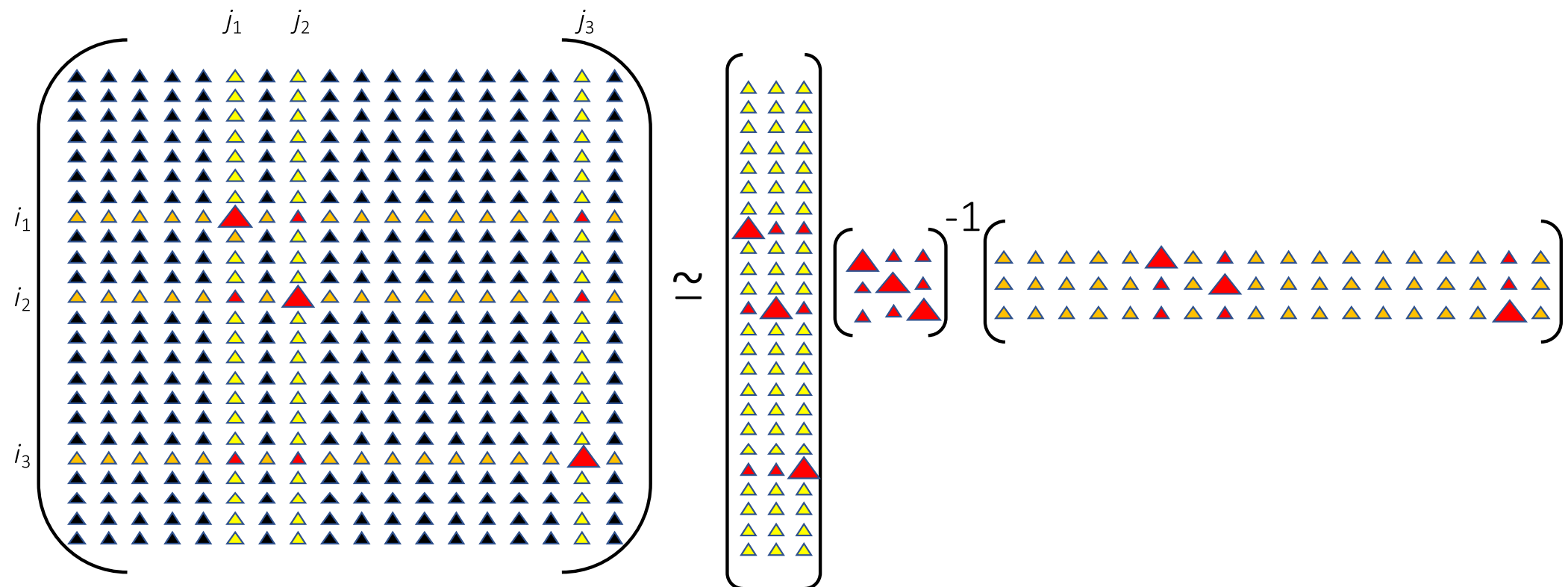


Figure 12. Real robot implementation of one of the TTGO solutions for the pick-and-place task. The motion from the initial configuration to the final configuration (same as the initial configuration in this case) via the picking configuration and placing configuration is depicted.

Tensor Cross Interpolation

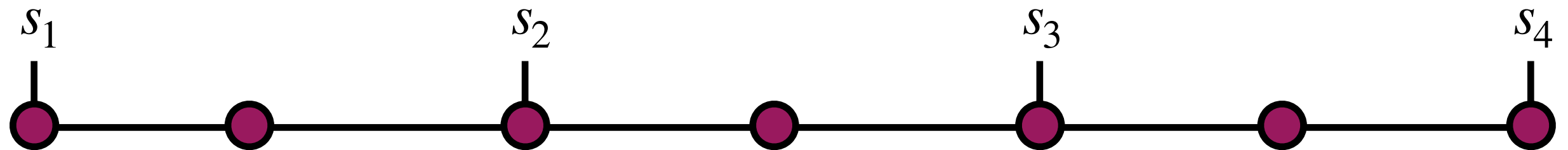
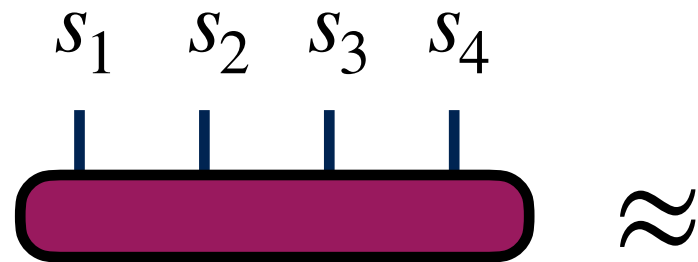
How does tensor cross interpolation work in a bit more detail?

Foundation is "cross approximation" of a matrix



Tensor Cross Interpolation

Tensor cross interpolation extends this to
MPS tensor network

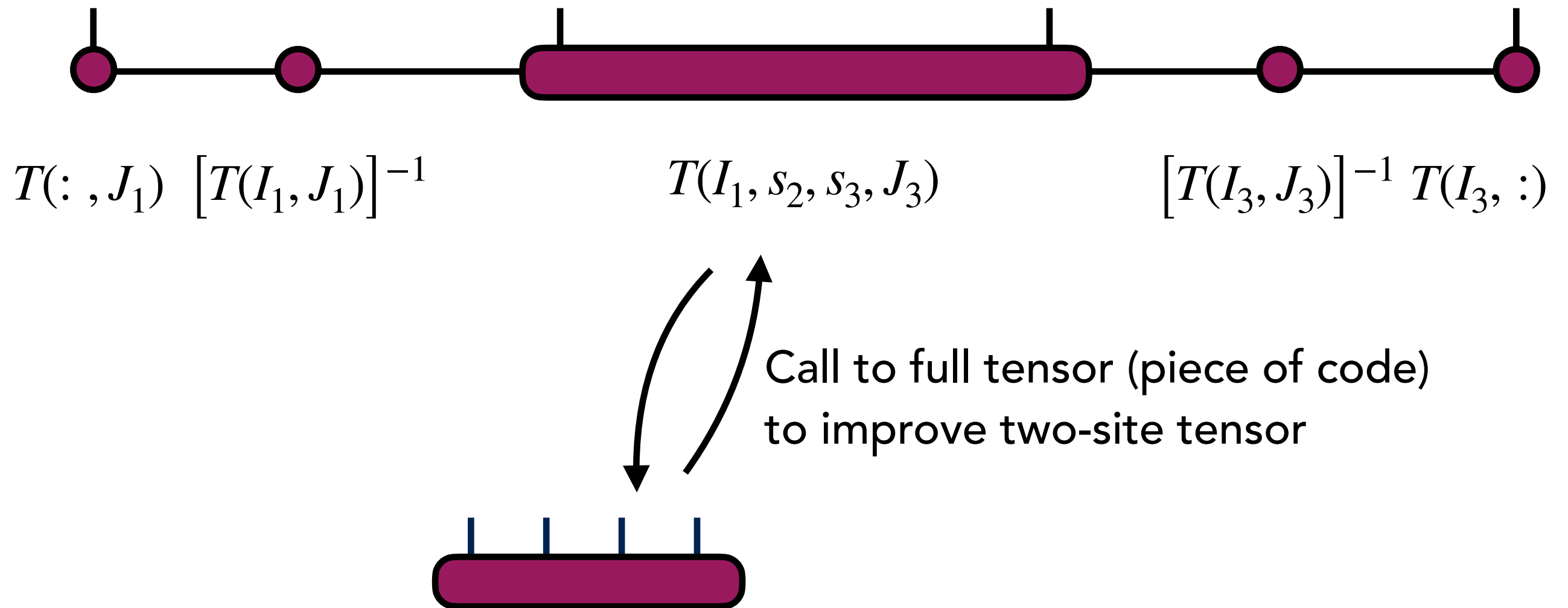


$$T(s_1, J_1) [T(I_1, J_1)]^{-1} T(I_1, s_2, J_2) [T(I_2, J_2)]^{-1} T(I_2, s_3, J_3) [T(I_3, J_3)]^{-1} T(I_3, s_4)$$

Entries of small tensor are exactly entries of big tensor
Interpolation points or pivots

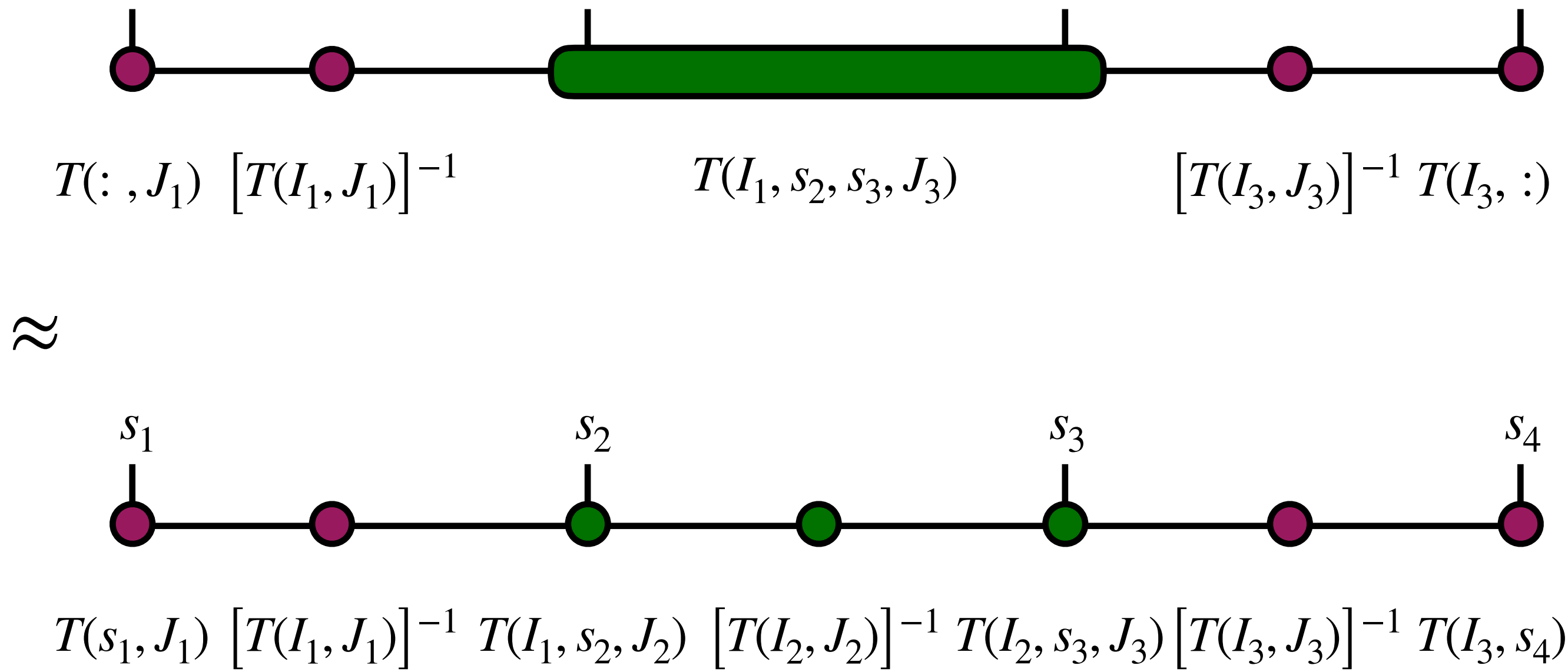
Tensor Cross Interpolation

Tensor cross interpolation proceeds by merging pairs of small tensors then improving pivots



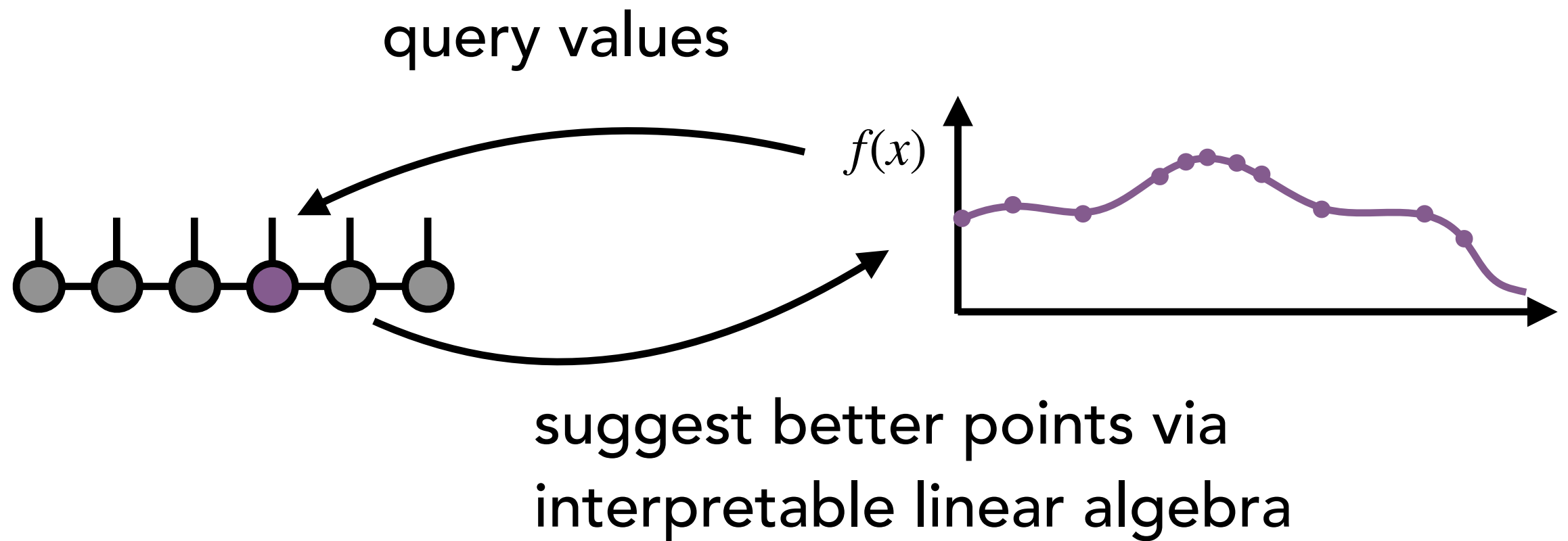
Tensor Cross Interpolation

Tensor cross interpolation proceeds by merging pairs of small tensors then improving pivots



Tensor Cross Interpolation

Result is an "active learning" algorithm



Tensor Cross Interpolation

A leading TCI code is by Marc Ritter (**LMU** → **CCQ**)

<https://github.com/tensor4all/TensorCrossInterpolation.jl>

TensorCrossInterpolation

docs dev CI passing

The [TensorCrossInterpolation module](#) implements the *tensor cross interpolation* algorithm for efficient interpolation of multi-index tensors and multivariate functions.

This algorithm is used in the *quantics tensor cross interpolation* (QTCI) method for exponentially efficient interpolation of functions with scale separation. QTCI is implemented in the [QuanticsTCI.jl](#) module.

Installation

This module has been registered in the General registry. It can be installed by typing the following in a Julia REPL:

```
using Pkg; Pkg.add("TensorCrossInterpolation")
```



Results easy to convert to an ITensor MPS

Tensor Recursive Sketching



Yoonhaeng
Hur



Jeremy
Hoskins

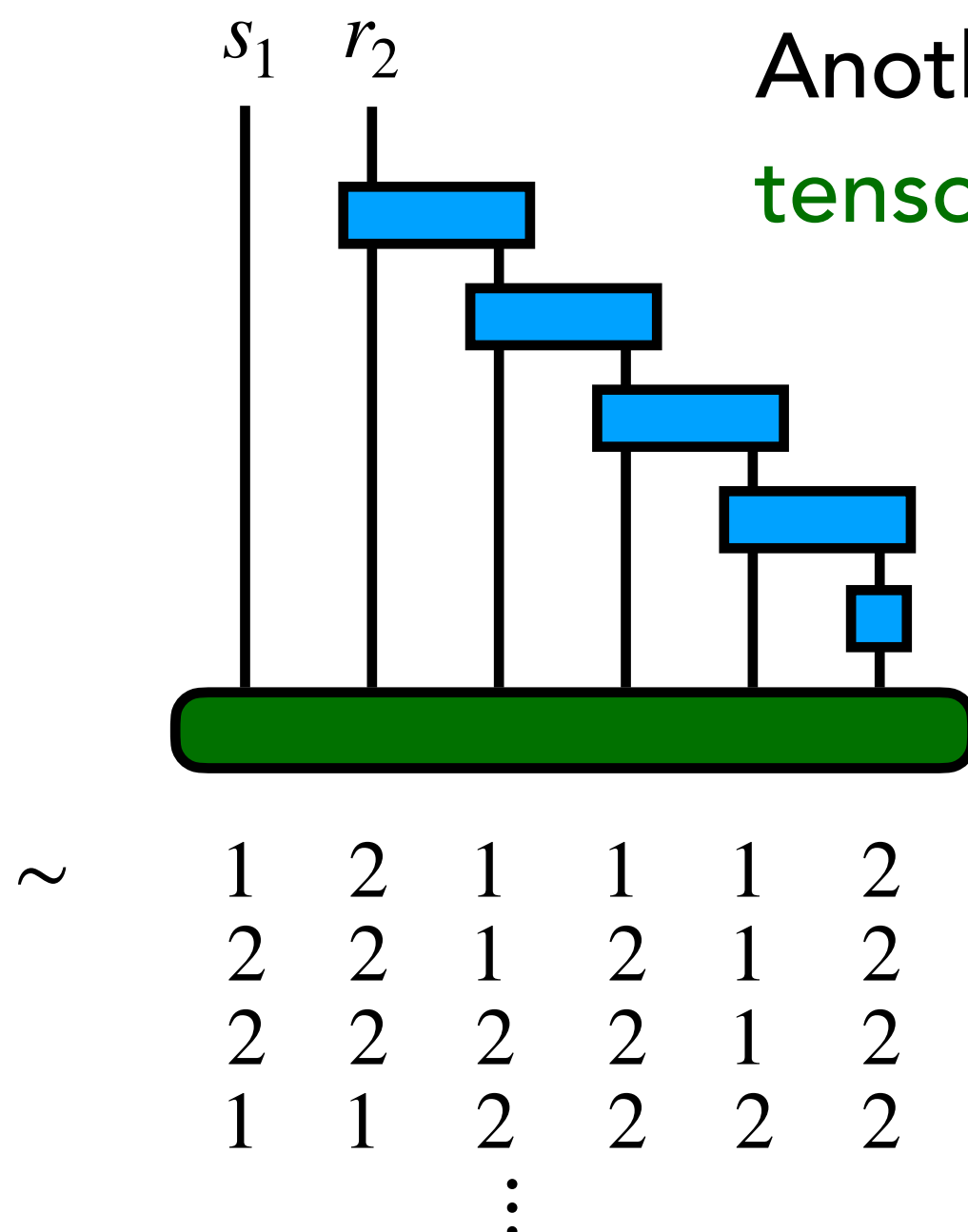


Michael
Lindsey

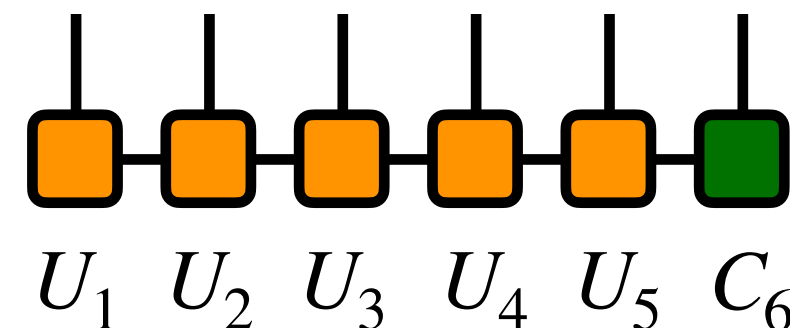


Yuehaw
Khoo

Can we also learn a tensorized function from data / samples? Yes!



Another new M.L. algorithm is
tensor train recursive sketching [1,2]

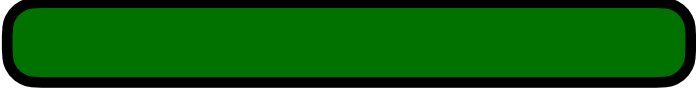


[1] Generative Modeling via Tensor Train Sketching, arxiv: 2202.11788

[2] Generative Modeling via Hierarchical Tensor Sketching, arxiv: 2304.05305

Tensor Network Machine Learning

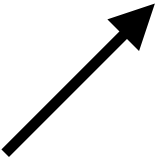
Given samples from true distribution

$$p(s_1, s_2, \dots, s_N) =$$

$$\sim$$

1	2	1	1	1	2	2	2
2	2	1	2	1	2	1	2
2	2	2	2	1	2	1	1
1	1	2	2	2	2	1	2

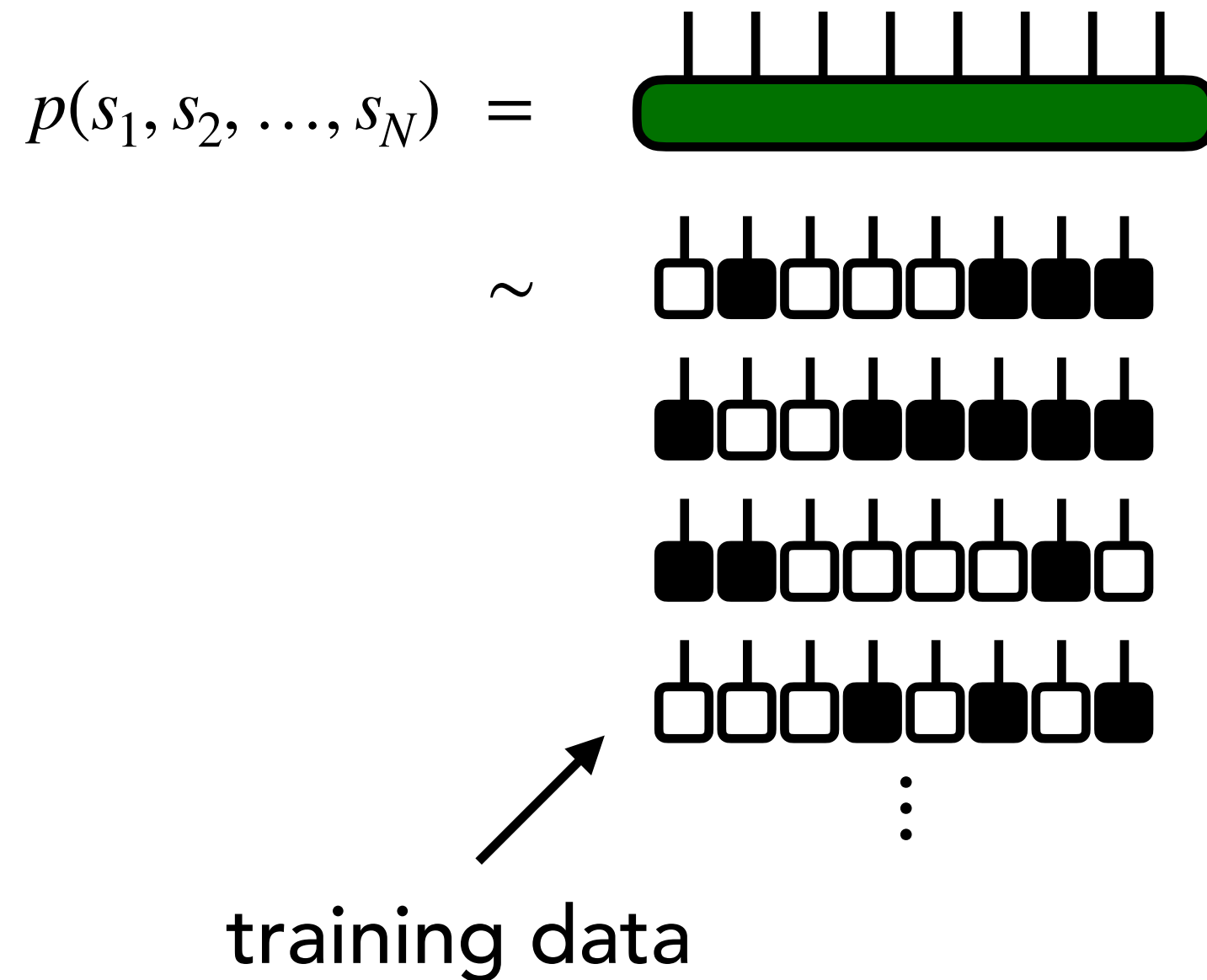
⋮

training data



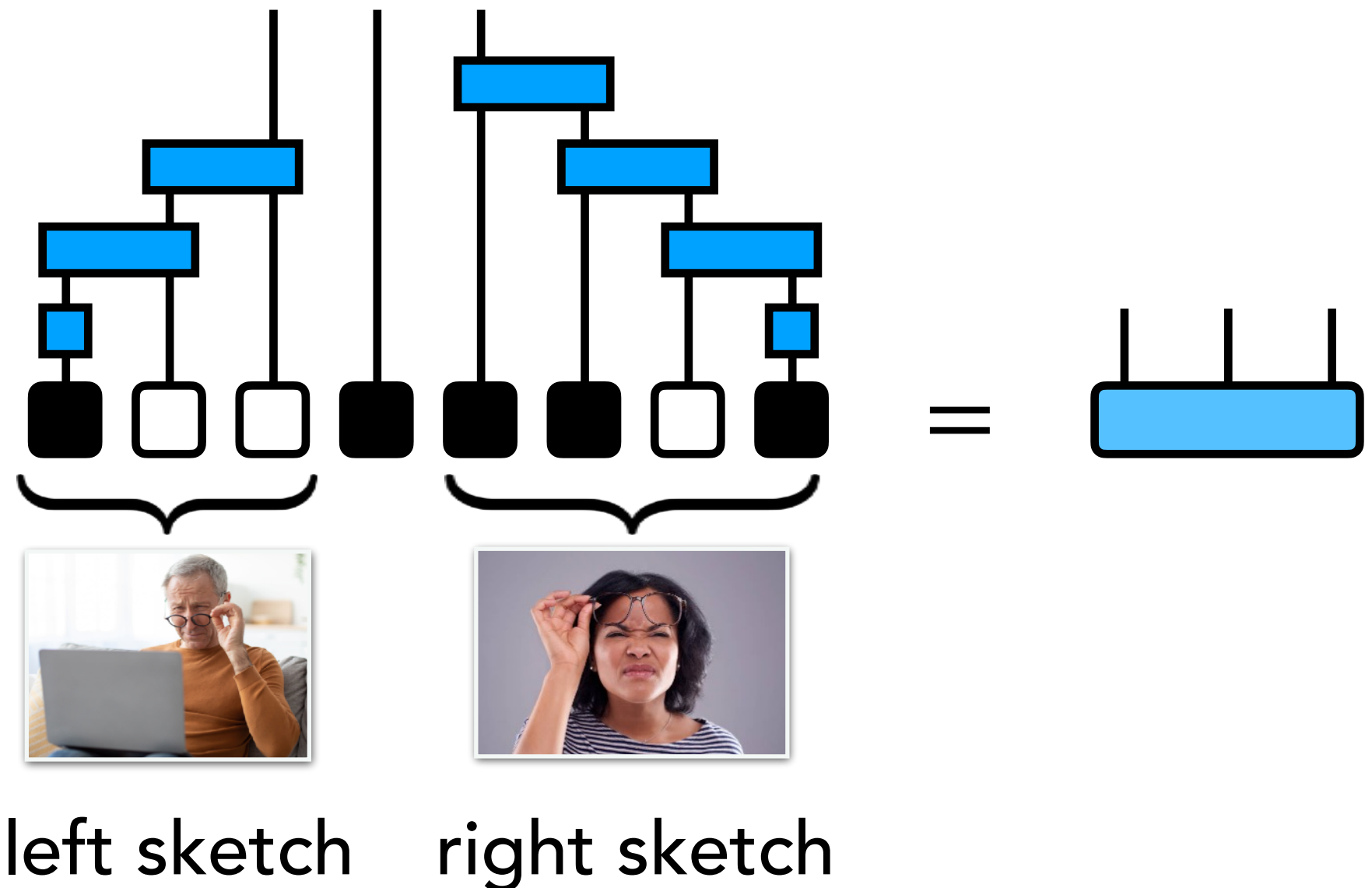
Tensor Network Machine Learning

Given samples from true distribution



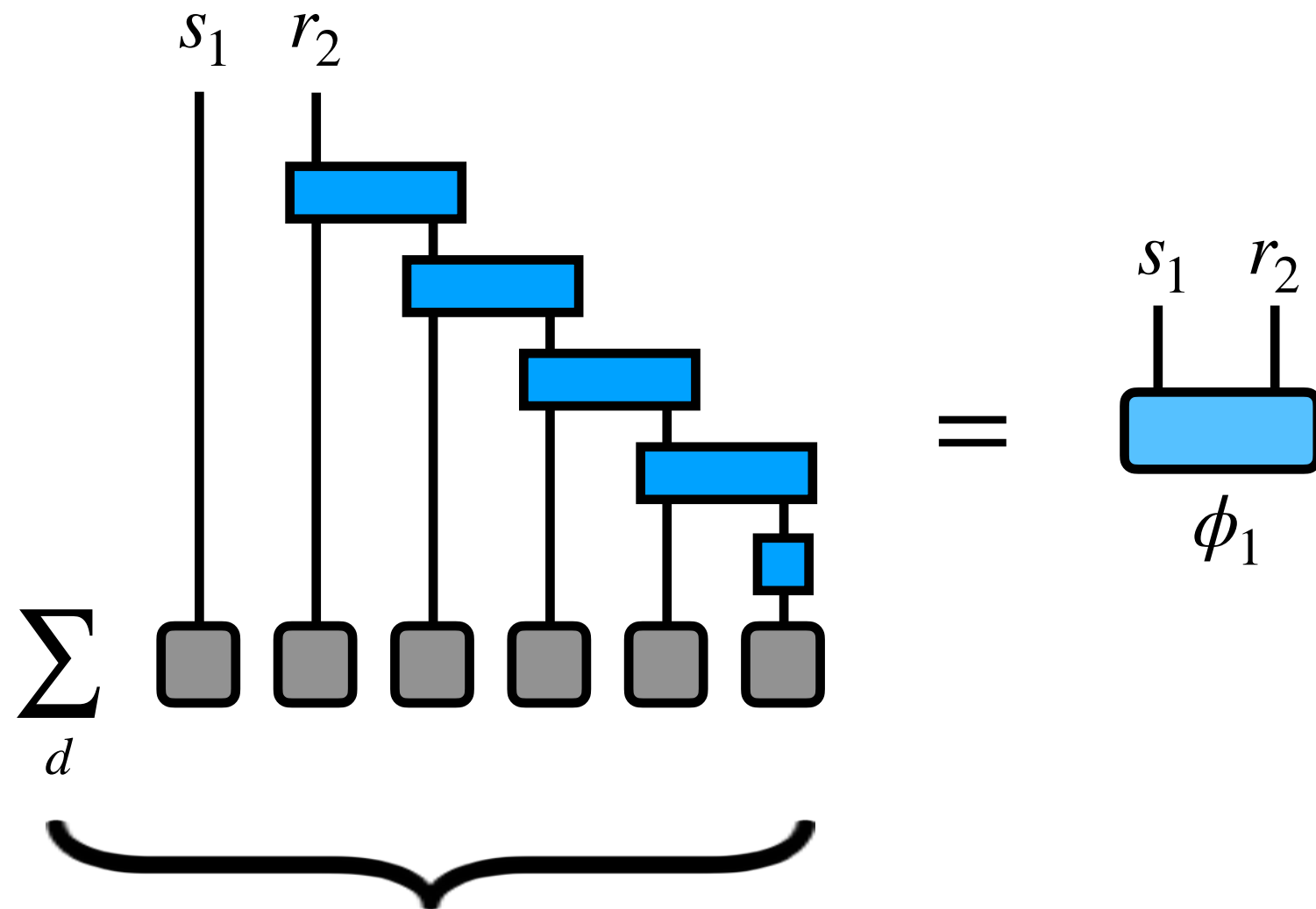
Tensor Network Machine Learning

Apply empirical **sketches** to "blur" & project data to lower-dimensional space



Tensor Network Machine Learning

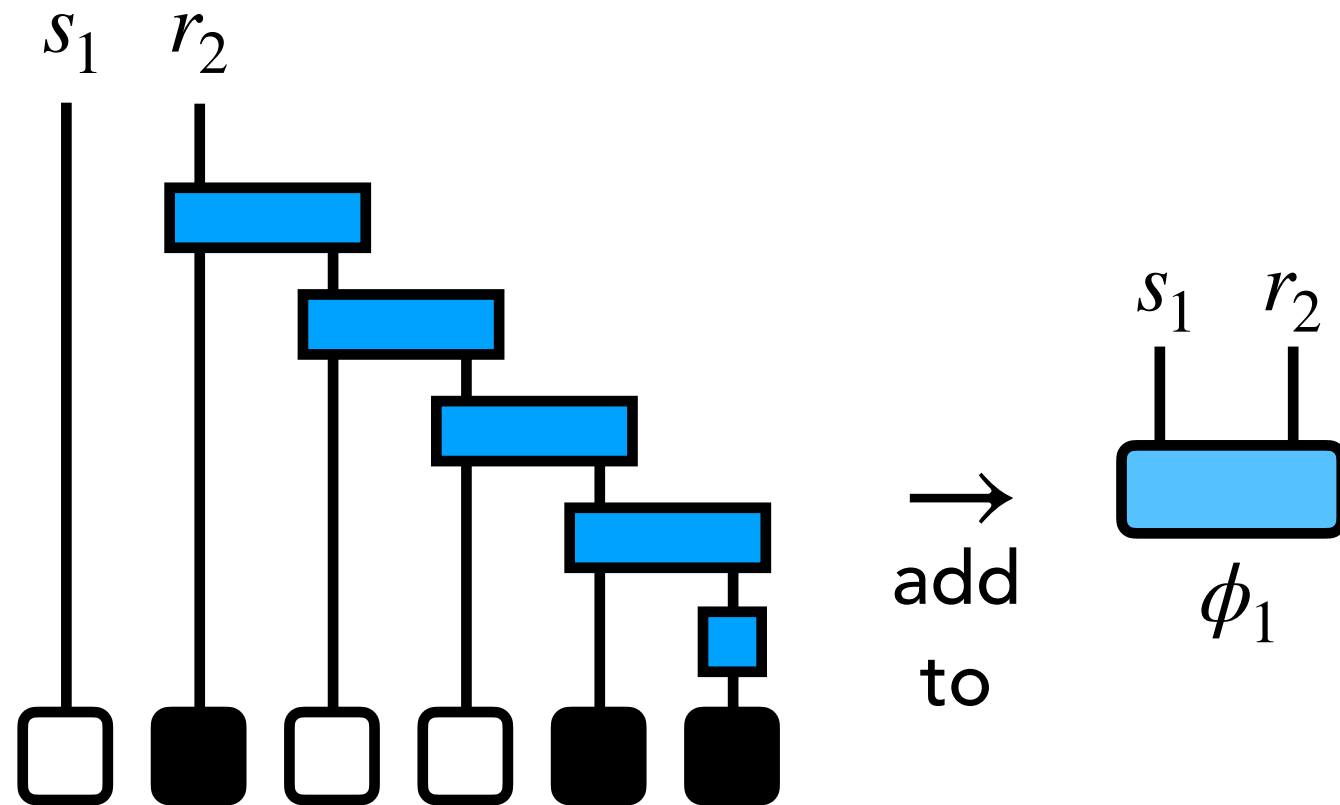
Sum up data through sketching process



$\hat{p}$, sum over all data in training set
(product states / rank-1 tensors)

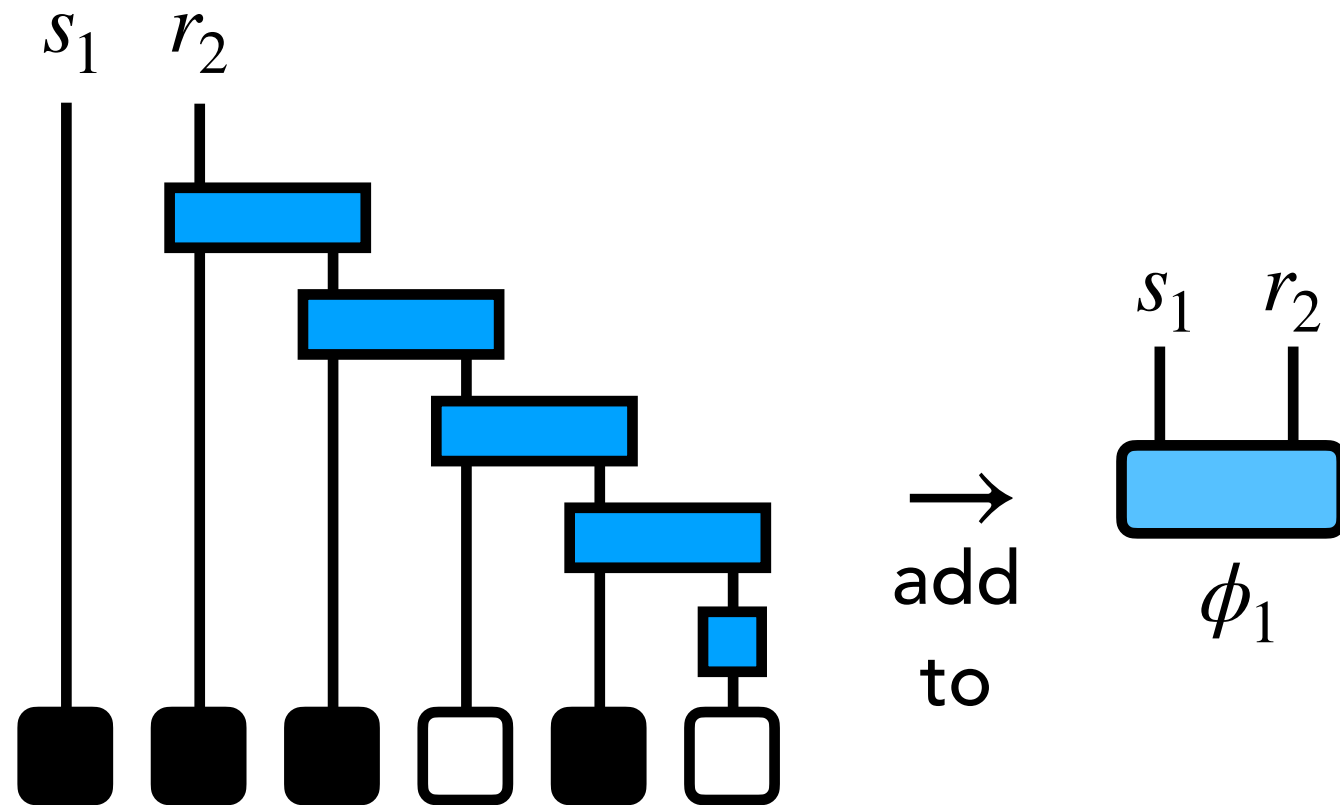
Tensor Network Machine Learning

Sum up data through sketching process



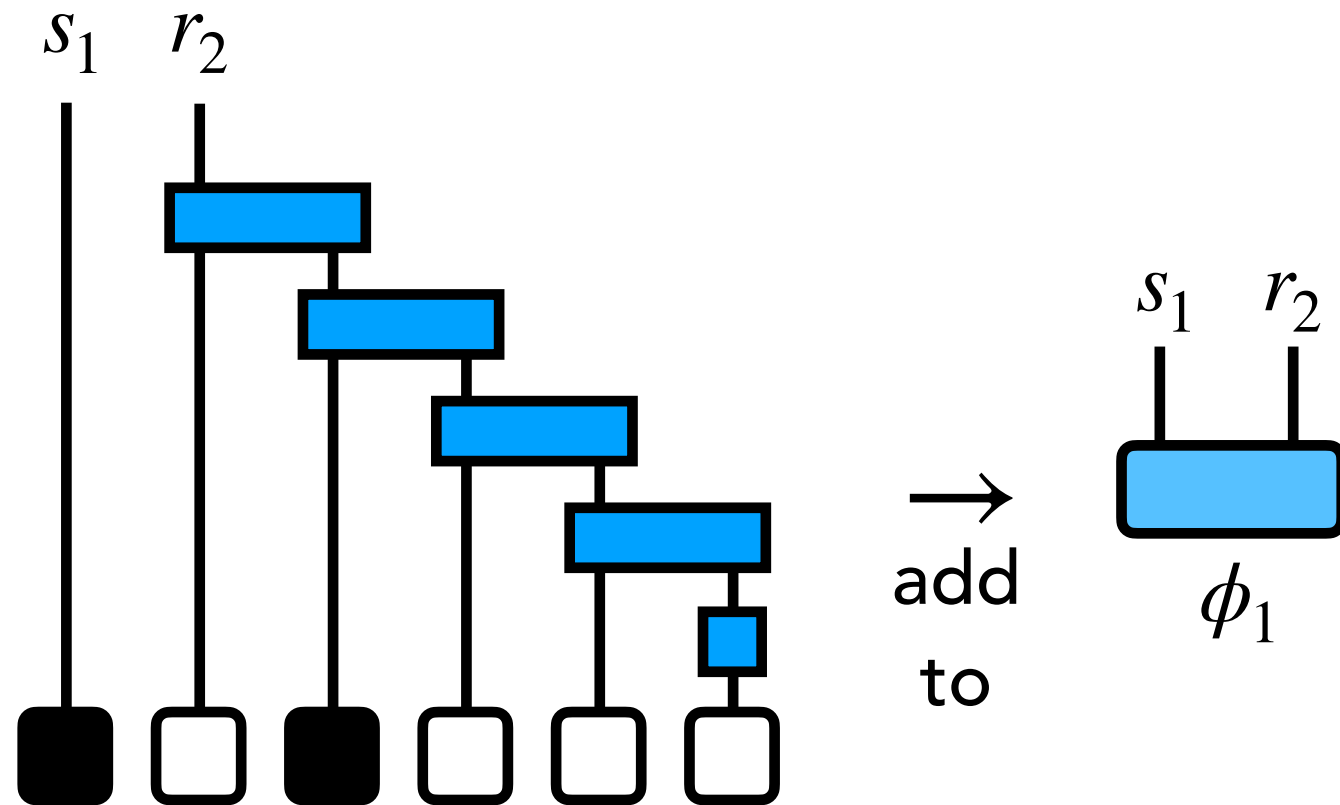
Tensor Network Machine Learning

Sum up data through sketching process



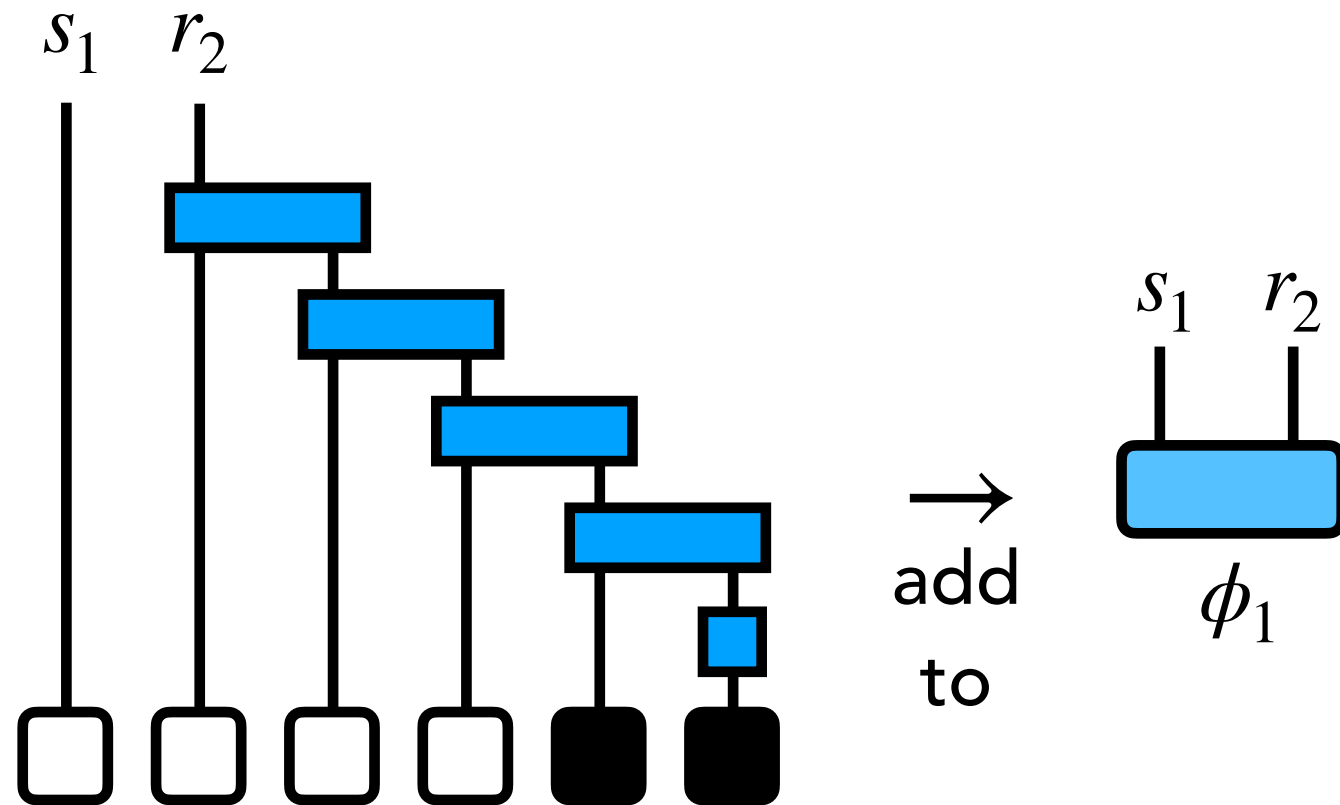
Tensor Network Machine Learning

Sum up data through sketching process



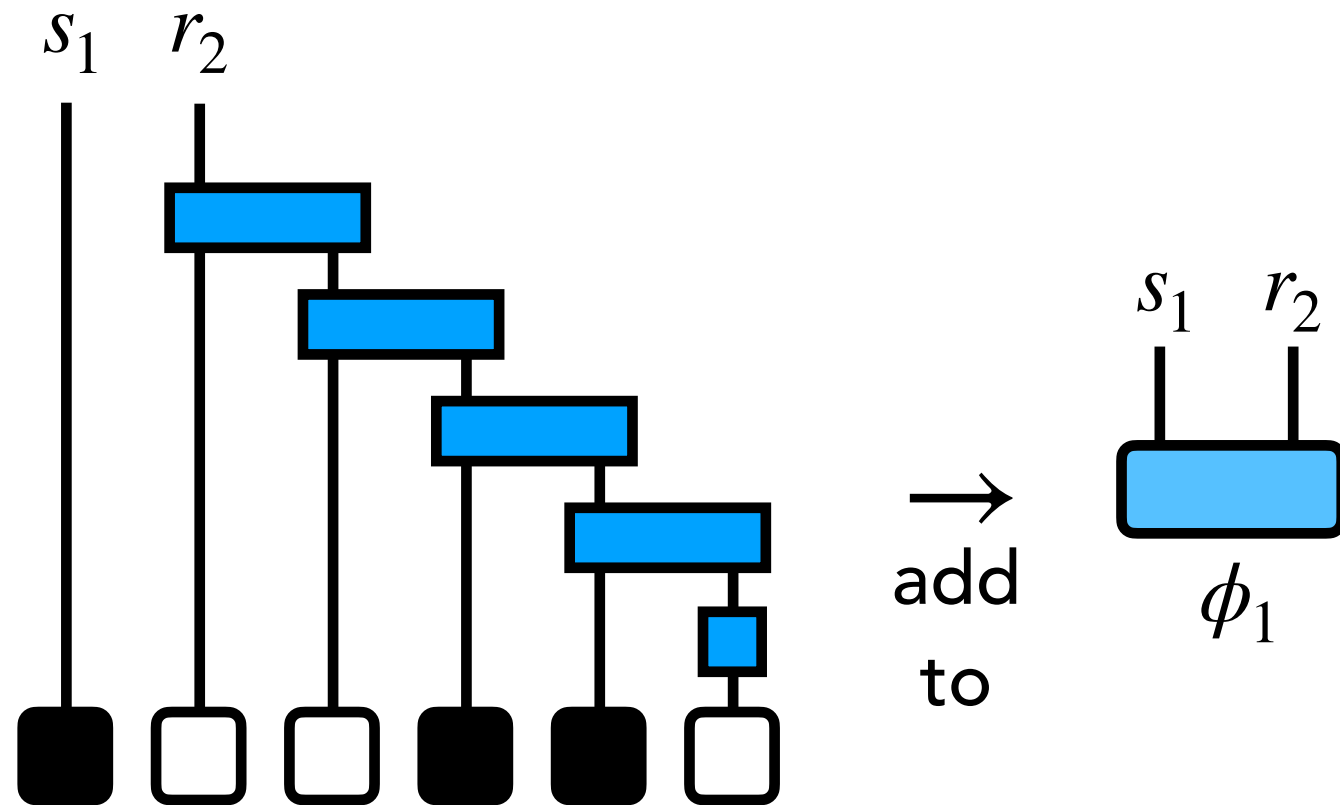
Tensor Network Machine Learning

Sum up data through sketching process



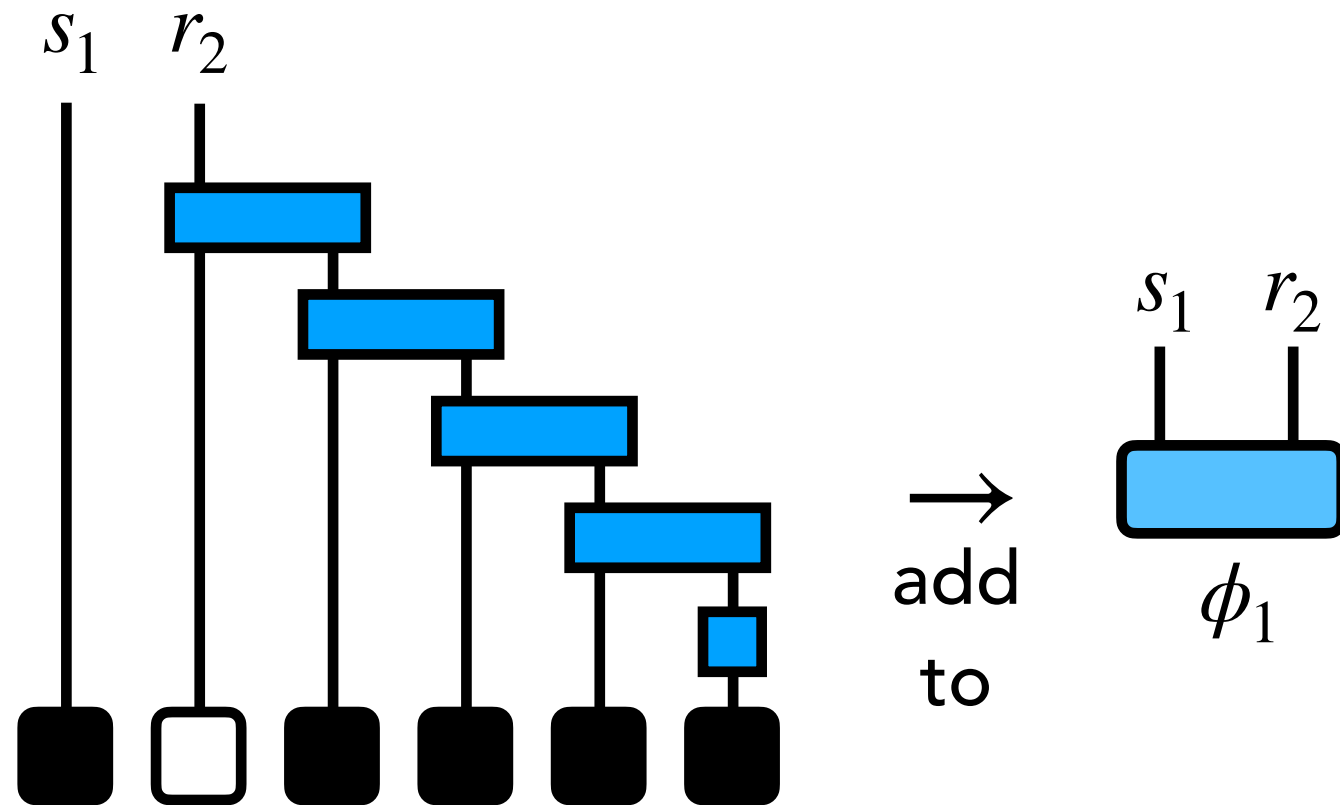
Tensor Network Machine Learning

Sum up data through sketching process



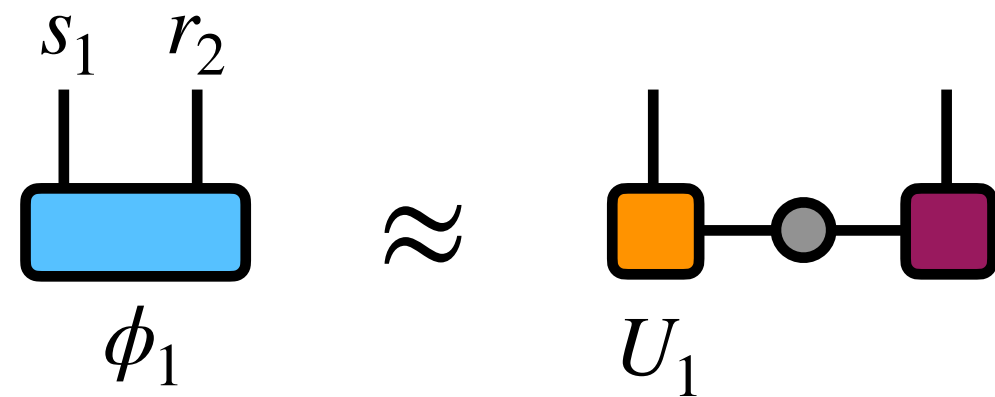
Tensor Network Machine Learning

Sum up data through sketching process

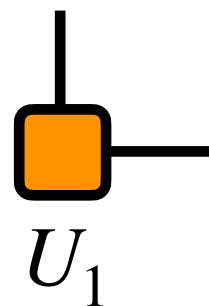


Tensor Network Machine Learning

SVD of reduced data tensor gives
first MPS tensor U_1

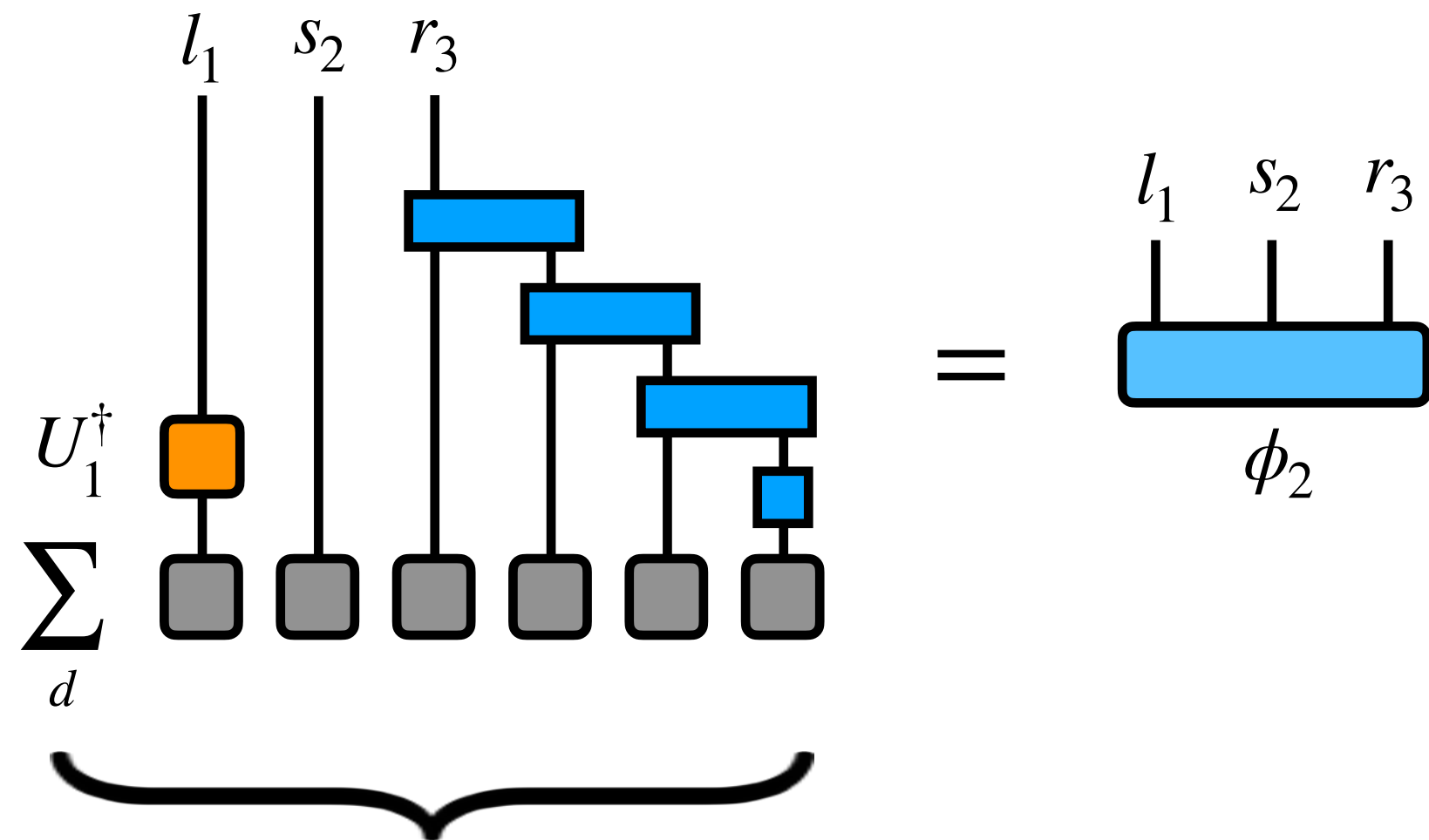


Save U_1 only



Tensor Network Machine Learning

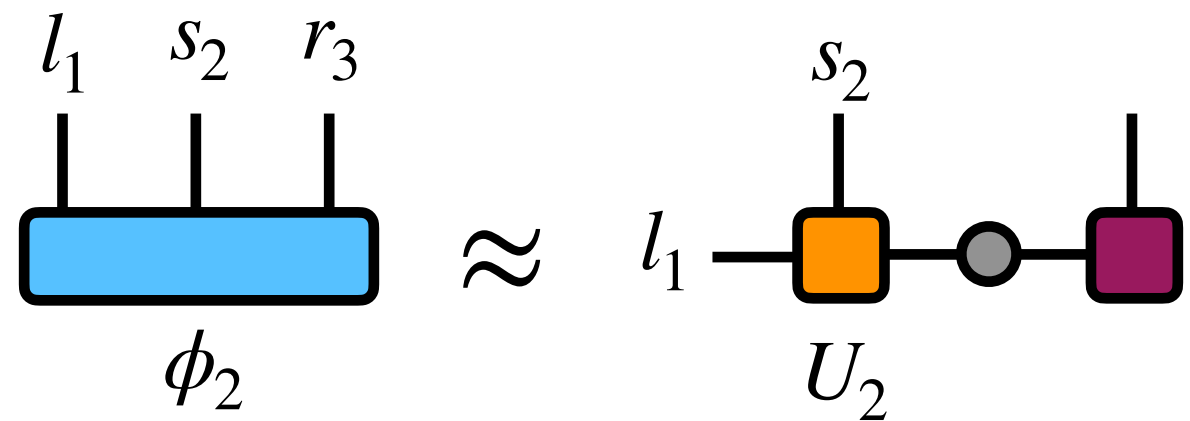
Recursively summing and sketching data continues process



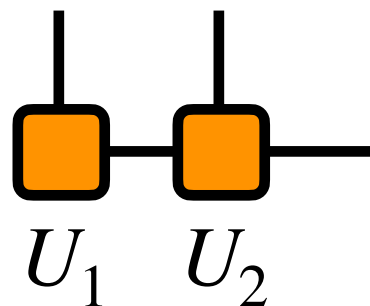
$\hat{p}$, sum over all data in training set
(product states / rank-1 tensors)

Tensor Network Machine Learning

SVD of reduced data tensor gives
second MPS tensor U_2

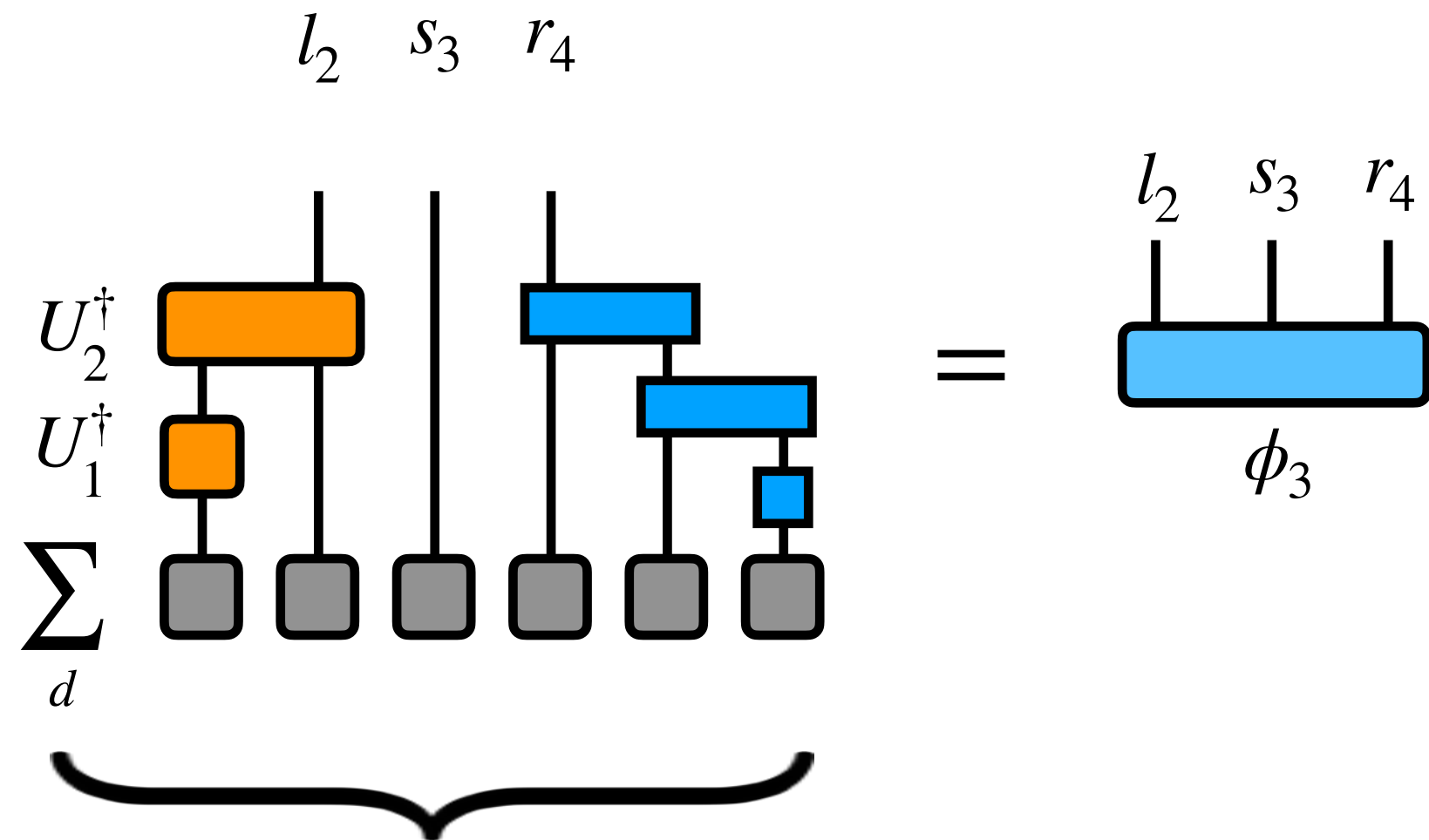


Save U_2 only



Tensor Network Machine Learning

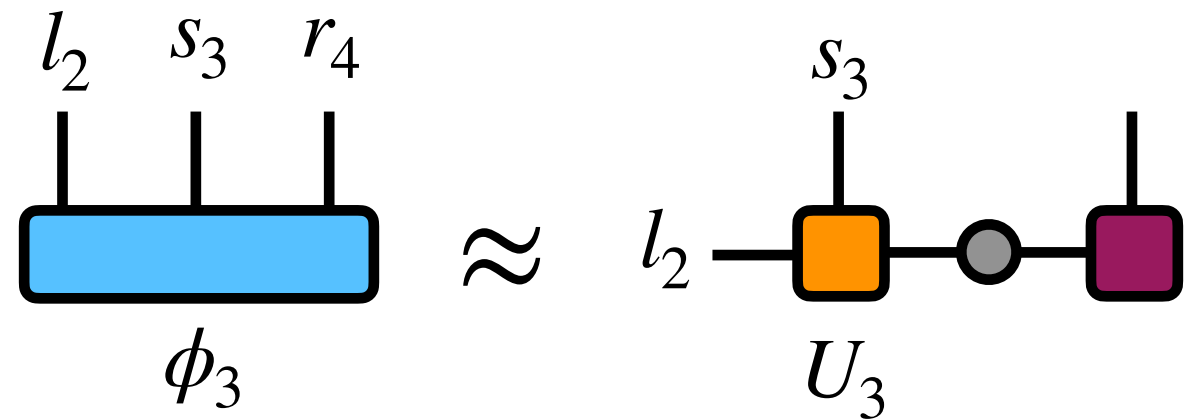
Recursively summing and sketching data continues process



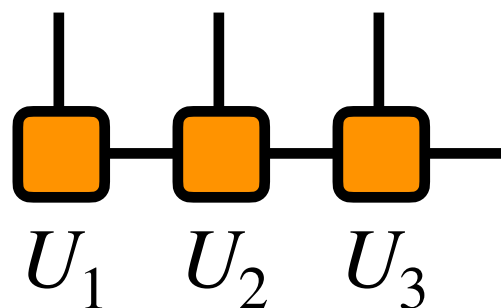
$\hat{p}$, sum over all data in training set
(product states / rank-1 tensors)

Tensor Network Machine Learning

SVD of reduced data tensor gives
second MPS tensor U_3



Save U_2 only



Tensor Network Machine Learning

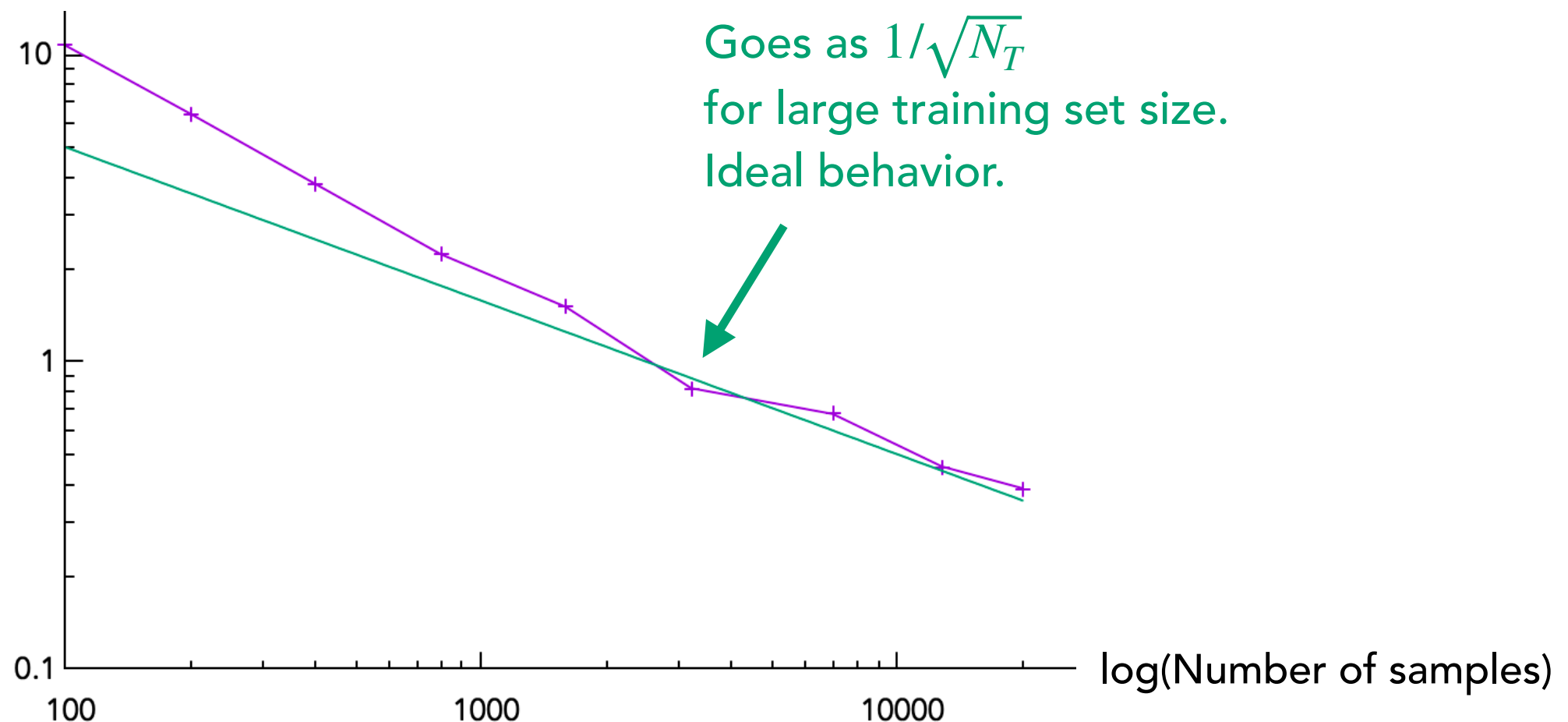
Recursive sketching able to reconstruct distribution of *disordered* Ising chain



Distance of
estimate from
true distribution

$$\frac{\|p - A(\hat{p})\|}{\|p\|}$$

log scale



Tensor Network Machine Learning



Ziang
Yu

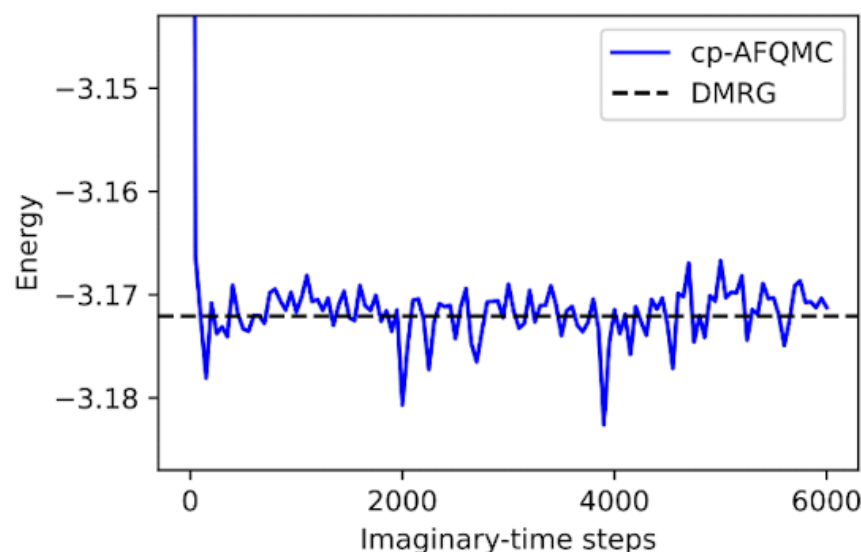


Shiwei
Zhang

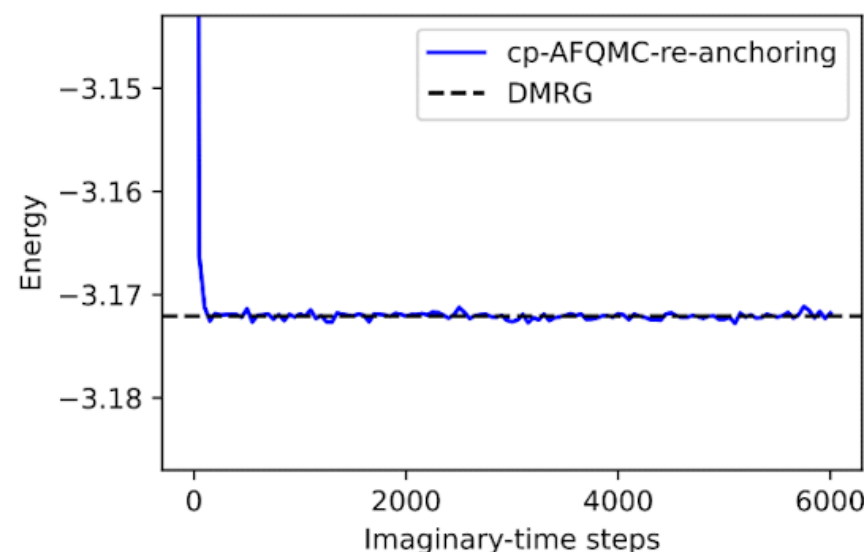


Yuehaw
Khoo

Recursive sketching recently used to
learn trial wavefunctions for AF-Quantum Monte Carlo



(a) Vanilla cp-AFQMC



(b) cp-AFQMC-re-anchoring

FIGURE 6. Energy convergence of (a) vanilla cp-AFQMC and (b) cp-AFQMC-re-anchoring for 11×11 system with open boundary at $g = 3$. The reference

Summary

Tensor networks can represent many types of functions, including functions of **continuous variables**

Powerful, interesting, and interpretable algorithms to **learn high-dimensional functions** as tensor networks

Up Next

- Hands on with tensorized functions
- **Later today: 2D and 3D tensor networks**

Bonus Slides

Tensor Networks versus Neural Networks

What's the difference?

Tensor networks, neural networks...

Sound similar...

Can do learning with both...

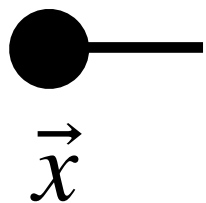
What's the difference?

Tensor notation can clarify

"neural diagram" notation

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

tensor diagram notation

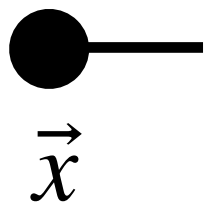


Tensor notation can clarify

"neural diagram" notation

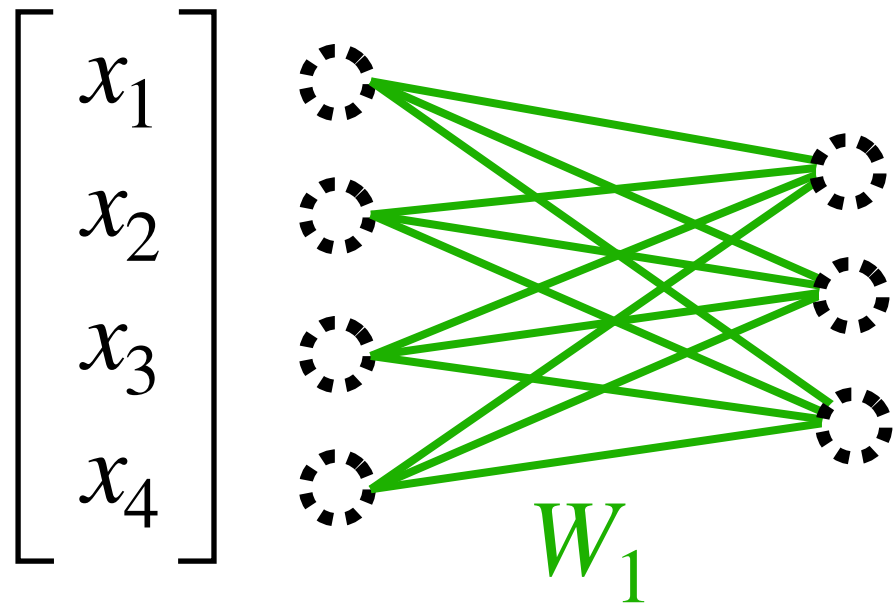
$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \begin{array}{c} \odot \\ \odot \\ \odot \\ \odot \end{array}$$

tensor diagram notation

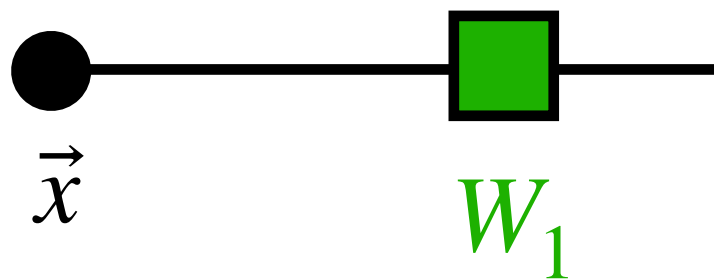


Tensor notation can clarify

"neural diagram" notation

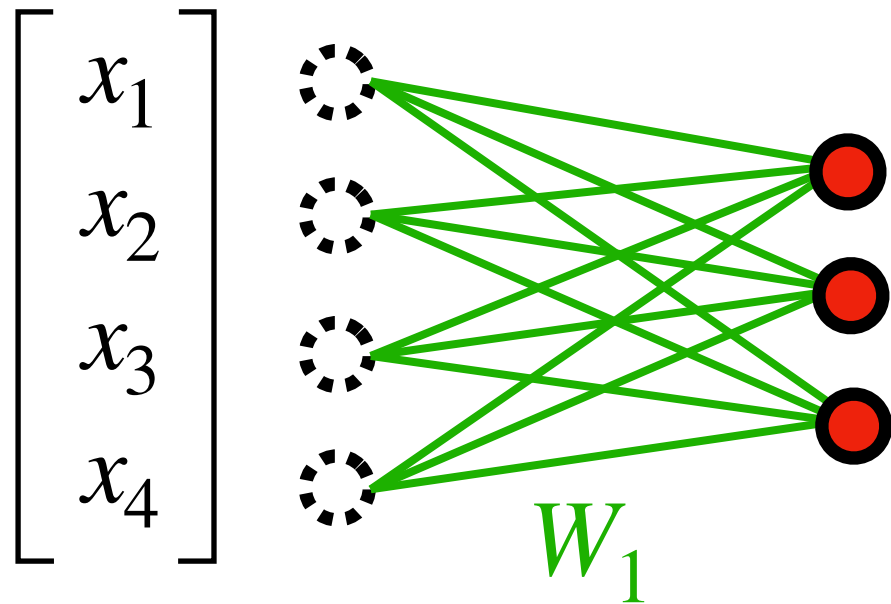


tensor diagram notation

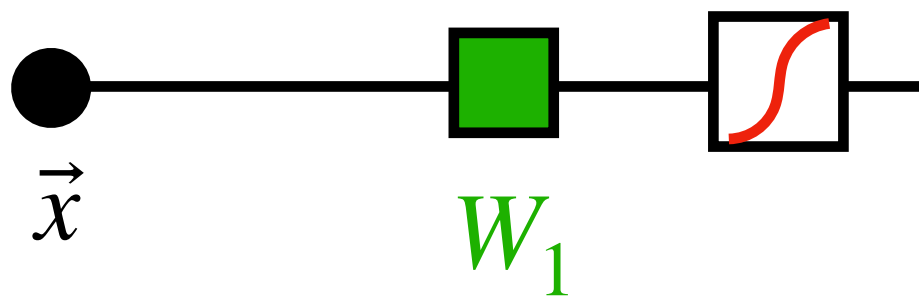


Tensor notation can clarify

"neural diagram" notation

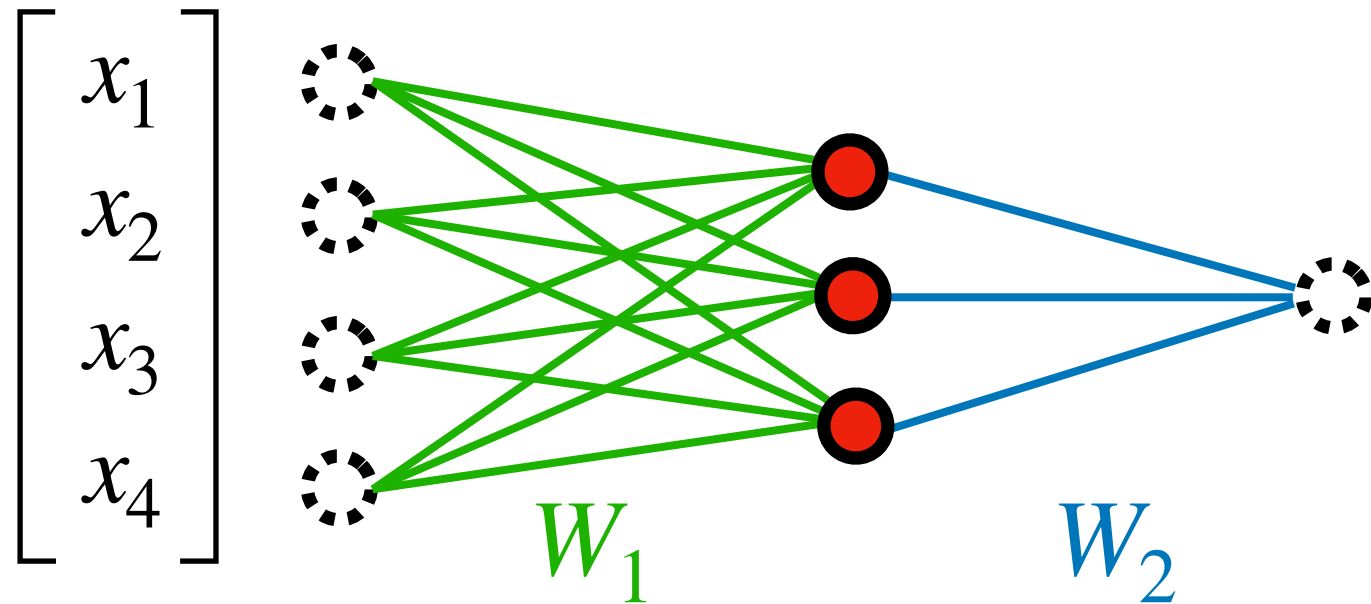


tensor diagram notation

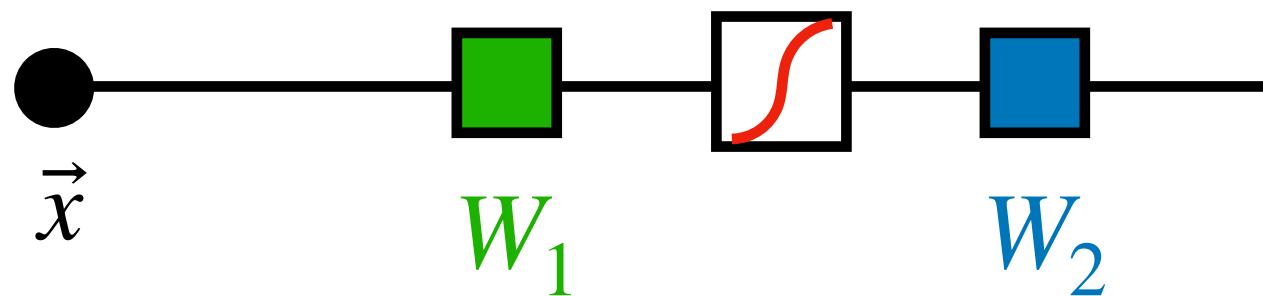


Tensor notation can clarify

"neural diagram" notation

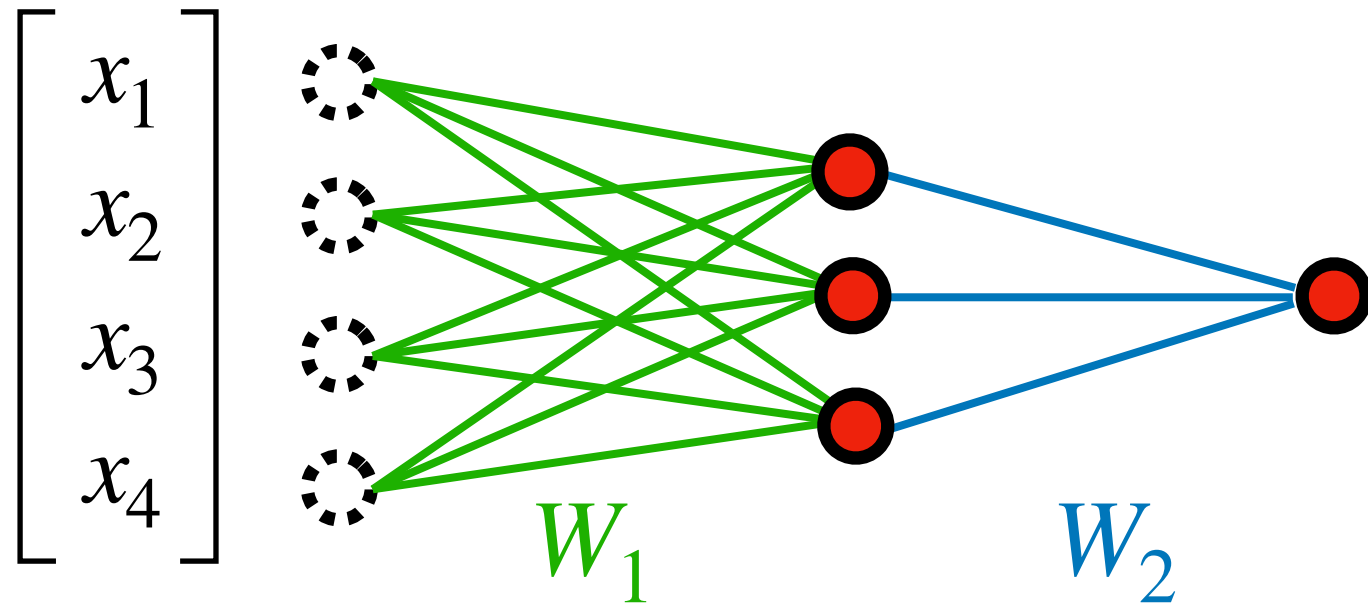


tensor diagram notation

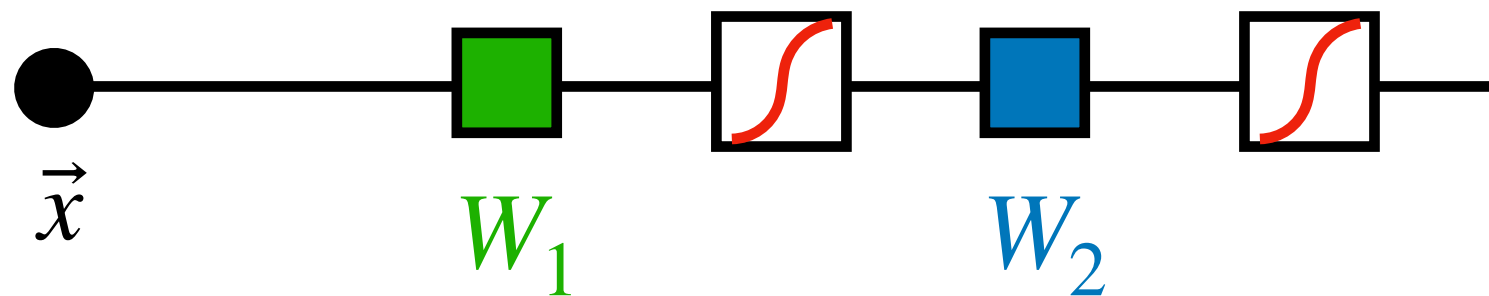


Tensor notation can clarify

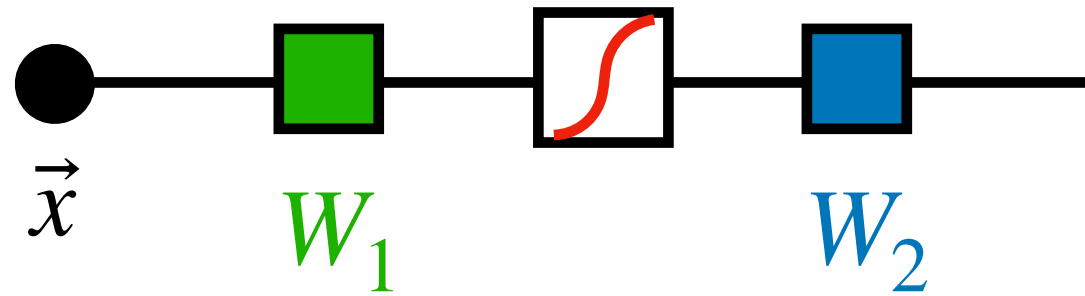
"neural diagram" notation



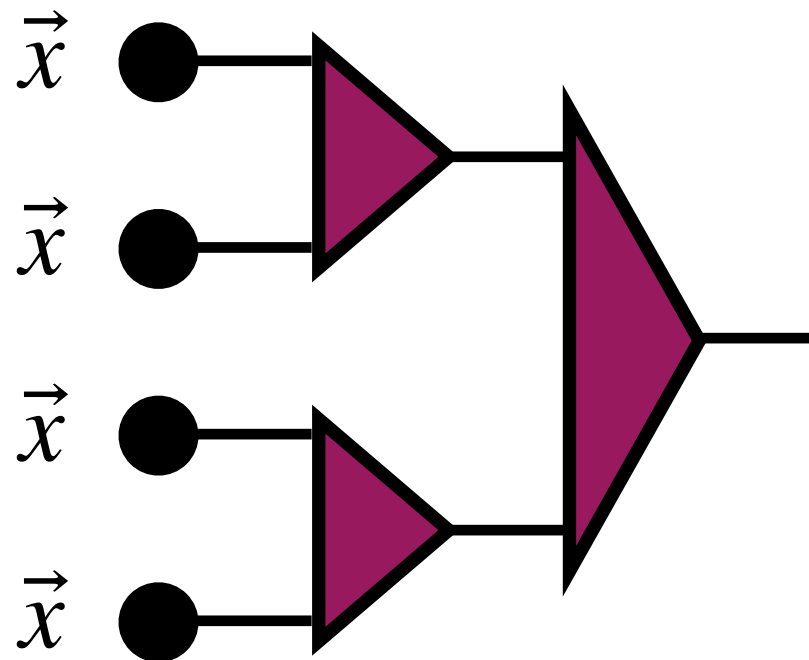
tensor diagram notation



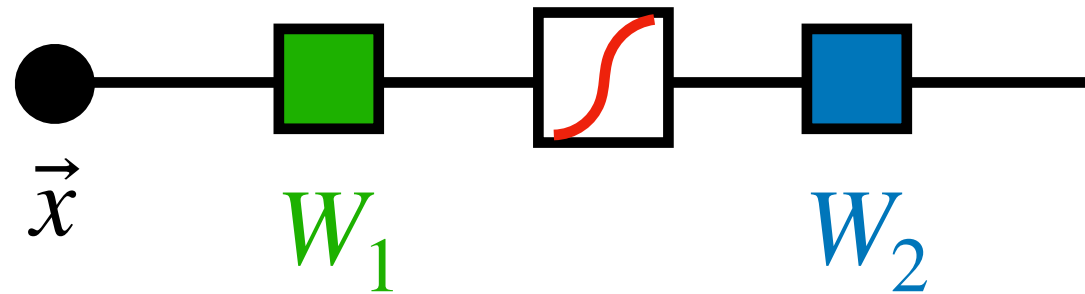
Neural network (feedforward, 2 layers)



Tensor network ('tree tensor network' with 'reuploading')



Neural network (feedforward, 2 layers)



Tensor network (MPS tensor network with 'feature map')

